Hands on Algorithm Acceleration using Heterogeneous Computing Systems

By

Mahgul Adnan (02-3-1-025-2019)

Soroush Javed Sulehri (02-3-1-070-2019)

Thesis submitted to the faculty of Engineering at PIEAS in partial fulfillment of requirements for the Degree of BS Electrical Engineering.

Department of Electrical Engineering,

Pakistan Institute of Engineering and Applied Sciences,

Nilore, Islamabad 45650, Pakistan

2023, June

Department of Electrical Engineering

Pakistan Institute of Engineering and Applied Sciences (PIEAS)

Nilore, Islamabad 45650, Pakistan

Declaration of Originality

We hereby declare that the work contained in this thesis and the intellectual content therein are the product of our own research. This thesis has not been previously published in any form nor does it contain any verbatim of the published resources which could be treated as infringement of the international copyright law. We also declare that we do understand the terms 'copyright' and 'plagiarism,' and that in case of any copyright violation or plagiarism found in this work, we will be held fully responsible for the consequences of any such violation.

Signature: _____

Name: Mahgul Adnan

Signature: _____

Name: Soroush Javed Sulehri

Date: June 19, 2023

Place: PIEAS

Certificate of Approval

This is to certify that the work contained in this thesis entitled

"Hands on Algorithm Acceleration using Heterogeneous Computing Systems "

carried out by

Mahgul Adnan (02-3-1-025-2019)

Soroush Javed Sulehri (02-3-1-070-2019),

It is fully adequate, in scope and quality, for the degree of BS Electrical Engineering from Pakistan Institute of Engineering and Applied Sciences (PIEAS).

Approved By:

Signature: _____	Signature: _____
Name: Dr Muhammad Aqil	Name: Muhammad Ali Akhtar
Academic Supervisor	Industrial supervisor (NESCOM)

Verified By:

Signature: _____

Head, Department of Electrical Engineering

Stamp:

To our parents and siblings, whose belief in our abilities never wavered.

Acknowledgements

To begin with, we would like to extend our deepest gratitude to our supervisor Sir Muhammad Ali Akhtar, and Co-supervisor Dr. Muhammad Aqil for guiding and supporting us throughout the project work, always showing patience, and sharing their knowledge. Their expertise in the field and willingness to share their knowledge have been invaluable. Their constructive criticism has always helped us refine our ideas and improve the overall quality of our work.

We would like to thank our parents, siblings, and friends for their unwavering support, love, and encouragement. Their constant motivation and willingness to lend a helping hand have always been the greatest support for us.

Contents

List of Figures **xii**

Abbreviations and Acronyms **xv**

Abstract **1**

1 Introduction **2**

1.1 Objectives 3

1.2 Motivation 4

1.3 Significance 5

1.4 Project Overview 6

1.5 Project Block Diagram 7

1.6 Thesis Outline 7

2 Literature Review **9**

2.1 Genome Sequencing 9

2.1.1 Human Genome 9

2.1.2 Genome Sequencing Process 11

2.1.3 Read Mapping 12

2.1.3.1 Indexing and Querying 13

2.1.3.2 Sequence Alignment 16

2.2 Algorithm Acceleration 18

2.2.1 Seed Filtering Techniques 18

2.2.2 Pre-Alignment Filtering 20

2.2.3 Choice of Algorithm 23
2.3 Sneaky Snake Algorithm 25
2.4 Cloud Based Read Mapping and Pre-Alignment Filtering 27

3 Theory and Background **29**
3.1 Heterogeneous Computing Sytems 29
3.1.1 Architecture of Heterogeneous Computing Systems 30
3.1.1.1 Single Instruction Stream-Single Data Stream (SISD) 31
3.1.1.2 Single Instruction Stream-Multiple Data Stream (SIMD) 31
3.1.1.3 Multiple Instruction Stream-Single Data Stream (MISD) 32
3.1.1.4 Multiple Instruction Stream-Multiple Data Stream (MIMD) 33
3.1.2 Heterogeneous Programming Languages 34
3.1.2.1 CUDA 34
3.1.2.2 OpenCL 35
3.1.3 Advantages of Heterogeneous Computing Systems 36
3.1.4 Problems of Heterogeneous Computing Systems 36
3.2 Sneaky Snake Algorithm 37
3.2.1 Approximate String Matching 38
3.2.2 Single Net Routing (SNR) Problem 38
3.2.3 Solving the Single Net Routing Problem 40
3.3 Cloud Computing 42
3.3.1 Service Models 43
3.3.2 Deployment models 45
3.3.3 Some Famous Cloud Providers 46

4 Methodology **48**

4.1 Research Design and Data Collection 48
4.1.1 Performance Measurement 49
4.2 Sneaky Snake Performance Measurement on CPU 50
4.2.1 Determine Execution Time 50
4.2.2 Determining CPU Utilization 50
4.2.3 Determining Memory Utilization 51
4.2.4 Determining Accuracy 51
4.2.5 Performance with Respect to Read Length 52
4.2.6 Performance with Respect to Edit Distance Threshold . . . 52
4.2.7 Performance with Respect to Thread Number 52
4.2.8 Procedure 52
4.3 Sneaky Snake Performance Measurement on FPGA 54
4.3.1 Determining Accuracy 54
4.3.2 Determining Resource Utilization 54
4.4 Sneaky Snake Performance Measurement on Cloud 55
4.5 Test Data Sets 55
4.6 Material Needed 56
4.7 Ethical Consideration 56
4.8 Analysis 57

5 CPU Implementation of Sneaky Snake Algorithm 58

5.1 Design and Implementation 58
5.1.1 Main Function 59
5.1.2 Sneaky Snake 59
5.1.3 Input and Output 60
5.1.4 User Interface 61
5.1.5 Pseudocode 61
5.2 Parallelism 64
5.2.1 OpenMP 65
5.2.2 Implementation Considerations 66

5.3 Hardware Used 67
5.4 Software Requirements 67

6 FPGA Implementation of Sneaky Snake Algorithm 69

6.1 Design and Architecture 69
6.1.1 Design Philosophy 72
6.1.2 Block Diagram 73
6.2 Theoretical Basis 74
6.2.1 Proposed Solution 76
6.2.2 Mathematical Foundation 78
6.2.2.1 4-Bit Leading Zero Counter 78
6.2.2.2 Re-purposing the 8bit LZC to 4bit LZC 82
6.2.2.3 Hierarchical Comparator 83
6.3 Implementation 83
6.3.1 Hardware and Software 84
6.3.2 Implementation Process 84
6.3.3 Shifted Hamming Distance Map/ Chip Maze 85
6.3.4 Leading Zero Counter 4 bit 86
6.3.5 Leading Zero Counter 8 bit 86
6.3.6 Comparator 3 bit 88
6.3.7 Longest Escape Segment 88
6.3.8 8-bit Sneaky Snake Module 89
6.3.9 Top Sneaky Snake Module 89
6.3.10 Block Read Only Memory 89
6.3.11 Clock 90
6.3.12 General Purpose Input Output (GPIO) 90
6.4 Data Modification and Generation 91
6.5 Hardware Used 91
6.5.1 Port Mapping 92
6.6 Software Used 92

6.7 Limitations 92

7 Cloud Implementation of Sneaky Snake Algorithm 94

7.1 Service Model 94

7.2 Deployment Model 94

7.3 Virtual Machine 95

7.3.1 Specifications 96

7.3.2 Communication and Connection 96

7.4 Implementation of Sneaky Snake on Microsoft Azure 97

7.5 Cost and Pricing 98

8 Results 99

8.1 CPU Implementation 99

8.1.1 Function and Output 99

8.1.2 Performance 101

8.1.2.1 Accuracy 101

8.1.2.2 Execution Time 102

8.1.2.3 CPU Utilization 106

8.1.2.4 Memory Utilization 107

8.1.2.5 Bottleneck and Problems 108

8.2 FPGA Implementation 108

8.2.1 Outputs 109

8.2.1.1 Simulation Results 109

8.2.1.2 Results of Hardware Implementation 114

8.3 Cloud Implementation 118

8.3.1 Function and Output 118

8.3.2 Performance 119

8.3.2.1 Accuracy 120

8.3.2.2 Execution Time 121

8.3.3 Comparison with CPU Implementation 122

9 Discussion and Conclusion **124**

9.1 Discussion 124

9.1.1 CPU Implementation 124

9.1.2 FPGA Implementation 127

9.2 Cloud Based Implementation 131

9.3 Conclusion 132

9.4 Future Works 133

References **134**

List of Figures

1.1 An Example of Heterogenous Computing System 3
1.2 Project Block Diagram 7

2.1 Double-Helix Structure Watson et al. 11
2.2 Read Formation 12
2.3 Brute Force Method 13
2.4 Read Mapping Process 14
2.5 Indexing Hashing / Suffix tree adapted from 15
2.6 Hashing 15
2.7 File Formats during Genome Sequencing 16
2.8 Pre-Alignment Filtering adapted from Alser et al. 21
2.9 Pigeon Hole Principle 22
2.10 Chip Maze SNR Problem 25

3.1 SISD Type of Parallelism 31
3.2 SIMD Type of Parallelism 32
3.3 MISD Type of Parallelism 33
3.4 MIMD Type of Parallelism 33
3.5 Left: Shows the chip maze for E=3, Right: Shows the Chip maze calculated using the equations 3.1-3.4 41
3.6 Calculating chip maze on the fly 42

5.1 Functions Hierarchy of Sneaky Snake CPU Implementation 60
5.2 Output Windows User Interface 61

5.3 Visual Studio Enterprise 68
5.4 Visual Studio Enterprise Performance Profiler 68

6.1 Sneaky Snake Architecture FPGA 72
6.2 Block Diagram Sneaky Snake Algorithm 73
6.3 Chip Maze Schematic 86
6.4 4 Bit Leading Zero Counter 87
6.5 8-Bit Leading Zero Counter 87
6.6 3 Bit Comparator 88
6.7 Complete Schematic 90
6.8 Xilinx Virtex VC-707 FPGA Evaluation Kit 91

7.1 Microsoft Azure VM Dashboard 98

8.1 Sneaky Snake Output Data Set: 100_bp_1, E = 5, Threads = 2 . . . 99
8.2 Sneaky Snake Output Data Set: 100_bp_1, E = 10, Threads = 2 . . 100
8.3 Sneaky Snake Output Data Set: 100_bp_1, E = 10, Threads = 2 . . 100
8.4 Sneaky Snake CPU Utilization Data Set = 100_bp_1, E = 200 . . . 100
8.5 Sneaky Snake CPU Utilization Data Set = 100_bp_1, E = 2000 . . . 101
8.6 Sneaky Snake CPU Utilization Data Set = 100_bp_1, E = 20000 . . 101
8.7 Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_1) 103
8.8 Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_2) 104
8.9 Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: long_1) 105
8.10 CPU Utilization vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_1) 106
8.11 CPU Utilization vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_1) 107
8.12 CPU Utilization impact shown due to strncpy_s operation 108

8.13 CPU Utilization impact shown due to strncpy_s operation 108
8.14 Simulation result of a 4-bit leading zero counter 109
8.15 Simulation result of an 8-bit leading zero counter 110
8.16 Simulation result of a 3-bit comparator 110
8.17 Simulation result of the longest escape segment module 111
8.18 Simulation result printed on the serial monitor 111
8.19 Simulation result of the designed Chipmaze 111
8.20 Simulation result of an 8-bit Sneaky Snake module 112
8.21 Simulation result printed on a serial monitor 112
8.22 Simulation result of the Sneaky Snake module 113
8.23 Simulation result of the final Sneaky Snake module 113
8.24 Clock Summary 113
8.25 Design Timing 114
8.26 Resources utilized 114
8.27 Power Summary 114
8.28 FPGA board being programmed 115
8.29 Only one read accepted (COUNT=1) 115
8.30 Two reads accepted (COUNT=2) 116
8.31 Read rejected (COUNT=0) 116
8.32 Read rejected again (COUNT=0) 117
8.33 Reset button pressed 117
8.34 SHH based connection to Virtual Machine 118
8.35 Output Edit Distance = 5 Dataset = 100_bp_1 119
8.36 Output Edit Distance = 5000 Dataset = long_1 119
8.37 Parallel Execution of Sneaky Snake on Cloud 120
8.38 100_bp_1 E = 20 Output 121
8.39 100_bp_1 E = 10 Output 121
8.40 100_bp_1 E = 5 Output 121

Abbreviations and Acronyms

AI	Artificial Intelligence
FPGA	Field Programmable Gated Arrays
CPU	Central Processing Unit
GPU	Graphical Processing Unit
SISD	Single Instruction Stream-Single Data Stream
SIMD	Single Instruction Stream-Multiple Data Stream
MISD	Multiple Instruction Stream-Single Data Stream
MIMD	Multiple Instruction Stream-Multiple Data Stream
DNA	Deoxyribonucleic Acid
ASM	Approximate String Matching
SNR	Single Net Routing
DP	Dynamic Programming
VM	Virtual Machine
BROM	Block Read Only Memory
CLI	Command Line Interface

Abstract

Heterogeneous systems became the need of the hour when a sudden rise in computationally extensive applications like artificial intelligence, Machine Learning, etc. was observed. The objective of this thesis was to explore heterogeneous computing systems by using them to accelerate the performance of an application. The application, the pre-alignment filtering technique, which is part of the genome sequencing process was accelerated using Heterogeneous Systems. The most efficient algorithm for a pre-alignment filter, termed Sneaky Snake was implemented on a CPU, FPGA, and cloud. Complete code profiling was done using Visual Studio Code and the efficiency of the program was observed and deduced from the results. For FPGA, the implementation was done on Xilinx VC707, and Microsoft Azure was used to implement the algorithm on the cloud. Cloud implementation is the novelty of this project since the pre-alignment filter has never been implemented on cloud. The implementation involving only a single processor (4 cores) was done on the CPU, along with that involving all the cores of the CPU. Then parameters, such as Edit Distance Threshold, Read Length, and number of reads were changed and the results were observed and compared. To exploit the parallel processing capability of FPGA, the reads were all broken down into pieces so that simultaneous processing might lead to less execution time. The results obtained and shown in this thesis matched the results available from previous implementations. This thesis contains a complete discussion of the theory of Heterogeneous Systems, Genome Sequencing, the Pre-alignment filtering technique called Sneaky Snake, and the implementation of Sneaky Snake on CPU, FPGA, and cloud.

Chapter 1

Introduction

Human beings are at the dawn of the heterogeneous era [1]. The world is progressing faster than a human being can even fathom. With the huge amounts of technological advancements and the introduction of fields such as Artificial Intelligence (AI), machine learning, unmanned driving, industrial simulation, etc., the need for more computing power, extra storage, and faster processing has increased significantly. The normal CPU implementation of such applications has become inefficient because the bottleneck problem is encountered more frequently, which includes low parallelism, high latency, and insufficient bandwidth. Furthermore, Moore's Law has also lost its efficacy in recent years as silicon devices have approached their physical limits and economic expenses have increased. Processor design is constrained by Moore's Law and the power wall together. For this reason, the idea of heterogeneous computing systems came into being.

Heterogeneous computing systems involve the integration of different processors together so that the computational speed can be increased by allocating tasks to each processor based on its specialty. It is a hybrid system that performs specialized calculations by combining several computing units, such as the CPU, GPU, FPGA, DSP, ASIC etc. with diverse types of instruction sets and architectures. These systems are readily being used to accelerate many different applications including data compression, image recognition, machine learning etc.

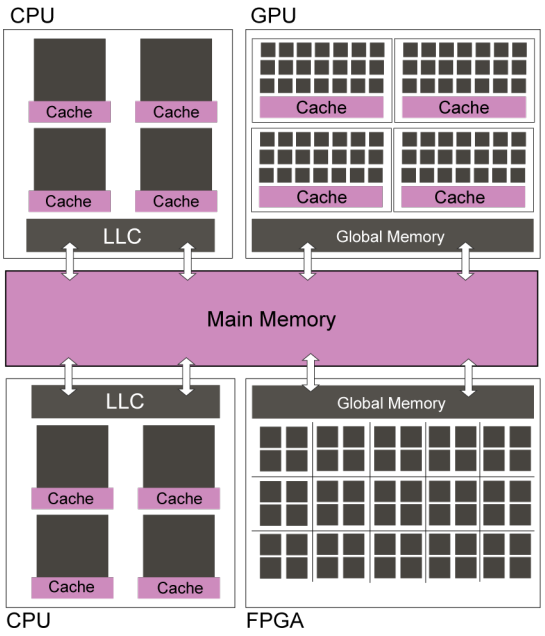

Figure 1.1: An Example of Heterogenous Computing System

In today's world, the field of bioinformatics is growing considerably and showing great advancements. This thesis discusses the use of a heterogeneous computing system in the acceleration of applications part of the bioinformatics field. More specifically, the thesis discusses the acceleration of the Pre-Alignment Filtering technique, discussed in Chapter 2 of the thesis, used in Genome Sequencing. Chapter 2 also gives a detailed account of genome sequencing and its use in today's world.

1.1 Objectives

The main objective of this project is to explore the implementation of Sneaky Snake algorithm (a pre-alignment filter used in the process of genome sequencing) developed by researchers at ETH Zurich on various computing platforms helping profile their performance for a heterogeneous computing system. The algorithm will be implemented on a CPU, cloud, and FPGA after which performance profiling will be conducted and the performance of the algorithm on each type of processor will be observed.

Bioinformatics is the application of tools of computation and analysis to capture and interpret the biological data [2]. Until now, genome sequencing is the biggest achievement in the field of bioinformatics, which will lead to studying mutations in genes associated with disorders and diseases and come up with drugs to target these diseases and formulate a cure. The world was greatly affected by the pandemic COVID-19 which halted life. Diagnosis of COVID wasn't easy and the cure came out after a long time. With a faster, more accessible genome sequencing algorithm at hand, diagnosis of such diseases will become much easier. Thus, the final objective of this project is to utilize cloud computing to make the pre-alignment filtering algorithm accessible to people all around the world. This would show that if the entire genome sequencing algorithm is to be placed on the cloud, the entire process becomes more hands-on and easier, leading to quick analysis and diagnosis.

1.2 Motivation

Owing to the advantages of heterogeneous computing systems, great advancements have been seen in their development over the years. Also, because new fields and their associated applications have become more power, energy, and speed consuming that the shift towards heterogeneous computing systems was inevitable. People now have different needs for storage space in the Internet sector because of the increasing popularity of information technology and the rapid growth in data volume. The need for computing power and the variety of application scenarios has both expanded quickly as a result of computer applications like artificial intelligence and big data. The focus of research has shifted to heterogeneous computing. This acts as the main motivation behind the project.

The application which is to be implemented on a heterogeneous computing system belongs to the field of bioinformatics. Pre-alignment filtering algorithm, which is to be accelerated using a heterogeneous computing system, is a part of the process of genome sequencing. The Human Genome Project invented whole

genome sequencing, which allows us to read a person's unique genome and, among other things, spot variances from the typical human genome. Such variations (mutations) are frequently related to disorders and diseases. These variations can also be linked to other aspects like disease resistance or sensitivity to environmental disturbances like sunshine or exercise. Identification of these variations can lead to the formulation of cures and can help diagnose and cure potentially fatal diseases. Since, this is also an emerging field, and the world will slowly move towards diagnosis through genome analysis using heterogeneous computing systems (another emerging technology) with bioinformatics means another innovation on its own.

Finally, the project will end by using cloud computing to make the algorithm more hands-on and accessible to people all over the world. This will lead to mobile genomics and analysis of the genome will become easier and less time-consuming. Thus, the above-stated reasons are the main motivation behind undertaking this project.

1.3 Significance

The significance of this project can be deduced from the following points.

1. Human Beings are at the dawn of the heterogeneous era [1]. The world will eventually shift towards using heterogeneous computing systems.

2. The need for heterogeneous computing systems was strongly felt after the introduction of fields such as artificial intelligence, machine learning, Unmanned vehicles, etc, and since these fields are of top interest to researchers all around the world, heterogeneous computing systems will be used more frequently, and more advancements will be observed.

3. Moreover, this emerging technology will bring new industries and new jobs [1]. This will result in promoting economic growth, social progress, and even the welfare of all human resources.

4. Similarly, genome sequencing is also an emerging field. It has undergone massive expansion in the past 10 years [2]. This field will continue to flourish since researchers are continuously trying to come up with better diagnosis techniques and cures for potentially fatal diseases.

5. Genome sequencing, has allowed researchers to transform their knowledge about the biology and study complex diseases such as cancer, autoimmune disorders, etc. This has led to identifying the changes occurring in the genome for these diseases and formulating cures for them accordingly

6. Future contributions to the functional understanding of the human genome, which will improve the identification of pharmacological targets and enable individualized treatment, are among the opportunities in the field of bioinformatics [2].

1.4 Project Overview

The project starts with the implementation of the SneakySnake algorithm on the CPU i.e., on the PC. Performance profiling was done, and the computational speed was observed. Then, implementation was done using FPGA and for this Xilinx Virtex 7 VC707 was used. The first two steps of local implementation were the CPU and FPGA implementation. Based on the results obtained, the advantages of using heterogeneous computing systems were deduced.

Once the local implementation was completed, CPU based application was developed on Microsoft Azure Cloud to test the capabilities of this algorithm in the cloud environment.

1.5 Project Block Diagram

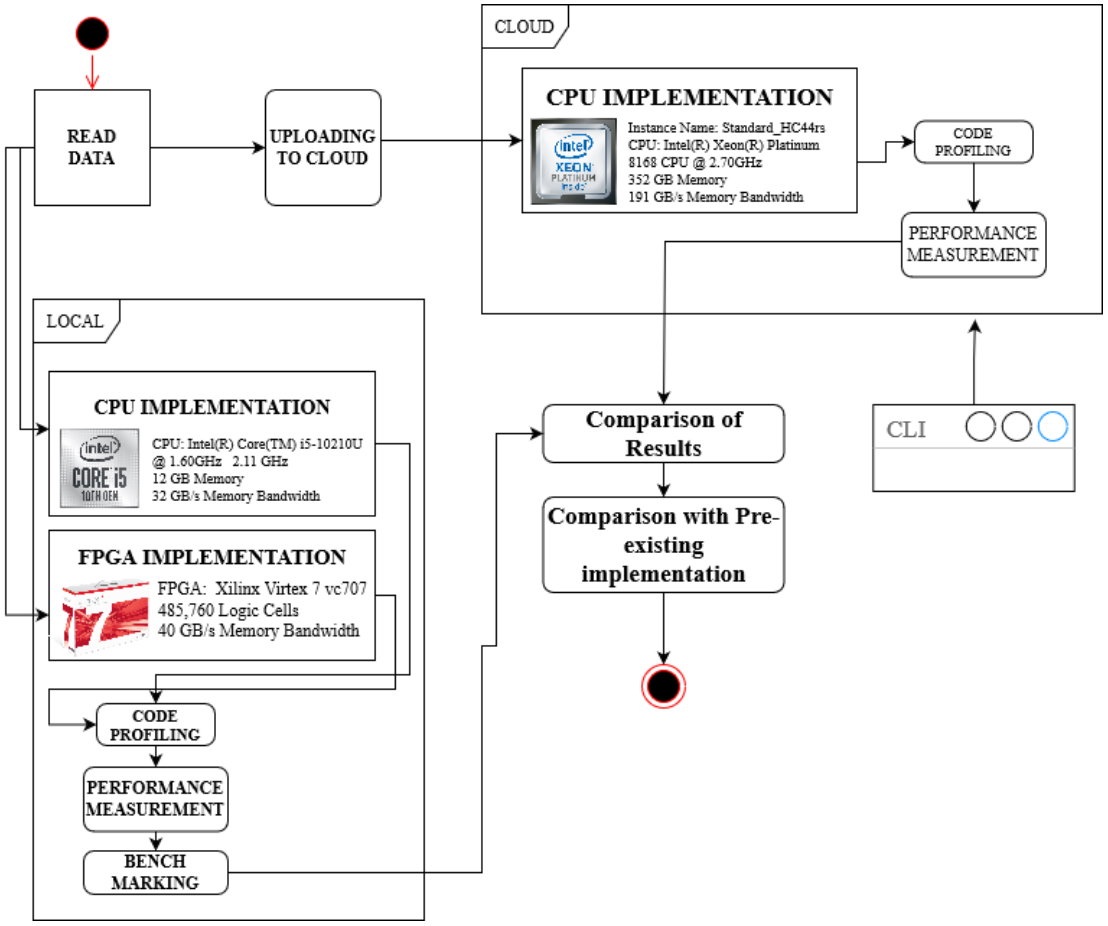

Figure 1.2: Project Block Diagram

1.6 Thesis Outline

Chapter 1 introduced the project. It includes the objectives of the project and the motivation behind the work done in this project. It also discusses the significance of the project and gives a project overview.

Chapter 2 will discuss the literature review. It will give detailed explanations of what genome sequencing is and the entire genome sequencing process. Furthermore, it will talk about the pre-alignment filtering technique and its associated algorithms, which is the main aim of this project. Finally, it discusses the implementation of the application on the cloud using the concept of cloud computing.

Chapter 3, the theory section, will give an in-depth account of heterogeneous computing systems. Their background, architecture, programming languages, and advantages will be discussed. Along with this, chapter 3 will also discuss the specific algorithm which will be used to implement the pre-alignment filter. Thus, the SneakySnake algorithm will be discussed.

Chapter 4, the methodology section, will discuss in detail the methodology used to implement the pre-alignment filter. It will talk about the step-by-step procedure of the implementation along with the means to measure the performance of the three implementations to be done in the project.

Chapter 5 will discuss the implementation on the CPU. It will include the complete design, pseudo codes, hardware, and software requirements.

Chapter 6 will discuss the implementations of the Sneaky Snake algorithm on the FPGA including the details about design, algorithm, architecture, hardware, and software requirements.

Chapter 7 will discuss the cloud-based implementation of Sneaky Snake including the deployment model, service model, process, and cost.

Chapter 8 will discuss the results obtained from the CPU implementation, FPGA implementation, and the cloud-based implementation of the Sneaky Snake algorithm.

Chapter 9 will discuss the complete report and make the final remarks.

Chapter 2

Literature Review

This chapter introduces and reviews the existing literature present on Genome Sequencing Algorithms with a focus on Pre-Alignment filtering. In this chapter, concepts relevant to genome sequencing, genome alignment, pre-alignment filtering and Sneaky Snake are outlined and explained.

2.1 Genome Sequencing

Genome Sequencing is the process of making sense out of the enigma of DNA. In technical terms, it refers to determining the genome of an organism in its entirety or near to that. It enables researchers to determine the intricacies hidden behind the coded sequence that living cells contain. Before diving deep into the process of genome sequencing it is of utmost importance to understand what exactly the human genome is.

2.1.1 Human Genome

The intricacies of human biology have long fascinated scientists across the world. A perfect harmony exists between the bodily processes and is governed by the information encoded in our genome. The human genome is the complete set of all possible nucleic acid sequences for homo sapiens, encoded as DNA within 23 chromosome pairs. The human genome consists of three billion base pairs,

forming one long polymer, carrying all the information about the species, and is classified as coding and non-coding DNA. Genes are normally referred to as protein-coding DNA. They are responsible for the formation of proteins inside the body having their own unique sequence. Inside the nucleus of a cell are found chromosomes which in turn are made from genes containing the DNA. The difference between chromosomes, genes, and genomes is a very important distinction to understand. Gene is a short segment of encoded information that is responsible for the formation of proteins. In order to make sure the information in genes is kept intact they are coiled into structures called chromosomes. All the possible combinations and sequences of genes and other DNA form the whole genome of a specie.

The DNA (deoxyribonucleic acid) was discovered to have a Double-Helical structure by Watson et al [3]. in 1953 as represented in Figure 2.1. The long chains of DNA store the information using four base pairs: Adenine(A), Thymine(T), Guanine (G), and Cytosine (C). In the double-helical complementary base pairs (A-T and G-C) are combined in chains as shown in Figure 2.1 to form the DNA molecule. The organizational sequence and structure along with chemical modifications and mutations are responsible for the expression of information inside the genome and in addition provides genome the ability to replicate and repair. The human genome has a very interesting property. No two humans on Earth share the exact same genomic sequence. However, except for minor differences a lot of the genome among species is the same and changes in the genome can reflect information about the relationship between population groups of humans, the origin of the human species, and health propensities and risks. Therefore, the better genome is understood the more can be unveiled about the human species. This brings out the need to sequence the genome.

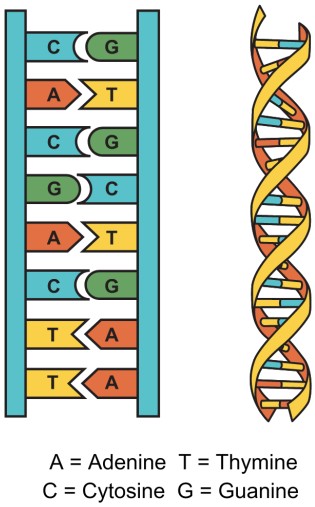

Figure 2.1: Double-Helix Structure Watson et al.

2.1.2 Genome Sequencing Process

DNA contains such vital information that its understanding can unveil the working of human biology. In order to compare, analyze and understand the information present in the DNA, sequencing the DNA becomes a necessity. The process of sequencing starts with taking the cells from the blood and reading their constituents. This is a biological process. DNA sequencing is a process of determining the sequence of nucleotides inside a piece of DNA. Today many technologies exist to take the strand of DNA and provide the sequence as the output. The problem however is that at once it is not possible for the sequencer to deal with the whole DNA. There exists no machine that can take the whole DNA as input and provide a whole output. Thus, the DNA is broken into pieces and these pieces are sequenced to be storing them as **READS**. The lengths of these reads depend on the technique and sequencer used. They can be short (700-900 base pairs) or long (10,000 base pairs). Figure 2.2 shows the general process of DNA sequencing. As the sequencer only outputs short sequences the job remains to fit them into a complete long chain. The process is referred to as genome assembly. To assemble the genome faster a reference is needed. Understanding this need, in 1990, the

National Institute of Health and the US Department of Energy collaborated on the Human Genome Project with an aim to sequence the whole genome. Completed in 2003, Human Genome Project yielded a reference genome accurately sequenced. This reference is used today to accelerate the sequence alignment process and thus in turn accelerate the sequencing process. While we do have a reference to compare today's developments in sequencing have been made as well. The process of sequencing was once long and expensive, today, thanks to next-generation sequencing and the existence of high throughput sequencers the attainment of reads, is fast and cheap.

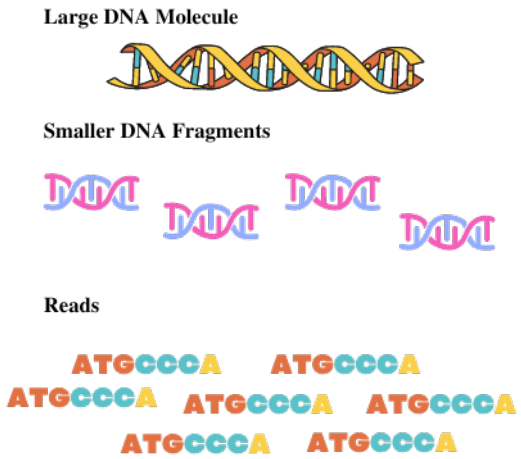

Figure 2.2: Read Formation

With the presence of a reference genome and the attainment of reads from the sequencer, the next part of genome sequencing is referred to as read mapping.

2.1.3 Read Mapping

As mentioned, read mapping is the process of taking the reads from the sequencer and mapping them onto the reference genome obtained through the Human Genome Project. The goal of read mapping is taking a read and locating all the possible subsequences of a reference genome allowing a certain edit distance

Figure 2.3: Brute Force Method

threshold E. The human genome can be downloaded as.FASTQ file while the reads are found in.FASTQ file. The simplest method of mapping a read is using a brute force-based algorithm. The read is taken as a frame and then compared with the whole reference genome until a certain match is found as shown in Figure 2.3

However, the brute force algorithm is very expensive mainly due to it taking a long time. The time complexity of the algorithm can be represented as:

$$O(m^2kn) \tag{2.1}$$

Where m is the read length, k is the number of reads and n is the length of the reference genome. This process can be made efficient using certain techniques and processes to reduce the complexity of both time and space. The process of read mapping involves the following processes:

1. Indexing the reference genome
2. Querying the read data
3. Read Alignment

The steps during a general read mapping process are visualized in Figure 2.4

2.1.3.1 Indexing and Querying

Before moving toward the mapping process. The reference genome needs to have an index. The index allows the algorithm implemented later to rapidly find the short sequences embedded in the reference genome that can be then used to compare them to the short reads obtained from sequencing the DNA. The process of

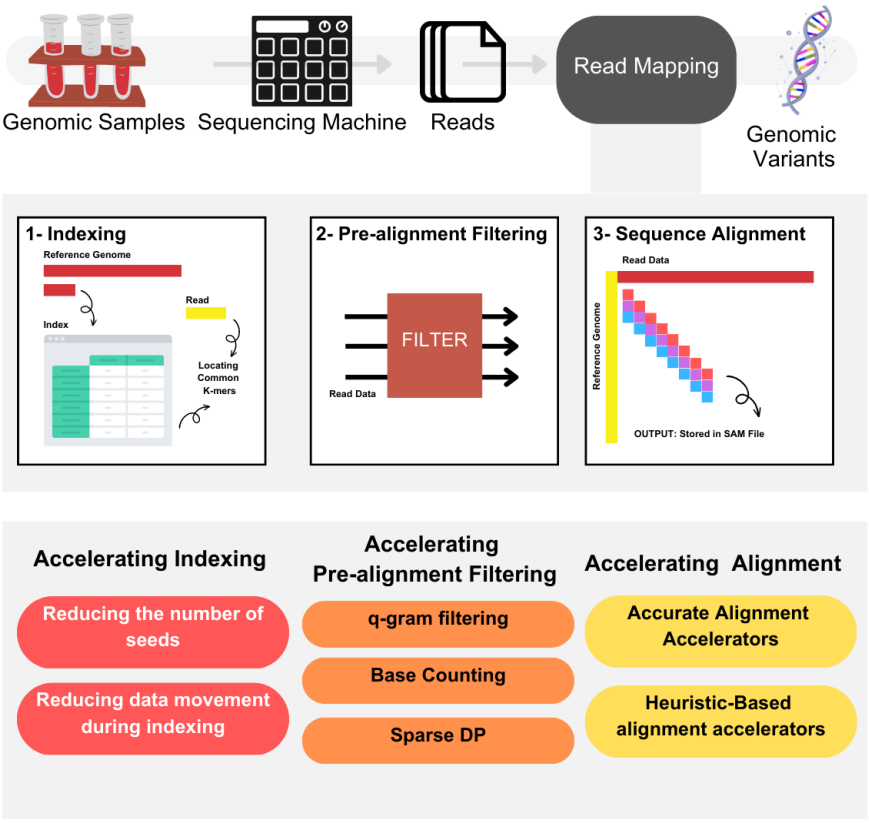

Figure 2.4: Read Mapping Process

indexing a genome is similar to building an index for a book. K-mers are substrings of length k which are extracted from the DNA sequence. These K-mers are referred to as seeds. Seeds are extracted from the reference genome and then all these possible occurrences are stored in a Hash Table.

The process is shown in Figure 2.5. Hashing is the most used method for indexing the reference genome. According to Alser et al., [4] 60.8% of read mapping tools use hashing as their primary algorithm for research, and is also the most popular method most of the read aligners can handle. Indexing based on hashing is easy to implement, can search for exact matches, has a large index size, and takes a small amount of time [4].

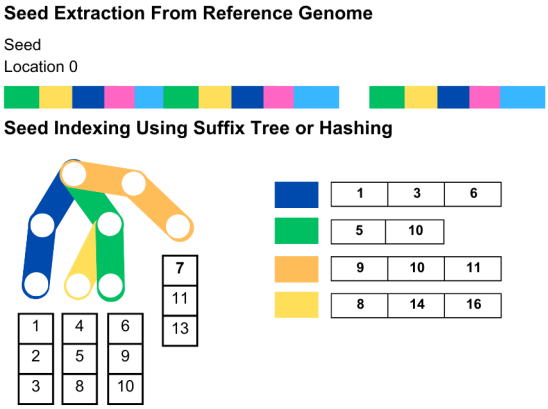

Figure 2.5: Indexing Hashing / Suffix tree adapted from

Once the genomic seeds are produced and the hash map is built, the algorithm further acts to extract the seeds from each of the reads using a similar process and uses them as a key to query the hash table-based index. The location of all the read seeds in the reference genome is returned as the output.

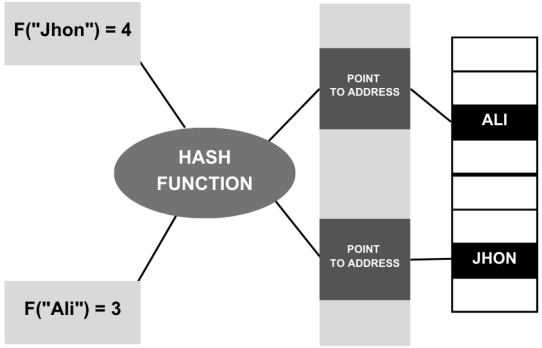

Figure 2.6: Hashing

The second most common method used for indexing is suffix-tree-based indexing (Figure 4) a generally faster approach used by 36.5% of the read aligner tools [4]. The suffix tree-based method is implemented using a suffix tree where different branches represent various suffixes of the genome, they can handle both

exact and inexact matches. The most common use of the said technique is seen in Burrows-Wheeler Transform (BWT) [5]. However, the quality of the method employing this technique degrades as the error rate increases or the genetic difference between the read and the reference becomes more likely to occur [6].

2.1.3.2 Sequence Alignment

Once the data is indexed the next step towards genome sequencing is sequence alignment. The read data and indexed reference genome are used to map the reads onto possible positions in the genome. Sequence alignment can be more accurately defined as the process of arranging the sequence obtained from a .FASTQ file. Figure 2.7 shows various file formats during various steps of genome sequencing

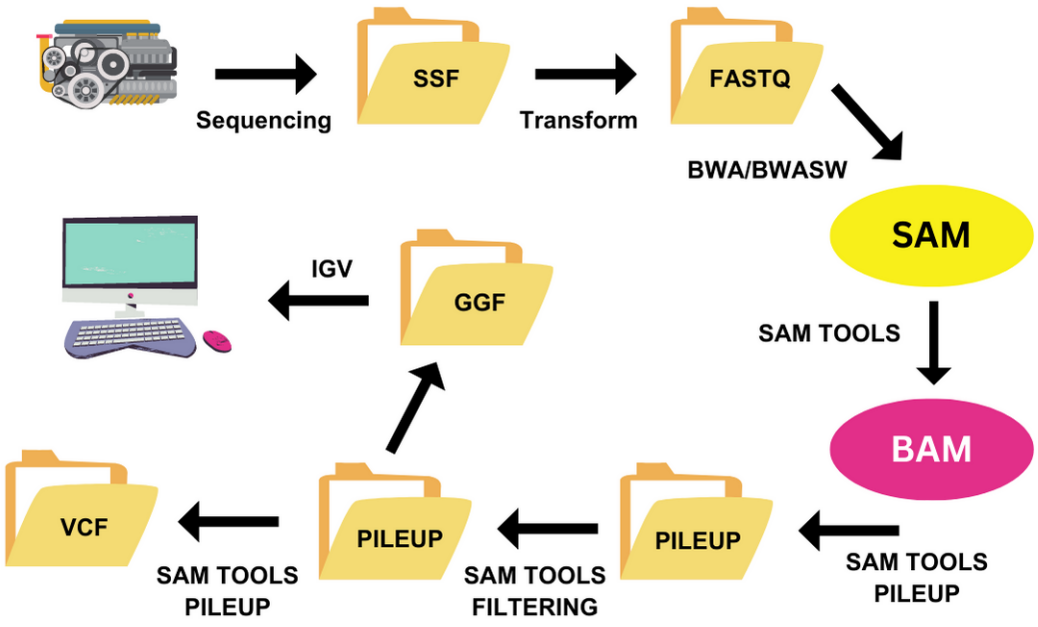

Figure 2.7: File Formats during Genome Sequencing

Once the data is arranged verification also needs to be performed in order to make sure that the match is accurate and not a result of some error. The job of a sequence aligner is to check whether the sequencing pairs that are identified by the indexing and querying are actually similar. This is a computational problem known as approximate string matching (ASM). The comparison between the read

and the reference string needs to be done in the shortest possible time and yield an output that is accurate. The sequence alignment's output is dependent on the edit threshold. **Edit threshold (E)** is defined as the minimum number of edits that are needed to make the read sequence match the reference sequence exactly the same. Many algorithms exist for sequence alignment that can be mainly classified into three main categories: Hashing-based methods, suffix-tree-based methods and merge sort-based methods.

Algorithms based on hash tables started off with BLAST [7, 8] which acts as the main concept behind the modern hash table-based algorithms as well. BLAST records the position of each K-mer where K-mer acts as the key when querying. BLAST joins the seeds without gaps and uses the Smith-Wasterman algorithm to refine the results [9, 10]. To date, the Smith-Wasterman algorithm remains the most popular hashing-based algorithm [4]. The second most popular alignment algorithm is called bfhamming distance algorithm bringing forth a bit-wise comparison between two strings. Both of these methods fall under the category of pairwise alignment. Discussion of modern alignment algorithms extends beyond the scope of this thesis. The algorithms of pairwise alignment are based on dynamic programming [11] involving a recursive structure to solve the problem.

Important consideration needs to be placed on the fact that many challenges are faced in the process of read mapping due to the nature of the problem. Read mapping needs to find multiple mappings for each of the reads. It should be able to tolerate variances and errors and need to work fast and on bidirectional strands [4]. Read mapping is slowed down by multiple reasons. The main reason is that the data being processed has to be stored far from the memory. The limitations imposed by the bandwidth of the data movement slow down the process. Moreover, solving the problem using dynamic programming results in quadratic time, data dependencies that limit parallelism, and the need for complete computation before backtracking for a number of differences. Completion of edit distance calculations is impossible in subquadratic time [12]. Thus to solve the problem in a

more efficient manner much intelligent algorithms are needed along with dedicated hardware.

2.2 Algorithm Acceleration

Read mapping being dynamic programming [12] problem can not be solved on itself in sub quadratic time complexity [12]. Hence, in order to accelerate the process of genome sequencing multiple technologies and processes exist. Namely the acceleration can be divided into three main types:

1. Seed Filtering Technique
2. Pre-alignment Filtering
3. Read Alignment Acceleration

Seed Filtering techniques are focused on reducing the number of seed locations. It limits the number of mapping locations for each seed. Pre-Alignment filtering is focused on reducing the number of invalid mappings that would otherwise take a lot of time to process during the read alignment process. Lastly, using dedicated accelerators and algorithms the process of read alignment can be accelerated.

2.2.1 Seed Filtering Techniques

The first step of the process of read mapping is indexing. Indexing can be accelerated by reducing the number of seeds that are to be stored in the index. Minimap [13] a read mapping algorithm works by reducing the number of seeds as stated by finding the minimum set of seeds from a group of seeds in a region of the genome. Heuristics can be applied to reduce the processing of reoccurring seeds more than a user-set threshold value [13]. Indexing is also a memory-intensive process and hence a better approach to deal with this problem is proposed in RADAR [14] which proposes processing in-memory architecture to avoid unnecessary movement of data between different devices.

Pre-Alignment filtering is the removal of highly dissimilar strings before sending them to the read alignment algorithm. Sequence alignment is a computationally expensive process and thus reduction of the load from pre-alignment filtering has a huge reduction in processing time. The conceptual idea behind pre-alignment filtering is to see if two strings (k-mers) differ more than a certain threshold (Edit Distance) and reject those strings that do before performing the alignment. This project focuses on the acceleration of pre-alignment filtering and hence is covered in much detail in the next heading. Once the data has reached the sequence alignment step the acceleration can be performed by either using hardware or heuristics. Using heuristics results in the sacrifice of accuracy for the sake of reduced computation time. Even after three decades of work on sequence alignment [4] the fastest algorithm [15] still has the time complexity of

$$O(\frac{m^2}{log_2 m}) \tag{2.2}$$

Which as proven [12] cannot be solved in sub quadratic time. Hence to accelerate the process dedicated hardware can be used. Multiple direction can be taken when talking about hardware acceleration mainly being:

1. SMID capable CPUs
2. Multicore architecture
3. Application Specific Integrated Circuits (ASCIs)
4. FPGAs
5. Processing in Memory

Algorithms such as Ukkonen's banded algorithm [16] work a way around by only computing the needed part of the dynamic programming matrix (diagonal vectors) rather than the whole matrix. Parasail [17] and KSW2 (minimap) [13] exploit Ukkonen's algorithm and use SIMD-capable CPUs to compute the DP

matrix. This allows high parallelism to be possible reducing the time needed for execution. GASAL 2[18] uses the multicore architecture of GPUs for parallel computing reducing the time needed for alignment. Heuristic-based algorithmic curation employed by Levenshtein distance [19]and Myer's bit-vector algorithm [20] employ restrictive scoring functions sacrificing the optimally of alignment for a reduced time. However, due to the fact that ASM is a DP-based problem the work around it during the alignment step is hard and expensive to implement.

We focus the project towards the acceleration of the Pre-Alignment filtering algorithm as it can significantly reduce the processing needed to be done in the alignment process by removing any strings in read data that might be wasting the resources. Even if the alignment algorithm is accelerated the examination of highly dissimilar strings consumes a huge number of resources and time. Hence the need for pre-alignment filtering becomes visible.

2.2.2 Pre-Alignment Filtering

Pre-Alignment Filtering is a part of the read mapping process aimed towards reducing the need for the DP algorithm used by eliminating dissimilar strings pertaining to an Edit Distance Threshold defined by the user. The representation of pre-alignment filtering use is represented in Figure 2.8. The pre-alignment filter aims towards removing most of the dissimilar string while keeping the similar strings and performing the process quickly.

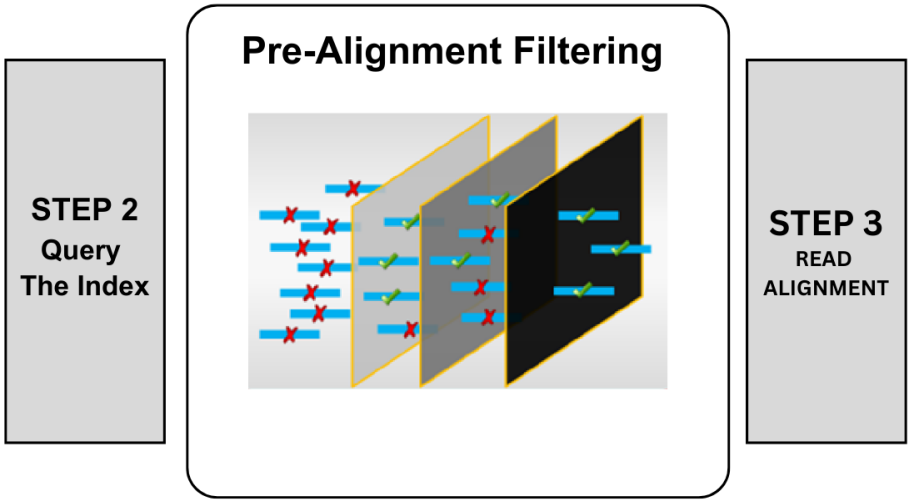

Figure 2.8: Pre-Alignment Filtering adapted from Alser et al.

Once the potential mapping locations of the reads are mapped on the reference genome by using indexing and querying a test of similarity and similarity needs to be performed. To avoid the computation of dissimilar strings a heuristic-based approach is used which is referred to as pre-alignment filtering. Any pre-alignment filtering algorithm works by estimating the number of edits between a given sequence and the reference genome and deciding whether the read will be suitable for the DP algorithm. Multiple researchers have shown that only the data that does not exceed an edit distance threshold E (use-defined) prove useful for genomic studies [21, 22, 4].

The first approach used by pre-alignment filtering is the Pigeonhole Principle. It goes by claiming that if E items are placed in E + 1 boxes then one of the boxes would be empty [4] as shown in Figure 2.9. If two genomic sequences differ by E edits then according to the principle, they are bound to share at least one subsequence that is free of edits among E+1 subsequences [22]. If we have a read of length n and there are a maximum of E edits between the read and reference.

A read is considered to be similar if it shares at most E + 1 non-overlapping substrings, and with a total length of each being $m - E$.

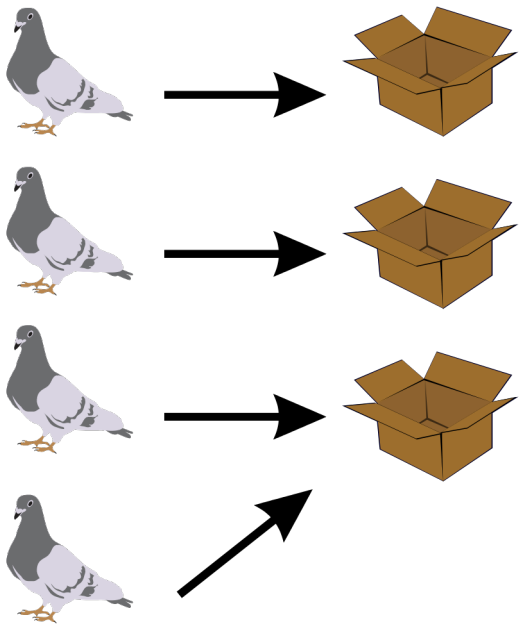

Figure 2.9: Pigeon Hole Principle

The algorithms that employ this principle are benefited from the property that as non-overlapping strings are used the problem is highly parallelizable due to their independent nature. This gives the pigeonhole principle-based algorithm an edge over other methods acting as a better-suited implementation for acceleration. The algorithms utilizing the pigeonhole principle can work without even specialized hardware. Shifted Hamming Distance [23] utilized any SIMD capable CPUs to provide high-speed filtering however is limited by the length of SIMD register widths to 128 base pairs at a time. Gate Keeper [24] can provide a large amount of parallelism by exploiting the FPGA architecture overcoming the limitation placed on shifting hamming distance. However, the Gatekeeper algorithm has the tendency to provide false positives (mismatches categories as matches) which needs to be verified later. MAGNET [25] provides an analysis of the inaccuracies of both GateKeeper and SHD. Shouji [22] provides an accuracy of pre-alignment

filtering by double those of previous algorithms discussed. Finally, Sneaky Snake [26] maps the problem of pre-alignment filtering to the Single Net Routing problem on VLSI Chips and provides $x4$ times better filtering and accuracy. Besides that, this is the only algorithm that can be implemented on CPUs, FPGA's and GPU's making it highly optimizable for any platform.

Base Counting-based pre-alignment filtering compares the number of bases in read and reference. The difference in the count is equal to the number of edits in the sequence. However, this being a simple approach rejects most of the dissimilar sequences [27] providing a less accurate result in comparison to the pigeonhole principle.

Lastly, the q-gram filtering approach considers all the possible overlapping substrings of a length q. The approach is to use a sliding window of length q to find m-q+1 substrings from the read and the fact that E edits in a data cannot affect more than E.q-grams. GRIM-Filter [28] uses this principle along with the benefits of stacked memory to accelerate q-gram-based filtering. However, q-gram-based approaches can only handle a small number of edits.

2.2.3 Choice of Algorithm

It has been established so far that in order to accelerate the process of read mapping multiple methods exist. Due to the nature of pre-alignment filtering being highly parallelizable and not a DP-based problem the best choice for accelerating the process of read mapping for this project is considered to be pre-alignment filtering.

Further within pre-alignment filtering multiple methods exist. After the review of the said methods, the nature of algorithms based on the pigeonhole principle is preferred due to the following reasons:

1. The substrings are independent of each other and thus can be implemented in a highly parallel fashion

2. The algorithms based on the pigeonhole principle can handle a large number of edits

3. The algorithms based on pigeonhole do not reject major dissimilar sequences like base counting-based algorithms.

Among the reviews algorithms those that work on pigeonhole principal are the following:

1. Shifted Hamming Distance [23]

2. Gatekeeper Algorithm [24]

3. MAGNET [25]

4. Shouji [22]

5. Sneaky Snake Algorithm [26]

These algorithms all have their own advantages and disadvantages over each other. The algorithm of choice for this project is Sneaky Snake Algorithm due to the following reasons:

1. SHD algorithm is limited by the capabilities of SIMD machines. It is only capable of taking 128-bit input read at a time.

2. SHD algorithm due to its nature of sequential comparison is slower in comparison to other algorithms.

3. Gatekeeper [24] algorithm is inaccurate after a certain edit threshold.

4. Sneaky Snake can be implemented on CPUs, GPUs, and FPGA, being the only algorithm capable of doing that.

5. Sneaky Snake improves the accuracy of pre-alignment filtering by 4 times in comparison to the remaining four state-of-the-art algorithms [26].

The choice becomes visible once these benefits of sneaky snake are reviewed over other existing algorithms.

2.3 Sneaky Snake Algorithm

The logical choice of algorithm to be implemented on heterogeneous computing systems is clear from the discussion in the previous heading. Sneaky Snake is the only algorithm that can take advantage of all the hardware accelerators and still yield results when used with state-of-the-art read mappers. This section attempts to cover the theoretical aspects of the Sneaky Snake algorithm. Just as any pre-alignment filtering solution, Sneaky Snake is focused on accelerating the sequence alignment process by examining sequence pairs before sending the reads to the sequence aligner. It being a pigeonhole principle-based pre-alignment filter decides which string to send based on estimating the edits between two sequences. Given that two sequences differ more than a certain edit distance threshold the algorithm classifies them as dissimilar and reject them. In the case of Sneaky Snake, the edit distance estimated should either be less or equal to the edit distance threshold provided to the algorithm. Sneaky Snake reduced the ASM problem to an SNR problem. It targets to find the routing path between two points encountering the minimum number of obstacles. The algorithm treats all the edits or obstacles equally without any priority eliminating the need to examine various arrangements of edit operations. A sample of a chip maze can be seen in Figure 2.10 below.

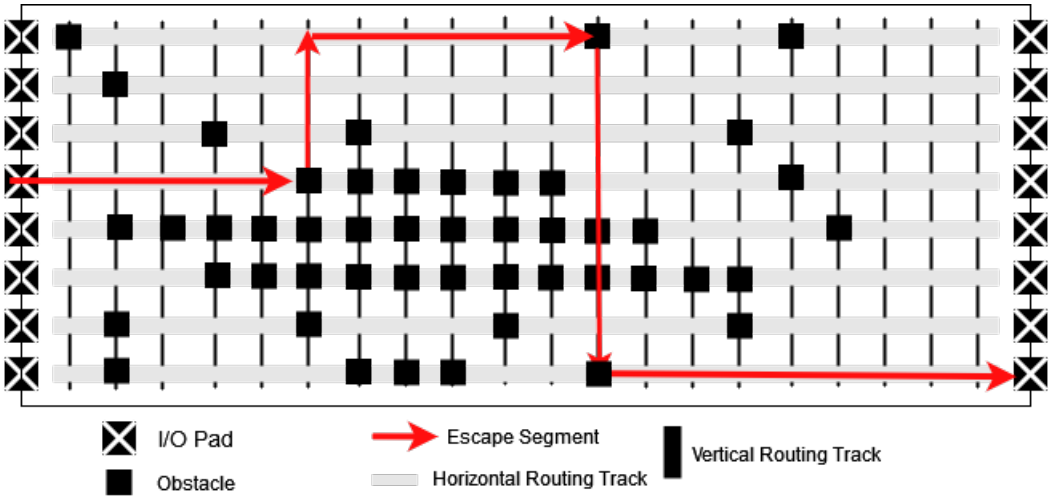

Figure 2.10: Chip Maze SNR Problem

Implementation of Sneaky Snake by Alser et al. [26] was carried out on a 2.3Ghz Intel Xeon Gold 5118 CPU hosting 48 threads and 192 GB RAM and on Intel E3-1225 with 32 GB RAM. The FPGA-based implementation of the algorithm was carried out on Xilinx Virtex 7 VC709 and in the case of the GPU-based implementation Nvidia GeForce RTX 2080 Ti was used. The authors claim the implementation to be independent of the vendor of GPUs and FPGAs.

The performance of the sneaky snake is evaluated for three main criteria. Filter Accuracy, Filtering Time, and benefits of coming it with state-of-the-art algorithms for sequence alignment. Sneaky Snake was found out to provide 31412 x more accuracy than GateKeeper, 20603 x more accuracy than Shouji [22], and 64.1 x more accuracy than MAGNET. Sneaky Snake is concluded to have a very low false acceptance rate and a 0% false rejection rate. In short sequence alignment Sneaky Snake reduces the execution time of Edlib and Parasail [17]by 37.7 x when using a 0 edit distance and 43.9 x when using E = 2. Sneaky Snake is 43 x faster than Edlib and Parasail in sequence pair examination. Sneaky Snake gives 8.9 x and 40 x speedup to Edlib and Parasail end-to-end execution time. The execution time of Parasail reduces by almost 536 x when implemented with the FPGA-based implementation of Sneaky Snake. Sneaky Snake accelerated the sequence aligners even when using a high edit distance threshold when integrated with long sequence aligners. The verification of the potential of the Sneaky Snake algorithm makes it ideal to be implemented on heterogeneous systems as the very nature of the algorithm makes it well suited for such implementation. However, the reason for the speed achieved by Sneaky Snake is possible due to the use of top-end processing hardware. Alser et al. [26] themselves have utilized expensive processing equipment to achieve the results discussed. Pre-Alignment filtering is a step done prior to read alignment and most of these algorithms are not utilizing any form of processing in memory. Hence implementation of this algorithm on a cloud-based platform can help greatly make the filter accessible to anyone willing to use it without the need for purchasing expensive equipment.

2.4 Cloud Based Read Mapping and Pre-Alignment Filtering

Without a doubt cloud computing has made its stride into modern world. Cloud Computing enables the user to utilize high-end computing power without the need to buy the infrastructure itself. Genome Sequencing, specifically read mapping being a computing-intensive job can take advantage of cloud computing to make the tools for read mapping accessible to anyone willing to use them without the need for setting up an environment on local hardware and purchasing the hardware in the first place.

Tools capable of performing the whole process of genome mapping have already started appearing on various cloud computing services. Strombow [29] utilizes Amazon web services enabling anyone with a Linux computer to access the cloud and use read mapper without the need for any special hardware. STROMSeq [30] is a saleable tool based on the cloud for read mapping providing a pipeline for personal genome processing. Amazon Storage Service S3 is used to upload the data and a pipeline cluster is used to load data into EC2 Amazon web services with live progress notifications available to the user. MapReduce technique is used in a heterogeneous implementation of read mapping on the cloud proving a low-cost acceleration solution [31]. Eoulsan [31] and Cloud-MAQ [32] are yet again the implementation of read mapping and genome assembly solutions on the cloud. Eoulsan allows users to easily set up an instance on Amazon web services and do multiple sample testing at once. Cloud-MAQ is an extension of Mapping and Assembly with a Quality system that suffers from being inherently single-threaded by redefining the problem to be implemented in a multi-threaded form and then integrated with the cloud for quick access. Most of the implementations of read mapping tools on the cloud as discussed aim to virtualize the whole process. Cloud Computing allows users to communicate within machines globally and securely. Hence, it seems much more appropriate to optimize the implementation of a single

algorithm on the cloud and then use networking to access the solution and reuse it in any tool that might be in need. This project takes the Sneaky Snake algorithm and implements it to the best of judgment on the cloud in order to achieve global distribution of the most efficient pre-alignment filter.

Chapter 3

Theory and Background

3.1 Heterogeneous Computing Sytems

The increasing difficulty of scaling the performance and efficiency of CPUs every year has created the need for turning computers into heterogeneous systems [1]. Heterogeneous systems are basically systems composed of different types of processors e.g., CPU, GPU, FPGA, ASIC (Application Specific Integrated Circuit) etc. These systems exploit the properties and advantages of each of these processors so that they can work together to provide the user with maximum speed and processing power. Heterogeneous systems came into being because Moore's law which stated that the number of transistors would double each year on a microchip exhausted its power and a further increase in the number of transistors did not affect the computing speed as much. A heterogeneous system is made up of different computing nodes and each of these computing nodes has its own unique capability and way of executing instructions. Thus, these systems are parallel systems, using different types of cores with different architectures. Heterogeneous systems became the need of the hour when computationally extensive applications came into being. These applications include artificial intelligence (AI), machine learning (ML), unmanned driving, industrial simulation, and other such fields. Different processors having different instruction sets are used in this hybrid system. This leads to more work for the designer since the source code written for one processor

needs to be modified before using it on any other processor. Thus, the process of code writing becomes cumbersome.

This section will discuss heterogeneous computing systems in detail, their architectures, and the different processors used. Along with this, the different programming languages, advantages, and main challenges of heterogeneous computing will also be discussed.

3.1.1 Architecture of Heterogeneous Computing Systems

Heterogeneous systems have a variety of different options when it comes to combining different acceleration devices with the host. The most simple and low-power consuming architecture is the combination of CPU with ASIC but other combinations providing better performances are also available which include CPU with GPU, CPU with FPGA, etc. A GPU evolved into a general-purpose parallel processor more than ten years ago so that various tasks other than those involving graphics can benefit from it. An example of such a task is Machine Learning because before the use of a GPU, machine learning and artificial intelligence required training to be completed in unreasonably long periods of time. Field-Programmable Gate Arrays (FPGAs) are another type of computer device that can offer outstanding performance and energy efficiency advantages. Some more examples of specialized accelerators include Tensor Processing Units, which are used for neural networks, and near-data processing architectures which place the computations close to or inside the memory reducing the problem of latency.

Today, the focus is on increasing parallelism in a computing system because this eventually increases the computation speed rather than the clock frequency. Thus, multi-chip, multi-core, or multi-context parallelism are important ways to boost performance. Michael Flynn in 1996 wrote a paper named “A very high-speed computing system” in which he described that four levels of parallelism exist in hardware [33]. Different heterogeneous systems are designed using different levels of parallelism and thus, the four levels of parallelism are discussed next.

3.1.1.1 Single Instruction Stream-Single Data Stream (SISD)

In this type of parallelism, a single data is processed by a single instruction. It is basically the structure of the simplest computer composed of a single control unit, a single processing unit, and a single memory unit. This system may or may not have internal parallel processing capability, but the instructions are executed sequentially. Figure 3.1 below shows the schematic of this type of parallelism.

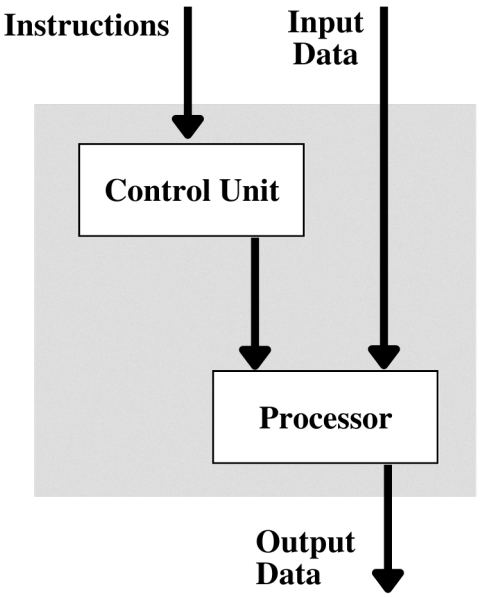

Figure 3.1: SISD Type of Parallelism

3.1.1.2 Single Instruction Stream-Multiple Data Stream (SIMD)

As the name suggests, in this type of parallelism, only one instruction is available with all the processors, but each processor has its own data available on which it executes the instructions. Increased performance is gained strictly by using more units [33]. This system is utilized for vector operation processing and digital signal processing, and the compiler typically completes its parallel role. For instance, the single-chip Cell Broadband Engine Architecture has a conventional CPU core as well as eight SIMD accelerator cores. It's adaptable and each core can run a

different application while interacting with one another across a quick on-chip bus. Its primary design goal is to maximize performance while using the least amount of power. Figure 3.2 below shows the schematic of this type of parallelism.

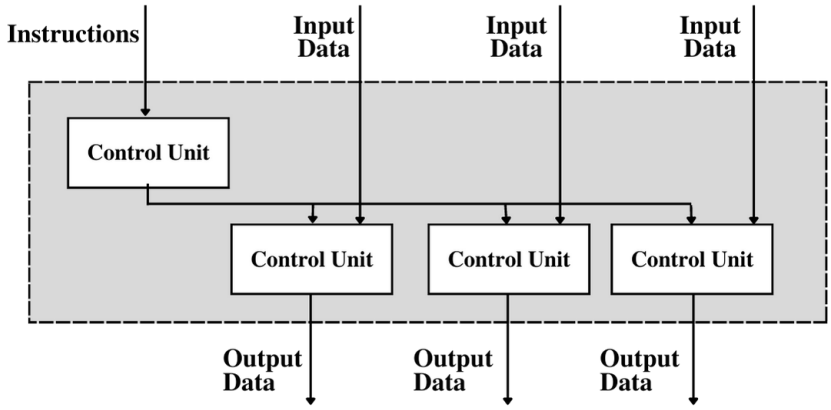

Figure 3.2: SIMD Type of Parallelism

3.1.1.3 Multiple Instruction Stream-Single Data Stream (MISD)

Multiple Instruction Stream-Single Data Stream is the least common structure among the four levels of parallelism defined by Michael Flynn. In essence, it uses high bandwidth dedicated execution unit. Then, 'n' number of virtual machines with independently running program sequences share this unit. The execution hardware is accessible to each virtual machine once every cycle. Every virtual machine has its own private instruction memory, and only the shared data memory allows interaction across instruction streams. The bandwidth of the common data storage must presumably be N times bigger than the individual instruction storage if there are N instruction units. The figure below shows the schematic of this type of parallelism.

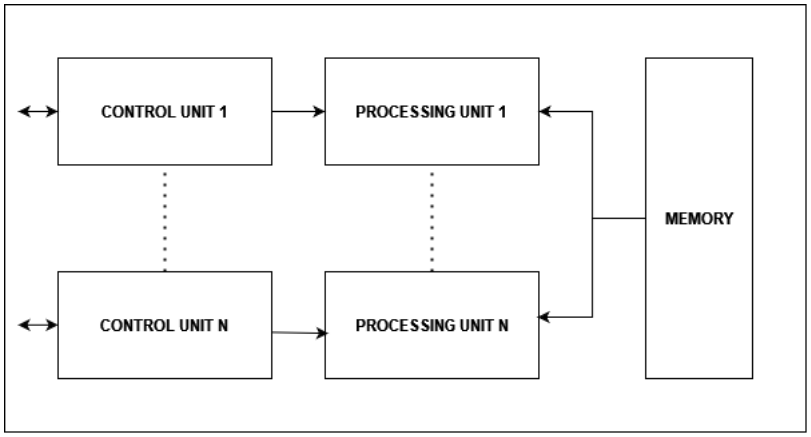

Figure 3.3: MISD Type of Parallelism

3.1.1.4 Multiple Instruction Stream-Multiple Data Stream (MIMD)

The last level of parallelism is the multiple instruction stream-multiple data stream (MIMD) type of parallelism. In this system, different processing units or cores are present each having its own independent instructions and data. Here, multiple instructions map to numerous data. MIMD is further divided into two categories i.e., single-program multi-data (SPMD) and multi-program multi-data (MPMD). This configuration makes the most of the execution unit's and memory unit's permitted bandwidths. Naturally, such an approach could face criticism on the grounds that the necessity of the teaching sequences' independence does not directly meet the needs of significant scientific challenges. The requirements of the timesharing and/or utility environment would be better served by it.

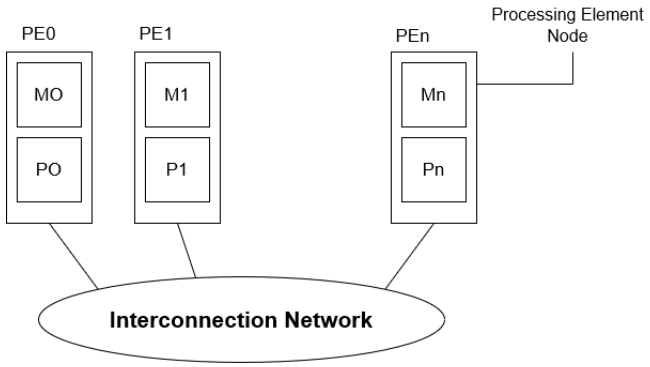

Figure 3.4: MIMD Type of Parallelism

3.1.2 Heterogeneous Programming Languages

In Heterogeneous Computing Systems, programming is a little extensive and cumbersome. This is because a heterogeneous computing system is an amalgamation of a traditional CPU with any accelerator such as GPU, FPGA, ASIC, etc. Programming in Heterogeneous Systems in mainly down using two platforms: CUDA and OpenCL. This section will give an overview of the two platforms and will discuss them in detail.

3.1.2.1 CUDA

In November 2005, NVIDIA introduced CUDA, a general-purpose parallel computing platform and programming model that leverages the parallel compute engine in GPUs to solve many complex computational problems in a more efficient way than a CPU. CUDA can be easily used by programmers using heterogeneous systems composed of a traditional CPU and a highly parallel GPU. This is because CUDA is only an extension of the C programming language. This is because CUDA is only an extension of the C programming language. Programmers can not only use C language with CUDA but various other popular programming languages as well including C++, Python, MATLAB, etc. and parallelism can be conveyed by expanding a few fundamental phrases.

CUDA is mainly used for GPUs so when CPU is combined with a GPU this comes into action. In applications where GPU and CPU are both used together, CPU runs the logical execution part of the workload whereas GPU runs the intensive computing part of the application by allowing them to run in parallel on the numerous cores present in GPU. Although CUDA is an extension of C, it must explicitly handle the data transport between the host memory and different device memories and adjust GPU memory consumption in accordance with the underlying GPU architecture.

CUDA is extensively used by researchers and engineers to perform extensive computations in various fields such as image processing, video processing, compu-

tational biology and chemistry, fluid dynamics simulation, CT image reconstruction, etc. Due to the emergence of such fields in recent years, sales of CUDA-based GPUs have increased significantly and have reached millions [1].

With the advantages CUDA brings with it, some drawbacks also exist. CUDA is not portable i.e., code written for one device cannot be used for the others. This means that the code needs to be modified for each device and thus, an increased workload exists on the designer. This problem is dealt with OpenCL which is discussed next.

3.1.2.2 OpenCL

OpenCL is an open platform developed by the Khronos family. In order to speed applications with task-parallel or data-parallel calculations in a heterogeneous computing environment made up of the host CPU and any attached OpenCL "devices", developers can use OpenCL, which offers a common programming language, programming interfaces, and hardware abstractions [34]. Like CUDA, OpenCL is also based on the C language and is capable of providing cross-platform parallel computation APIs. OpenCL was started with the objective that designers should have access to a cross-platform and the code written would be able to move between devices. In short, OpenCL enables the user to design or program portable parallel applications for heterogeneous systems.

One of the characteristics of OpenCL is memory sharing. The OpenCL programming imply heterogeneity between the host and attached devices in the sense that the attached devices do not share memory with the host CPU. All the attached OpenCL devices frequently have a distinct machine instruction set. OpenCL was created to address the quickly expanding demand for high-performance standard application development platforms of different parallel computing platforms and gets around the crucial limitations of earlier parallel computing system programming models that varied in terms of application

portability. Thus, this solves the problem of program portability presented in CUDA.

3.1.3 Advantages of Heterogeneous Computing Systems

The advantages of Heterogeneous Computing Systems are given as follows.

- Heterogeneous computing systems offer parallel task division technology i.e., the different parallel tasks going on in a program can be divided amongst the processors. This greatly increases the computation speed.
- These systems not only increase the computation speed but also play their role in reducing the power consumption which again is a major concern when new technology is invented.
- Development of Heterogeneous Computing Systems paved the way for the introduction of data communication technology between heterogeneous processors.
- A fine-grained algorithm with a GPU at its core was proposed by Shui, who also demonstrated how successful it is. Zhang built a genuine CPU + MIC heterogeneous cluster and examined its behavior and performance by looking at various message delivery and remote direct memory access techniques [1].

3.1.4 Problems of Heterogeneous Computing Systems

The problems of Heterogeneous Computing Systems are given as follows.

- Heterogeneous Computing Systems come with programming difficulties since different processors are combined together. Different processors mean different architectures, different instruction sets, and thus different programs. The traditional CPU code cannot be used for all the processors. This means that there is more extensive programming and the designer needs to do more work.

- The problem of program portability exists in Heterogeneous Computing Systems. This is because processors from different companies have different architectures so the code needs to be modified and in some cases is to be rewritten to allow for accurate working of the system.

- Another problem with heterogeneous computing systems is the complexity of data access. This is because a lot of memory storage elements are involved including the host's memory. host cache, device main memory, device cache, and registers [1]. This means that data is stored in all of these different memory-storing elements and accessed from them as well. Thus, program complexity and latency increase.

- It is challenging to effectively divide the activities and can easily result in uneven loads across heterogeneous processors due to the difference in processing capacity and their individual computing characteristics. It is particularly challenging for a task division strategy to adjust to the needs of heterogeneous processors since the architecture and parallel mode of heterogeneous processors differ [1].

3.2 Sneaky Snake Algorithm

The Sneaky Snake algorithm is the most efficient pre-alignment filtering technique developed by researchers at ETH University, Sweden. This is a highly accurate and highly parallel pre-alignment filter that greatly reduces the computations involved in sequence alignment [26]. By reducing the number of computations involved, this technique in turn reduces the cost involved in the sequence alignment process. The focus of this technique is to reduce the approximate string matching (ASM) problem faced in the process of sequence alignment to the single net routing (SNR) problem in the VLSI chip layout. Before moving on to any further discussion, the term "Edit Distance Threshold" will be defined since it acts as a very important

parameter in the Sneaky Snake design. The "edit distance threshold" is defined as the minimum number of edits needed to convert one sequence into the other.

The discussion of approximate string matching and single-net routing is given next.

3.2.1 Approximate String Matching

Sequence alignment is the most fundamental step in the genome sequencing process. This step is basically called "Approximate String Matching," and it works by examining the edit distance between two given sequences and the type and location of each type of edit present between the two given sequences. The approximate string matching problem is a dynamic programming problem since it needs to keep track of all the pairs of prefixes that provide the minimum edit threshold [26]. To eliminate the need for dynamic programming, researchers at ETH came up with the idea of a pre-alignment filtering technique. This technique reduces the need for dynamic programming and, thus, reduces the number of computations and the cost incurred during the sequence alignment process. Hence, the Sneaky Snake algorithm was developed and it works by reducing the ASM problem to the SNR problem i.e., instead of calculating the sequence alignment, the Sneaky Snake algorithm basically Details about the SNR problem are discussed next.

3.2.2 Single Net Routing (SNR) Problem

The SNR problem in VLSI chip layout refers to the problem of optimally interconnecting two terminals on a grid graph while respecting constraints. This problem basically finds the path between the source and the destination terminals, called the signal net, that has the least number of obstacles in it. The number of obstacles should be less than or equal to the defined edit distance threshold.

The chip layout is such that it consists of two layers of evenly spaced metal routing tracks. One layer moves through the chip horizontally through the horizontal routing tracks (HRT) while the second layer moves through the chip vertically

through the vertical routing tracks (VRT). These tracks, when combined, form a two-dimensional grid. Also, each obstacle encountered adds a certain amount of propagation delay, $t_{obstacle}$.

The main objective of the SNR problem is to find the optimal signal net. An optimal signal net is defined as the signal net that has the shortest and fastest routing track. Here, another term, "escape segment," will be defined because it will be used later in the text. An escape segment is a segment that has at most one obstacle in it, and the right side of the escape segment is called a checkpoint. Now, the ASM problem is reduced to the SNR problem by calculating a chip maze. Each entry of the chip maze represents the pairwise comparison result of a character from one sequence with another character from the other sequence [26]. If a mismatch occurs, it is represented by a "1" in the chip maze. Likewise, if a match occurs, it is represented by a "0" in the chip maze. To design the chip maze, a total of four equations are used, which are given as follows [26]:

$$Z[i,j] = 0,\ if\ i = E + 1,\ Q[j] = R[j] \tag{3.1}$$

$$Z[i,j] = 0,\ if\ 1 \leq E,\ Q[j-i] = R[j] \tag{3.2}$$

$$Z[i,j] = 0,\ if\ i > E + 1,\ Q[j+i-E-1] = R[j] \tag{3.3}$$

$$Z[i,j] = 1,\ otherwise \tag{3.4}$$

In the equations, Q[j] represents the query sequence and R[j] represents the reference sequence. These equations take into account all the possible pairwise comparisons between the two input sequences. Equation 3.1 compares the j^{th} entry of the query sequence with the j^{th} entry of the reference sequence. Equations labeled 3.2 and 3.3 checks for deletions and insertions between the two input sequences. Thus, by doing this, the equations take into account substitutions, deletions, and insertions between the two input sequences.

Once, the chip maze has been computed the next problem is to find an optimal signal set of non-overlapping escape segments. In the chip maze, HRTs represent the rows and VRTs represent the columns. Thus, the next step is to solve the single net routing problem

3.2.3 Solving the Single Net Routing Problem

The fundamental function of the Sneaky Snake method is to find the optimal signal net by solving the SNR problem. Now, there are two main objectives behind solving the SNR problem. These objectives include:

- Finding the minimum number of escape segments that link the source terminal with the destination terminal.
- By just taking into account each escape segment once and in a monotonically increasing order of their start index, the signal net can be made as short as possible (or end index).

The Sneaky Snake needs to fulfill these objectives in order to solve the SNR problem. This is done by using five simple steps which are given as follows:

1. The first step is to calculate the chip maze by using the four equations labeled 3.1-3.4. Once, the chip maze has been constructed, the first column is taken to be the first checkpoint and this is where the iteration starts.
2. The Sneaky Snake algorithm always chooses the longest escape segment at each subsequent checkpoint, enabling the signal to go as far as possible before hitting an obstruction. It determines the length of the first horizontal segment of consecutive entries with the value "0" that begins at a checkpoint and ends at an obstacle or the end of the current row for each row of the chip maze. The length of each 2E + 1 computed horizontal segment is compared by the Sneaky Snake algorithm, which then chooses the longest segment and uses it as well as the first obstacle after it as an escape segment. Following a

checkpoint, all rows begin with an obstruction, therefore the Sneaky Snake algorithm is unable to discover a horizontal segment, it considers one of the hurdles is regarded as the longest escape segment. It views the computed escape segment as a component of the SNR problem's resolution.

3. In the third step, a new checkpoint is created after the longest escape segment.

4. In the fourth step, step number two and three are repeated to find whether the signal net has reached its destination or the total propagation delay has become more than E x $t_{obstacle}$.

5. The final steps take decisions i.e., if an optimal signal net has been calculated this would mean that the number of edits between the two input sequences is less than the edit distance threshold. Only these sequences will be passed onto the costly and computationally extensive sequence alignment step and all the others will be discarded.

This is shown in the Figure given below

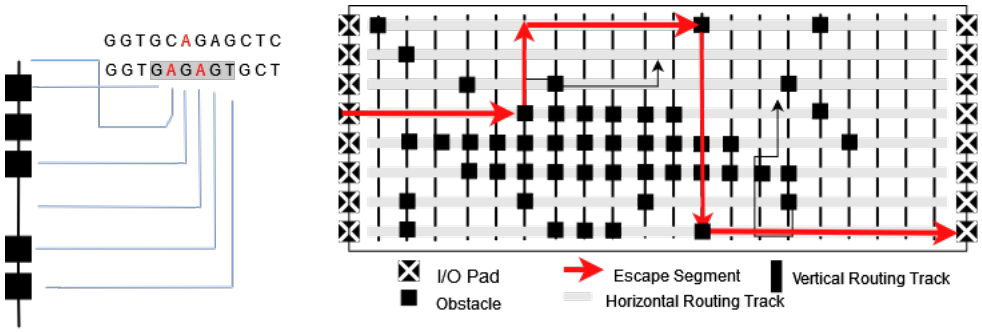

Figure 3.5: Left: Shows the chip maze for E=3, Right: Shows the Chip maze calculated using the equations 3.1-3.4

We utilize an implicit representation of the chip maze to effectively implement the Sneaky Snake algorithm. That is, until it encounters an obstacle (i.e., Z[i,j] = 1) or reaches the end of the current row, the Sneaky Snake algorithm begins computing each entry of the chip maze on the fly for each row. As a result, the

entries that are actually calculated for each row of the chip maze are only those entries that are situated between each checkpoint and the first obstacle in each row following this checkpoint, as we demonstrate in Figure below.

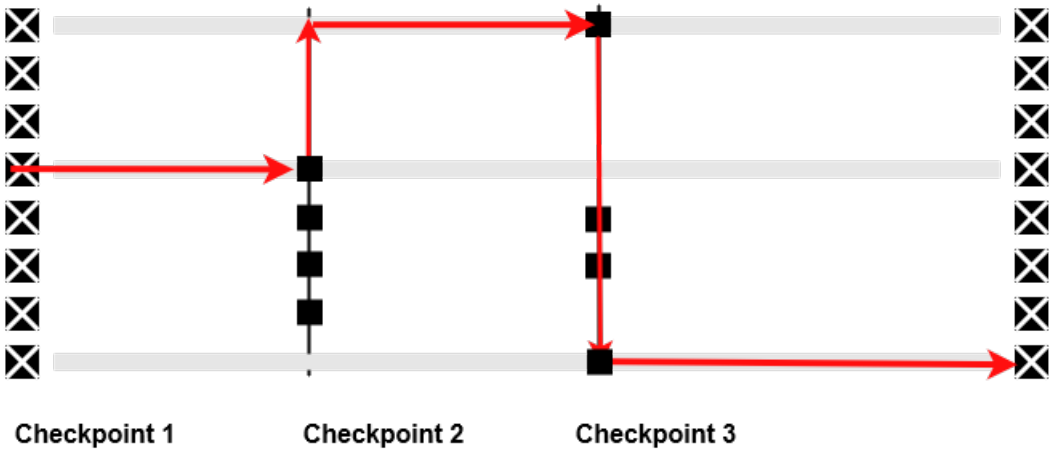

Figure 3.6: Calculating chip maze on the fly

The Sneaky Snake algorithm requires less computations as a result, which is a substantial reduction. Thus, the Sneaky Snake is both accurate and efficient in solving the SNR problem

3.3 Cloud Computing

The ever-increasing use of computers led to the introduction of cloud computing. In recent years, cloud computing has seen a rise in popularity and within the distributed computing field, the term has become popular. Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction [35]. The primary reason for the rise in the popularity of cloud computing is that it can be used in many applications. Cloud computing consists of five essential elements which are explained as follows.

1. On-demand self-service: A user can automatically (i.e., conveniently, self-serve) access computing resources (such as CPU time, network storage, soft-

ware use, and so on) at a specific time without having to engage with the resource providers in person.

2. Broad network access: These computing resources are made available over a network (like the Internet) and are used by a variety of client applications running on various heterogeneous platforms (like mobile phones, laptops, and PDAs) at the customer's location.

3. Resource pooling: A key idea in cloud computing is resource pooling, which describes the collection of computing resources from various clients or tenants into a single shared infrastructure. It entails pooling and effectively allocating resources, including computing power, memory, storage, and network bandwidth, to serve a variety of users or applications in a fluid and dynamic way.

4. Rapid elasticity: One of cloud computing's key properties is quick elasticity. It describes a cloud service's capacity to swiftly and automatically scale computing resources up or down in response to demand. Rapid elasticity enables cloud customers to quickly adjust their usage of computer resources, such as processing power, storage, and network bandwidth, to meet changing needs.

5. Measured Service: The cloud infrastructure is able to employ appropriate mechanisms to measure the utilization of these resources for each individual consumer through its metering capabilities, despite the fact that computing resources are pooled and shared by several customers (i.e., multi-tenancy).

3.3.1 Service Models

Service models are used to categorize cloud services. A service model as used in the context of cloud computing, is a structure that established the degree of control and accountability that customers have over the computer resources and services they obtain from a cloud provider. It defines the service's parameters and

specifies how the roles of the cloud provider and cloud user will be split up. There are three very commonly used service models which are explained as follows.

1. Software as a Service (SaaS): SaaS offers fully working programs for customers to access and use directly using a web browser or a client interface. These apps are delivered over the internet. Users don't need to be concerned about infrastructure administration, software upgrades, or maintenance when using SaaS. Everything related to the service is handled by the cloud provider, including the platform, software, and infrastructure. Users subscribe to the software and pay for it that way. Dropbox, Google Workspace, and Salesforce are a few examples of SaaS applications.

2. Platform as a Service (PaaS): Users that utilize PaaS can develop, test, and deploy apps without having to worry about managing the underlying infrastructure because it offers a comprehensive development and deployment environment. While the cloud provider manages the infrastructure, including servers, storage, networking, and operating systems, users may concentrate on creating and executing their applications. Databases, middleware, and programming frameworks are frequently included as part of PaaS services. Heroku, Google App Engine, and Microsoft Azure App Service are examples of well-known PaaS services.

3. Infrastructure as a Service (IaaS): Users that utilize PaaS can develop, test, and deploy apps without having to worry about managing the underlying infrastructure because it offers a comprehensive development and deployment environment. While the cloud provider manages the infrastructure, including servers, storage, networking, and operating systems, users may concentrate on creating and executing their applications. Databases, middleware, and programming frameworks are frequently included as part of PaaS services. Heroku, Google App Engine, and Microsoft Azure App Service are examples of well-known PaaS services.

With IaaS giving customers the most power and responsibility, PaaS abstracting away infrastructure maintenance, and SaaS supplying ready-to-use apps, these service models constitute a stack of increasing abstraction. Taking into account aspects like control, scalability, customization, and administration requirements, organizations can select the service model that best suits their needs.

3.3.2 Deployment models

A cloud deployment model defines the cloud services being consumed and the responsibility model for who manages them. It defines the cloud architecture, scalability of the computing resources, what can be changed, the services provided, and how much of it is built by the user [36]. There are a total of four cloud deployment models available which are explained as follows.

1. Private cloud: Regardless of whether it is housed on or off premises, the cloud infrastructure is only used by a single organization and is maintained by that organization or a third party. There are various reasons for setting up a private cloud within a company. First, to make the best possible use of already available internal resources. Second, many businesses consider the private cloud as a solution due to security issues including data privacy and trust. Third, there is still a considerable expense involved in moving data from a local IT system to a public cloud. Fourth, organizations must always have complete control over all operations that take place behind their firewalls. Finally, academics frequently construct private clouds for academic purposes.

2. Community cloud: The same cloud infrastructure is built and shared by several organizations, together with their shared policies, needs, principles, and worries. The cloud community develops to a point of democratic stability and economic scalability. A third-party vendor or one of the community's organizations may host the cloud infrastructure.

3. Public cloud: The current cloud computing deployment model that predominates is this one. The broader public uses the public cloud, which is fully owned by the cloud service provider, with its own policies, values, profits, costs, and charging structures. Public clouds include Google AppEngine, Amazon EC2, S3, and Force.com, among many other well-known cloud services.

4. Hybrid cloud: Public and private clouds are combined in the hybrid cloud paradigm. By allowing data and applications to be transferred between both, it enables organizations to take advantage of both environments' advantages. For instance, a business can use a public cloud for scalability and cost-effectiveness while using a private cloud for confidential information and applications.

Organizations select the best deployment model depending on their particular demands, regulatory requirements, security issues, and economic considerations. Each deployment model has advantages and considerations of its own.

3.3.3 Some Famous Cloud Providers

A number of well-known cloud providers provide a variety of cloud services. The most well-known cloud service companies are listed below, along with a brief description of what they provide.

1. Amazon Web Services (AWS): Amazon Web Services (AWS) is a comprehensive and well-liked cloud computing platform that is made available by Amazon. It offers a wide range of services, such as computing capacity, storage, databases, machine learning, analytics, networking, and more. AWS is a well-liked option for businesses, developers, and startups because of its scalability, worldwide reach, and broad service offering.

2. Microsoft Azure: Microsoft offers a cloud computing platform called Azure. Virtual machines, databases, storage, AI and machine learning, the Internet

of Things, and developer tools are just a few of the many services it provides. For businesses already utilizing Microsoft technology, Azure is popular for its tight integration with the company's software ecosystem.

3. Google Cloud Platform (GCP): Google's cloud computing platform, or GCP, offers a range of services for big data, machine learning, networking, storage, computing, and more. GCP is renowned for its potent data processing powers and provides powerful analytics and AI capabilities. Additionally, it has a worldwide infrastructure that crosses numerous continents.

4. IBM Cloud: A full-stack cloud platform, IBM Cloud provides software-as-a-service (SaaS) solutions, platform services, and infrastructure services. It offers a variety of AI, blockchain, IoT, security, data, and analytics solutions. The enterprise-grade security, hybrid cloud capabilities, and integration with a broad range of software services and products from IBM are all hallmarks of the IBM Cloud.

These are only a few instances of well-known cloud service providers; each one has distinctive advantages, service offerings, price structures, and a worldwide infrastructure. Businesses decide which cloud provider to choose depending on their unique needs, geography, industry emphasis, current technology stack, and cost considerations.

Chapter 4

Methodology

In this project, the implementation of Sneaky Snake is performed on three different platforms: CPU, FPGA, and Cloud. Each platform is considered separately for the execution of Sneaky Snake. To evaluate the performance improvement of the Sneaky Snake algorithm across various processors and environments, it is necessary to develop a framework for quantifying performance. This framework enables us to determine the optimal processors for algorithm implementation. In addition, by quantifying the algorithm's performance, the obtained results can be compared with data from the [26]. The study analyzes performance data collected from five unique implementations.

1. Local implementation on Intel(R) Core(TM) i5-10210U @ 1.60GHz 2.11 GHz
2. Local implementation on Xilinx Virtex 7 vc707 FPGA
3. Cloud-based implementation on Standard B20ms Microsoft Azure

4.1 Research Design and Data Collection

The performance of an algorithm can be described as the amount of useful work accomplished during execution in terms of time spent and resources utilized. It is integral for a Pre-Alignment Filter to not only be fast but also utilize the least

amount of resources possible. Sneaky Snake is the fastest Pre-alignment filter [26] available to the world today.

4.1.1 Performance Measurement

In the scope of this project, the performance of the algorithm is to be measured on CPU, FPGA, and Cloud. In the case of the CPU, the performance-measuring variables to be determined experimentally are:

- Execution Time (s)
- CPU Utilization (percent)
- Memory Utilization (bytes)
- Accuracy (percent)

In the case of the FPGA, the performance of the algorithm will be measured in reference to

- Accuracy
- Resource Utilization

For the final implementation on the Cloud, the performance measurement will be based on the measurement of:

- Execution Time (s)
- Accuracy (percent)

The decision to ignore CPU and memory utilisation for the cloud implementation of the algorithm is based on the approach's practical limitations. The code used to implement the algorithm in the CPU is profiled using the Visual Studio code profiler. However, running a comprehensive operating system with a graphical user interface (GUI) on the cloud would significantly increase resource

utilization, resulting in increased costs and decreased productivity. Consequently, the focus of the analysis is transferred to other performance metrics that can be efficiently obtained without incurring excessive overhead.

4.2 Sneaky Snake Performance Measurement on CPU

Central Processing Units execute code in a sequential manner. The data is sent through the code in a sequential fashion and the results are obtained likewise. Modern C++ offers parallel computing natively. Coupling this with the utilization of packages such as Open MP allows the CPU to execute code in a concurrent manner making the implementation more efficient. To determine the performance of the implementation on a CPU the methods for collection are addressed ahead

4.2.1 Determine Execution Time

The algorithm implementation is the only part of the code for which speed is an important factor. To determine the execution time the main part of the Sneaky Snake algorithm is placed between high-resolution clock measurement intervals as shown in the code below.

```
auto startTime = high_resolution_clock::now();
  \\ CODE FOR WHICH TIME IS TO BE MEASURED
auto stopTime = high_resolution_clock::now();
auto duration = duration_cast<microseconds>stopTime - startTime);
```

The difference between the *startTime* and *stopTime* is the execution time of the code.

4.2.2 Determining CPU Utilization

CPU utilization is measured using the code performance profiler that comes built-in with Visual Studio Enterprise. The tool provides the values for CPU utilized by various parts of the codes in terms of percentage. Using the Visual Studio gener-

ated reports the CPU utilization for a test run was considered the maximum CPU utilization through the whole run time. Furthermore, to realize the bottleneck in the code, the detailed report was used to see the most CPU-intensive process other than the functioning of the algorithm.

4.2.3 Determining Memory Utilization

When dealing with data as large as the human genome, memory becomes a bottleneck quickly. Hence, in order for any pre-alignment filtering algorithm to be classified as efficient, memory utilization is of utmost importance.

To measure memory utilization the memory usage tool of Visual Studio Enterprise is to used. The tool provides a real-time graph of the processor memory being used by the execution of the code. Using the graph an estimate of memory utilized for the total execution time determined was calculated and considered as the memory usage of the code.

4.2.4 Determining Accuracy

With an already available running and tested CPU implementation of Sneaky Snake [26] by using the same data sets the following two parameters will be calculated and considered as the measure of accuracy.

$$Accept\ Error = \frac{Theoretical\ Accepts - Experimental\ Accepts}{Theoretical\ Accepts} \times 100$$

$$Reject\ Error = \frac{Theoretical\ Rejects - Experimental\ Rejects}{Theoretical\ Rejects} \times 100$$

The theoretical accepts and rejects were obtained from the execution of the implementation of Sneaky Snake by Alser et al [26] on the same data sets.

4.2.5 Performance with Respect to Read Length

Data sets used are described in the next section are of three different read lengths, mainly: 100, 250, and 10000. The effect of the performance variables and characteristics is measured by testing the code on all three different read lengths. By having a diverse range of read lengths, the code's performance for both : short and long reads is analyzed through the results obtained.

4.2.6 Performance with Respect to Edit Distance Threshold

The data sets utilized have three different read lengths: 100, 250, and 10000. The effect of the performance variables and characteristics is tested by testing the code on all three different read lengths

4.2.7 Performance with Respect to Thread Number

To understand the relationship between the number of threads activated and the performance of code; the performance variables and characteristics are measured by testing the code with up to 8 threads.

4.2.8 Procedure

The implementation can be done on any machine with Windows Operating System and a C++ supported Integrated Development Environment. In order to obtain the data the following set of steps were followed

1. The implemented code is integrated with Visual Studio Enterprise 2022 with the data sets placed in a folder located at the root folder of the application.
2. Starting with the 100_bp_1 data set initially the following parameters were provided to the code:
    - Edit Distance Threshold: 0
    - Read Length: 100

- Number of Reads: 30000
- Threads: 1

3. The execution time, accepted reads, and rejected reads were noted.
4. The code was then profiled using Visual Studio code for CPU Usage and Memory Usage.
5. The accuracy of the results obtained was calculated by comparing the results from Alser et al. [26] implementation.
6. Edit Distance Threshold was changed according to Table 4.2 and steps 2-5 were repeated. All measurements were recorded.
7. Number of threads was increased from 1 with an increment of 2 up to 8 and Steps 2-6 were repeated. All the measurements were recorded.
8. The process was repeated for other data sets and the measurements were recorded.
9. The whole process was repeated using the implementation by Alser et al. [26]. All the measurements were recorded.
10. The collected data was tabulated and plotted using relevant plots to make comparisons and perform analysis.

Data Set	**Iteration No**				
	1	2	3	4	5
100_bp_1	E = 0	E = 2	E = 5	E = 10	E = 20
100_bp_2	E = 0	E = 10	E = 20	E = 40	E = 100
250_bp_1	E = 0	E = 4	E = 8	E = 10	E = 20
250_bp_2	E = 0	E = 20	E = 40	E = 50	E = 100
10K_bp	E = 0	E = 100	E = 1000	E = 2000	E = 5000

Table 4.1: Edit Distance Threshold for different iterations of execution

4.3 Sneaky Snake Performance Measurement on FPGA

The FPGA works in a concurrent and parallel fashion. The instances defined are independent of each other and work separately to complete their job for the whole module. It is this very nature of the FPGA that makes it the well-suited choice for the implementation of an algorithm like Sneaky Snake that involves multiple bit-wise operations and comparisons.

4.3.1 Determining Accuracy

In order to implement the Sneaky Snake algorithm accurately on an FPGA, it is necessary to provide the module with data that lies within a predetermined Edit Distance Threshold. The Edit Distance represents the minimal number of operations (such as insertions, deletions, or replacements) required to convert one string into another.

For Sneaky Snake to perform its intended function precisely, the FPGA implementation must take data with fewer edits than the predetermined threshold. This indicates that the module should take data samples that closely match the target pattern with a limited number of deviations. In contrast, the module should reject data samples with a greater number of adjustments or discrepancies that exceed the edit distance threshold.

4.3.2 Determining Resource Utilization

The Vivado Design Suite provides a comprehensive report on resource utilization upon completion of the design implementation, making it an ideal tool for the Sneaky Snake application to achieve maximum efficiency without sacrificing precision. By analyzing the report, designers can gain in-depth knowledge regarding the utilization of FPGA resources, such as logic, memory, components, and I/O. Sneaky Snake seeks to optimize performance while reducing hardware costs by minimizing resource consumption. Vivado's integrated synthesis and optimization algorithms can reduce logic complexity by eradicating redundancy and

simplifying operations. Memory optimization techniques, including the inference of reduced memory structures and the implementation of efficient access patterns, contribute to efficient resource utilization. In addition, pipelining and parallelization techniques decompose complex tasks, making use of the FPGA's parallel processing capabilities and conserving resources. By imposing area restrictions, designers can ensure that the implementation meets specified resource utilization goals. This combination of strategies guarantees that Sneaky Snake establishes a balance between optimal resource utilization and high precision, resulting in an FPGA design that is both efficient and precise.

4.4 Sneaky Snake Performance Measurement on Cloud

The measurement of execution time and accuracy for the cloud-based implementation of Sneaky Snake is exactly done using the same methods as that for CPU utilization. The data obtained will be compared with CPU utilization in case of execution time and with the Alser et al achieved data in case of accuracy.

4.5 Test Data Sets

In order to make sure that the implementation on either processor is accurate and relevant, real genomics data is needed. The experimental evaluation uses a total of five data sets. Three data sets are real and two are made using artifical means.

The real data set used is obtained Alser et al [26] by downloading the data set from EBI. The downloaded data set is then mapped to the human genome using a mrFAST and then is provided two different edit distance thresholds for different data sets. The obtained data sets are then used in testing with the Sneaky Snake implementation.

100_bp_1, 10_bp_2 and 250_bp_3 are available in the opensource code of Sneaky Snake [26]. In order to test the capabilities of the code the data sets are derived

from these sets. Using PBSIM a data set is generated by Alser et al. [26] with a read length of 10K. 10 pairs from the generated data are used for testing.

Data Set Name	**Source**	**Read Length**	**Sample Pairs**	**Edit Distance Threshold**
100_bp_1	https://www.ebi.ac.uk/ena/data /view/ERR240727	100	30000	2
100_bp_2	https://www.ebi.ac.uk/ena/data /view/ERR240727	100	3000000	2
250_bp_1	https://www.ebi.ac.uk/ena /data/view/SRR826471	250	30000	8
FPGA	Created Using Python	250	3000000	40
10K_bp	Alser et al.	10000	10	Default Error Count

Table 4.2: Data Sets used for testing(Read-Reference Pairs)

4.6 Material Needed

The implementations on CPU, GPU, and FPGA all need their own setup environment and apparatus to be implemented. In case of CPU implementation to run the code on any machine C++ environment is needed along with a Integrated Developing Environment for the local implementation as no Command Line Interface is built to support the direct execution of the code. The code is to be run on a Core i5 Intel processor. For FPGA Xilinx Virtex VC707 Evaluation Kit along with Vivado is needed to implement the algorithm. Cloud Implementation simply needs access and capital for the implementation of the algorithm using Virtual Machine on Microsoft Azure.

4.7 Ethical Consideration

The pre-alignment filter implemented is tested using sample data. Despite even accurate results the testing done is not enough to justify the use of implementation for any form of diagnosis and treatment in terms of medicines and medical research. In order for this algorithm to be adopted for such application extensive testing on

more real data and a thorough comparison of industrial standard Pre-alignment filters is needed.

4.8 Analysis

After gathering all the necessary data for conducting comparisons, various forms of visual representations, such as line graphs. These visualizations were employed to analyze the relationships between different variables and draw meaningful conclusions based on the observed trends. By examining the plotted data, valuable insights and findings were derived.

To validate the performance and accuracy of the implementation the results obtained from the implementation were compared with the implementation of Alser et al.[26] in terms of accepted reads, rejected reads, execution time, memory usage, and CPU usage.

A detailed discussion of the results obtained is done in the later chapter.

Chapter 5

CPU Implementation of Sneaky Snake Algorithm

The Sneaky Snake algorithm's design avoids most of the classic problems with Approximate String Matching algorithms. Hence, even though CPUs excel at sequential execution, the Sneaky Snake performs well when implemented. The implementation of Sneaky Snake on the CPU stems from the theoretical foundation discussed in the second chapter. Reduction of the SNR problem, too, as an ASM problem, the algorithm builds the chip maze on the fly and keeps track of the edit distance. The basic philosophy of the Sneaky Snake algorithm remains intact, "if an optimal path maze without exceeding the Edit Distance Threshold exists, the read is accepted". The design and the working principle of the CPU implementation are further elaborated in this chapter.

5.1 Design and Implementation

The design of the implementation is divided into two main parts. There is the main function in which the sneaky snake function is called on different reads and the function with the implementation of the Sneaky Snake algorithm itself. The sneaky snake implementation aims to build a Chip Maze on the fly while passing through it to find an optimal path, ensuring that the Edit Distance Threshold is

not exceeded. If the edit distance threshold is exceeded, then a read is rejected. If not, a read is accepted.

5.1.1 Main Function

- The Data is read from a file and stored in fstream. A loop is used to get the reads line by line from the file, and *strncpy_s* duplicates the read data onto Reference and Query Array.
- The duplicated data is called to the Sneaky Snake function, which either returns a 0 or 1 for Rejected and accepted, respectively, along with read size and Edit Distance Threshold.
- Counters are used to keep track of the total number of reads accepted or rejected
- A high-resolution clock is used to measure the time instance since the start of the program before and after the total data processing.

5.1.2 Sneaky Snake

The Sneaky Snake function itself uses three functions other than the main function:

1. Lower Horizontal Routing Track Function: Responsible for the calculation of the maximum distance that can be traveled without encountering a block in lower rows of the chip maze
2. Upper Horizontal Routing Track Function: Responsible for the calculation of the maximum distance that can be traveled without encountering a block in the upper rows of the chip maze
3. Main Horizontal Routing Track Function: Responsible for the calculation of the maximum distance that can be traveled without encountering a block in the main row of the chip maze

All three functions return an integer value for the longest escape segment. The value with the maximum magnitude is accepted to the longest escape segment and is used to mark the checkpoint.

Every time the checkpoint is updated, the value of propagation delay is incremented, and in case the value exceeds the Edit Distance Threshold, the read is rejected or otherwise accepted accordingly.

Figure 5.1 below shows the hierarchy of the functions, the implementation of Sneaky Snake uses

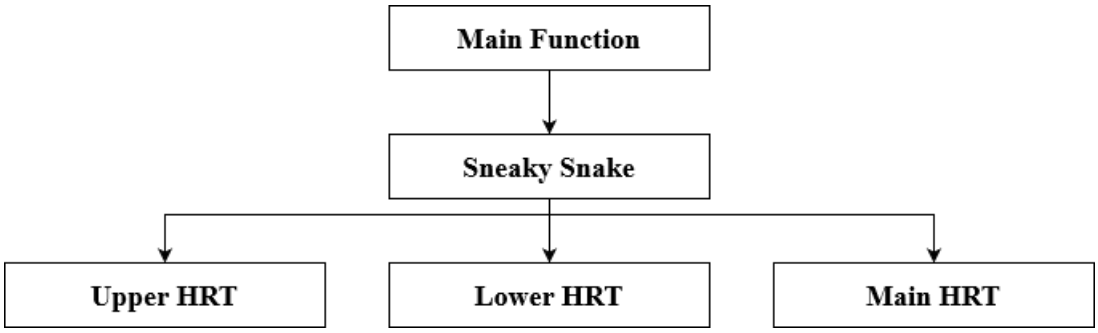

Figure 5.1: Functions Hierarchy of Sneaky Snake CPU Implementation

5.1.3 Input and Output

The following Table 5.1 represents the input and output variables of the Sneaky Snake Implementation on CPU.

Variable Name	Datatype	Input/Output	Contains
readfile	fstream	Input	The read data is contained within this data structure
N	int	Input	Total bases present in a read (read length)
E	int	Input	Edit Distance Threshold
threads	int	Input	No of threads to be used
lines	int	Input	Total number of reads
accepted_sum	int	Output	Total number of reads accepted
rejected_sum	int	Output	Total number of reads rejected

Table 5.1: Input and Output of the Sneaky Snake Implementation on CPU

5.1.4 User Interface

The output of the system is shown on the command prompt on Windows representing the total number of reads accepted, the reads rejected, and other necessary information. The example output window is shown in Figure 5.2

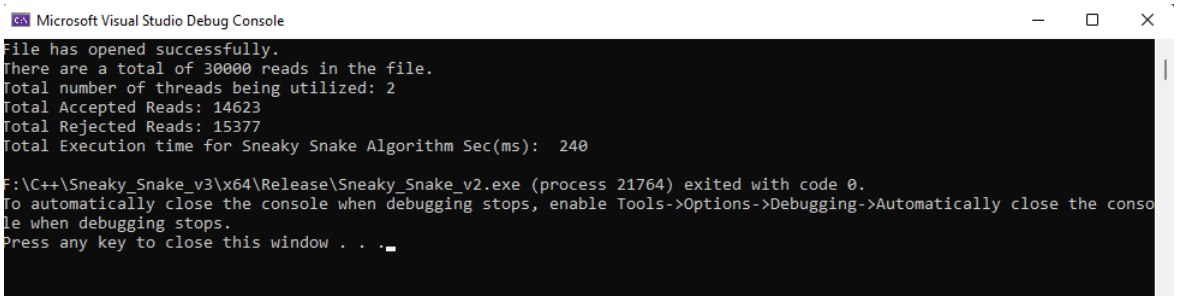

Figure 5.2: Output Windows User Interface

5.1.5 Pseudocode

In this section pseudocodes for the implementation of the Sneaky Snake algorithm are provided. A total of four algorithms are implemented three for the upper, lower, and main rows of the chip maze and one for the main Sneaky Snake function.

The first algorithm presents the implementation of Sneaky Snake's Main algorithm. The pseudocode presented here generalizes the implementation for any CPU and any programming language. The implementation, however, was done using C++ programming language for the testing of the algorithm on the CPU.

There are a total of four algorithms presented:

1. Algorithm 1: Sneaky Snake
2. Algorithm 2: UpperRows
3. Algorithm 3: LowerRows
4. Algorithm 4: MainRows

Algorithm 1 Sneaky Snake Main Function

Input: Reference Read (R), Query Read (Q), Edit Distance Threshold (E)
Output : 0 for rejected read and 1 for accepted reads
Functions Called : UpperRows(), LowerRows, MainRow() **Pseudocode**

Ensure: $checkpoint = 0$
Ensure: $PropagationDelay = 0$
  $N \leftarrow length(Q)$ ▷ Read Length Stored in a variable
  while $checkpoint < N$ & $PropagationDelay < E$ **do**
    $count \leftarrow 0$
    ▷ longest escape segment set as the longest escape segment from Upper rows
    $longest_es_seg \leftarrow UpperRows(Q[checkpoint : N-1], R[checkpoint : N-1], E)$
    $count \leftarrow MainRows(Q[checkpoint : N-1], R[checkpoint : N-1])$
    if $count == N$ **then**
      return 1 ▷ The read is accepted
    end if
    if $count > longest_es_seg$ **then**
      $longest_es_seg = count$
    end if
    $count \leftarrow LowerRows(Q[checkpoint : N-1], R[checkpoint : N-1], E)$
    if $count > longes_es_seg$ **then**
      $longest_es_eg \leftarrow count$
    end if
    $checkpoint \leftarrow checkpoint + longest_es_eg$
    if $checkpoint < N$ **then**
      $PropagationDelay++$
      $checkpoint++$
    end if
  end while
  if $PropagationDelay > E$ **then**
    return 0 ▷ The read is rejected
  else
    return 1 ▷ The read is accepted
  end if

Algorithm 2 UpperRows

Input: Query (Q[checkpoint:N-1]),Reference(R[checkpoint:N-1]), Edit Distance Threshold (E)
Output : lenght of longest escape segment
Pseudocode

Ensure: $longest_es_seg = 0$
  for r= E *to* 1 **do**
    $count \leftarrow 0$
    for n= checkpoint *to* length(Q)-1 **do**
      if $n < r$ **then**
        go to *EXIT*
      else if $Q[n-r]! = R[n]$ **then**
        go to *EXIT*
      else if $Q[n-r] == R[n]$ **then**
        $count++$
      end if
    end for
  end for
  EXIT
  if $count > longest_es_seg$ **then**
    $longest_es_Seg = count$
  end if
  return $longest_es_seg$

Algorithm 3 LowerRows

Input: Query (Q[checkpoint:N-1]),Reference(R[checkpoint:N-1]), Edit Distance Threshold (E)
Output : length of longest escape segment
Pseudocode

Ensure: $longest_es_seg = 0$
  for r= 1 *to* E **do**
    $count \leftarrow 0$
    for n= checkpoint *to* length(Q)-1 **do**
      if $n > N - r - 1$ **then**
        go to *EXIT*
      else if $Q[n+r]! = R[n]$ **then**
        go to *EXIT*
      else if $Q[n+r] == R[n]$ **then**
        $count++$
      end if
    end for
  end for
  EXIT
  if $count > longest_es_seg$ **then**
    $longest_es_Seg = count$
  end if
  return $longest_es_seg$

Algorithm 4 MainRow

Input: Query (Q[checkpoint:N-1]),Reference(R[checkpoint:N-1])
Output : length of longest escape segment
Pseudocode

Ensure: $longest_es_seg = 0$
  for n= checkpoint *to* length(Q)-1 **do**
    if $Q[n]! = R[n]$ **then**
      return $longest_es_seg$
    else if $Q[n] == R[n]$ **then**
      $longest_es_seg = longest_es_seg + 1$
    end if
  end for
  return $longest_es_seg$

5.2 Parallelism

The sequential execution of the code acted as a major bottleneck for the performance of the code. There are a lot of reads that need to be processed and yet only one logical processor of the CPU was being utilized.

Hence a better option here is to actually take multiple reads at the same time and use different threads to deal with them in a concurrent fashion. The need for parallelism was observed due to the following reason:

1. Most of the modern processors come with more than one core hence parallel processing is natively built into modern CPU architecture and can easily be exploited to our aid.

2. If the code was not implemented in a parallel fashion there would be no point in using a CPU with more cores and hope for accelerated performance.

3. Distribution of tasks among threads would take a load of just one thread on the CPU.

4. If the overhead time for thread creation is justified due to the read length or number of reads parallel execution would yield much faster results in comparison to simple sequential execution.

Rather than going into depth of multi-threading and creating a whole new implementation of parallel thread handling and execution, it was decided to base the code on the use of an external package. In the case of this implementation, Open MP was utilized.

5.2.1 OpenMP

OpenMP is an application programming interface supporting loop-level parallelism for multiple architectures and programming languages.

It became popular due to its capabilities of enabling multi-platform parallel programming. The models made using OpenMP are scalable and the API offers a simple and flexible interface.

Instead of using the pthreads library like used in the implementation by Alser et.al [26] the reason for choosing OpenM was simple. The API deals with all the low-level technicalities of the computing platform and programming language on itself and allows seamless concurrency of an application which mostly is more powerful than the personal implementation of parallel execution. This ensured that no time was spent on putting in efforts to recreate the proverbial wheel.

The major work done by the Sneak Snake Code was in the execution of the main loop where data is fed to the algorithm and the results are obtained. The rest of the code was mainly for the printing of data, the opening of files, and other tasks that had no direct relation to the performance of the algorithm itself. Hence OpenMP allowed us to simply take that loop and convert it into a parallel for loop. Data from multiple reads were taken by the API and provided to different threads. Optimization of the code led to an observation that with an ideal number of threads where the overhead had no impact on the code, a better performance was observed. The said loop to be made parallel was enclosed in the curly bracket with the following instantiating of OpenMP being called before the enclosing brackets. The addition of a few variables was done generally and many variables visibility was adjusted accordingly.

```
omp_set_num_threads(no_threads);
        #pragma omp parallel for schedule(static)
```

5.2.2 Implementation Considerations

Parallel implementation has its own limitation hence the coding needs to be done with vigilance and precision to ensure that the concurrent code is actually performing better than the sequential execution.

1. Parallel implementation introduces new problems such as data races and deadlocks which needs to be dealt with while coding. The said implementation uses only loop-level parallelism and the resources needed for the threads to execute are independent of each other hence deadlocks are avoided. To avoid data races the global variable on which the write operation is to be performed commonly is divided into multiple local variables and then the results are added together separately.

2. A parallel program is going to shine when the task at hand actually is intensive enough to be needing it. Hence in this implementation, a higher number of threads actually would result in lower performance for shorter reads as the activity is simply not computationally intensive enough. Hence the number of threads provided to the code is to be adjusted according to the data available to be processed. The golden rule here is that the task must be processor-intensive enough to overcome the parallelization overheads that come packed with using multiple threads.

3. Task granularity needs to be considered. Tasks need to have equal burdens and should be divided amongst just the right number of threads. The division of work between threads is taken care by the use of OpenMP and the thread number is changed according to the data being processed.

4. Memory Management needs to be considered while using OpenMP. The more memory is accessed by a loop, increases the time each thread is going to take.

Hence the implementation is taking care of this by making sure only global accesses are done when absolutely needed otherwise they are not. Most of the variables are localized to the loop so shared memory access is avoided. Despite that, some common variables need to be accessed globally to make sure code is reproducible for different sizes of data. In order to truly make the code efficient, the changes that change the data need to be made in arguments and are hard coded into the application so that global memory accesses are limited.

5.3 Hardware Used

The hardware used for implementing Sneaky Snake Algorithm is Intel(r) Core(tm) i5-10210u CPU @ 1.60 GHz. Table 7.1 below summarizes the characteristics of the hardware.

Characteristic	**Specification**
Cores	4
Logical Processors	8
Maximum Turbo Frequency	4.20 GHz
Base Frequency	1.60 GHz
Cache	6MB Intel Smart Cache
Memory Types	DDR4-2666, LPDDR3-2133, LPDDR4-2933

Table 5.2: Intel® Core™ i5-10210U Processor Specifications

5.4 Software Requirements

In order for the code to function an integrated development environment is needed to make sure any changes that need to be made can be implemented. The IDE used for the CPU implementation is Visual Studio Enterprise. Implemented in a Windows environment using Windows 11 the code was written in C++ 14. C++ 14 comes by default with Visual Studio Enterprise. Along with that in order to

run a multi-threaded application this version of C++ is best suited as the previous version did not support OpenMP or threading in itself.Figure 5.3shows the IDE used to run and test the code. Visual Studio Enterprise also offers code profiling tools that can be directly used to measure the CPU Utilization and Memory Usage of the code written. Figure 5.4 shows the profiling tools provided by Visual Studio Enterprise that are used to test the code and analyze the performance of the algorithm.

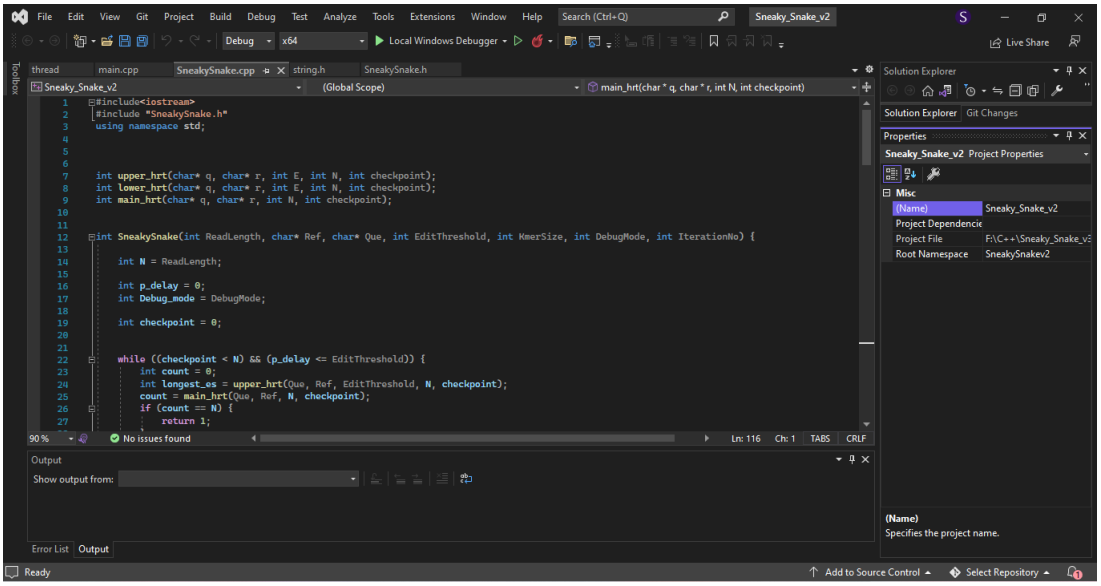

Figure 5.3: Visual Studio Enterprise

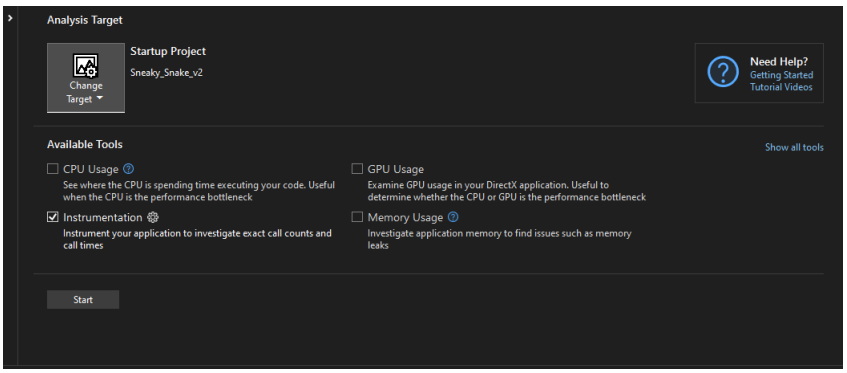

Figure 5.4: Visual Studio Enterprise Performance Profiler

Chapter 6

FPGA Implementation of Sneaky Snake Algorithm

6.1 Design and Architecture

Sneaky Snake algorithm implemented on the CPU worked on the basis of utilizing the capabilities of parallelism of a CPU and dividing the workload among multiple threads hence resulting in accelerated performance. In the case of FPGA, the same algorithm could not be used as it is for implementation. The reasons why the algorithm as implemented on CPU could not be implemented on FPGA are as follows:

- Dynamic Memory Allocation: Memory is allocated dynamically using pointers ('char*') within the code. Memory resources on FPGAs are limited, and dynamically allocating memory at runtime is not straightforward. Memory is pre-allocated and determined during the design phase in the majority of FPGA designs. Adapting the code to use static memory allocation would necessitate a thorough evaluation of memory requirements and modifications to the design.

- Complex Flow Control: The algorithm in the code is comprised of nested loops, conditional statements ('if' and 'goto'), and function calls. FPGA

designs are founded on parallelism and pipelining, in which concurrent operations are performed. Adapting the code's complex control flow to a parallel and pipelined FPGA architecture may necessitate extensive algorithm and control logic restructuring.

- Limited Support for Floating-Point Operations: Typically, FPGAs offer less support for floating-point operations than CPUs. If there are any floating-point calculations in the algorithm, they may need to be approximated or implemented using fixed-point arithmetic on the FPGA, which is not expressly included in the code you provided.

- Iterative Nature: The algorithm comprises iterative components, such as the 'while' loop and nested 'for' loops. Typically, FPGA designs prioritize parallelism and pipelining over iterative execution. Adapting the iterative nature of the code to efficiently exploit parallelism on an FPGA can be a difficult undertaking.

- Timing Closure: To assure proper functionality, FPGA designs must adhere to specific timing requirements. The algorithm's critical path must be meticulously analyzed and optimized so that all operations can be completed within a single clock cycle. Timing closure can be difficult to achieve, particularly when working with complex algorithms or limited hardware resources.

- Memory Access: The CPU-based implementation involves reading the data from a file and working on it. As an FPGA has no Physical Layer or Microprocessors hence no file system inherently exists to use in implementation.

To successfully implement the algorithm on an FPGA, it would be necessary to analyze and comprehend its requirements, constraints, and performance objectives. This analysis would guide the design and implementation process, which entails translating the algorithm into a hardware description language, optimizing for resource utilization and timing closure, and taking into account the FPGA

platform's specific constraints. Hence the major considerations while building the same algorithm for a pre-alignment filter on an FPGA are as follows:

- Instead of making the Chip maze on the fly, in FPGA-based Sneaky Snake the Chip Maze is constructed beforehand.
- Instead of using different threads to work on different DNA fragments, a single fragment is divided into sub-problems and solved in a parallel fashion to decide whether to accept it or not
- Direct comparison between the subproblems is made. Comparisons between data are something where FPGA outperforms CPU in terms of performance.
- The data needs to be coded into binary code in order for it to be provided to the FPGA.
- The implementation is specific to the Edit Distance Threshold decided beforehand and hence is not as flexible as the one done on CPU. Mainly, as the CPU-based implementation uses the power of iteration to allow full customization of the parameters no such feature exists in FPGA development. The implementation performed is based on an Edit Distance threshold of 5.

In conclusion, the CPU-implemented Sneaky Snake algorithm cannot be directly ported and implemented on an FPGA for multiple reasons, including dynamic memory allocation, complex flow control, limited support for floating-point operations, the algorithm's iterative nature, timing closure challenges, and the lack of a file system on the FPGA. The FPGA implementation requires a thorough examination of the algorithm's requirements, constraints, and performance goals, followed by the algorithm's translation into hardware description language (HDL) and optimization for resource utilization and timing closure. The FPGA-based implementation of the algorithm requires the construction of the labyrinth in advance, the division of the problem into subproblems for parallel processing, direct binary comparison, and a specific Edit Distance Threshold. It offers benefits such

as high parallelism, data processing efficiency, and hardware acceleration, making it suitable for certain applications, but it requires careful consideration and algorithm adaptation to the FPGA architecture.

6.1.1 Design Philosophy

The FPGA-based implementation of the Sneaky Snake algorithm is designed using a bottom-up approach. As explained in Figure 6.1 the chip maze is constructed for data of 100 base pairs. Chipmaze is constructed using Shifted Hamming Distance Map and the generated Chip Maze is provided to the Leading Zero Counters which act at multiple levels to provide the longest escape segment. The 8-bit Sneaky Snake algorithm takes the longest escape segment and updated the number of Edits. The number of edits is used to decide whether to accept or reject a DNA fragment.

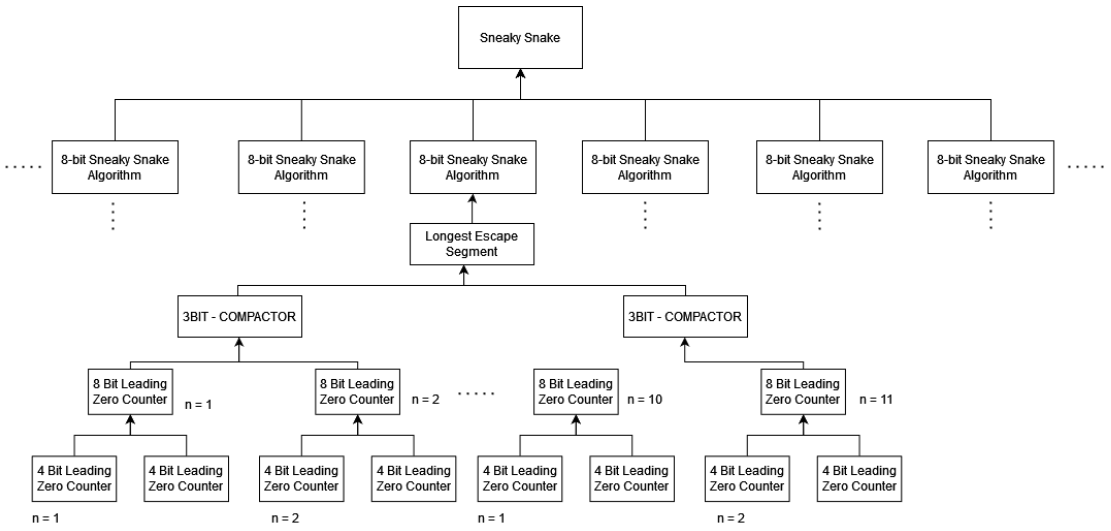

Figure 6.1: Sneaky Snake Architecture FPGA

As shown in Fig 6.1 two 4-bit Leading Zero Counters make up an 8-bit Leading Zero Counter. The Outputs of two 8-bit Leading Zero Counters are fed to a comparator. There are 11 instances of 8-bit Leading Zero Counter and their outputs are fed to 6. The output of these comparators is further provided as input to level two comparators. The largest value is returned to the longest escape segment from which the 8-bit sneaky snake instances take the value. Multiple

instances of 8-bit Sneaky Snake are fed to the top-level Sneaky Snake Instance to divide whether to accept or reject a DNA sequence.

In FPGA design, a bottom-up method has many advantages. First of all, it enables gradual and modular development, allowing designers to concentrate on particular functional blocks and individually optimize them. Because each module can be created, examined, and verified individually before being incorporated into the broader system, this method encourages reuse and scalability. Additionally, since problems may be identified and fixed at the module level, it promotes a simpler debugging and verification process.

6.1.2 Block Diagram

The FPGA design on the Sneaky Snake algorithm simply provides a means to implement the algorithm in an efficient manner on the FPGA. However, utilizing it on physical hardware requires multiple other components added to the design. The block diagram presented in Fig presents the complete FPGA design for the implementation of the Sneaky Snake algorithm on the FPGA board (Virtex-VC707).

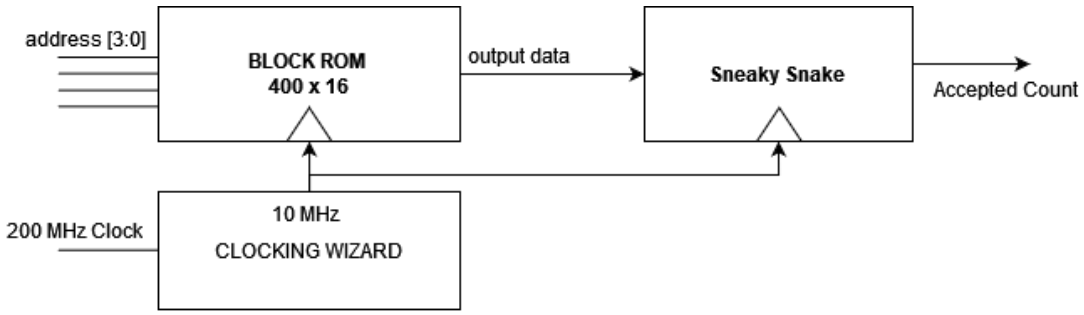

Figure 6.2: Block Diagram Sneaky Snake Algorithm

The Sneaky Snake Module, as described in the preceding section, operates by receiving input data from a Block ROM in the form of a 400-bit string. This string is then split into two equal parts, each consisting of 200 bits. The module processes these parts and generates an output that represents the total count of accepted queries. This output is displayed on the LEDs integrated into the FPGA.

To facilitate the functioning of the module, the Block RAM relies on addresses obtained from the DIP switches available on the Virtex-VC707 board. These addresses are used to access the necessary data stored in the Block RAM.

Furthermore, the Sneaky Snake Module employs a clocking wizard to handle the clock signals. It takes in a 200 MHz clock as an input and generates a 10 MHz clock signal as an output. This clock signal is crucial for ensuring the proper synchronization and timing of the module's operations.

6.2 Theoretical Basis

Sneaky Snake is a complicated algorithm that needs to be modified in order to be implemented on an FPGA. In this section, the theoretical basis of the Sneaky Snake Algorithm is explored.

The work on Sneaky Snake comes around to the identification of the longest escape segment. In the case of the CPU, this was done by constructing the Chip Maze on the fly and incrementing the Edit Distance until the threshold was met. In the case of FPGA, this approach is not possible as explained earlier. Hence the whole Chip Maze is constructed beforehand. The availability of a large number of Lookup Tables and massive bitwise parallelism allows the construction of chip maze easily. Hence implementing the SNR problem or the Sneaky Snake in an FPGA-friendly manner stems from dividing the problem into smaller problems that are non-overlapping. In the case of this project, the total length of the data to be tested was 100 and an error threshold of 5 was decided.

The larger problem of 100 base pairs was divided into subproblems of 8 base pairs. Hence the total subproblem is 13. Each subproblem has a general width of Vertical Routing Tracks and a height of 2E + 1. Each of the subproblems is solved independently from other subproblems. The advantages this provides are a reduction in search space and hence a lower resource utilization, a better design approach that is scalable and modular, and high parallelism.

The general architecture of Snake-on-Chip consists of the following modules:

Chip Maze Construction The entire chimpanzee is constructed beforehand for each subproblem. In terms of specifications each chip maze is going to have 2E + 1 rows and each row will be t bits long. In the case of this project 11 rows and each row is 8 bit long. In contrast to the CPU implementation, the FPGA implementation of the SneakySnake algorithm exploits the parallel processing capabilities inherent to FPGA chips. This enables extensive bitwise parallelism by permitting the concurrent computation of all bits of all bit-vectors in advance. In contrast to a CPU core, which executes instructions sequentially, an FPGA can simultaneously perform computations on multiple data elements. This parallel processing capability enables us to compute each row's entries without being restricted to those between a checkpoint and the first subsequent obstacle. Instead, we can simultaneously process all bit-vectors by leveraging the FPGA architecture's enormous parallelism.

By leveraging this parallelism, the FPGA implementation can compute and manipulate the bit-vectors in a parallel manner, substantially enhancing the efficacy and speed of the execution of the SneakySnake algorithm. This parallel processing approach is a fundamental departure from the sequential execution of instructions on a CPU core, making the FPGA implementation highly suitable for scenarios where massive bit-wise parallelism is desirable.

Leading Zero Counter The leading zero counter is based on [37] which computed the first horizontal routing track by counting the maximum number of zeros. The LZC of choice is based on a low utilization model hence easily integrating with the sneaky snake implementation.

Longest Escape Segment The HRT with the largest number of leading zeros is calculated using comparators. The number of comparators depend on the number of rows and the number of levels is derived by

$$\log_2 (2E + 1)$$

After finding the longest escape segments new checkpoint is established and the process is repeated to find the next longest escape segment. This is done by using bit shifting to change the chip maze. If the longest escape segment was x bit long then the shifting is done for x + 1 bits. This ensures no bit is repeated again. Next optimal escape segment is calculated starting from the least significant bit. The number of instances needed depends on the accuracy we are desiring.

The number of instances needed is y and once the whole process is completed for each sub-problem the maximum number of obstacles is calculated using:

$$\min(y, \sum_{k=1}^{y} x_k)$$

In Summary, when implemented on an FPGA, the Sneaky Snake algorithm must be modified to take advantage of the parallel processing capabilities of FPGA processors. In this method, the chip maze for each subproblem is constructed beforehand, allowing for enormous bitwise parallelism. The algorithm divides the primary problem into smaller, nonoverlapping subproblems, thereby reducing the search space and allowing for scalability and high parallelism. To compute the optimal escape segments, the FPGA implementation employs modules such as Chip Maze Construction, Leading Zero Counter, and Longest Escape Segment. The algorithm finds the longest escape segment iterative, updates the chip maze, and repeats the procedure until all subproblems are solved. The query is rejected if the cumulative number of obstacles exceeds a predetermined threshold; otherwise, it is accepted. Utilizing the parallel processing capabilities of FPGA chips, this FPGA-friendly implementation significantly enhances the Sneaky Snake algorithm's efficiency and speed.

6.2.1 Proposed Solution

The implementation of the Sneaky Snake algorithm on FPGA is limited to a certain edit distance threshold and a certain number of base pairs in the data. For the scope of this project, the following factors are decided.

- **Edit Distance Threshold**: 5
- **Number of Base Pairs**: 100

The algorithm for the Sneaky Snake Implementation for a data set containing 100 base pair long row and a total of 16 rows on FPGA is provided below.

1. Construct the chip maze with dimensions E = 5 and t = 100, resulting in a maze of size 100 x 11.
2. Divide the main problem into 13 subproblems, each consisting of 8-bit rows. Each sub-problem has 11 rows in total.
3. Feed each bit vector of each subproblem into an 8-bit Leading Zero Counter to compute the length of the first horizontal segment of consecutive zeros.
4. Use a 4-level comparator network to find the longest escape segment. The network compares the outputs of the Leading Zero Counters and identifies the bit vector with the largest number of leading zeros.
5. Update the chip maze by shifting the bits to the right by x + 1 positions, where x is the length of the longest escape segment found in the previous step. This shifting ensures that the previously found escape segment and one obstacle are excluded from the new search round.
6. Repeat steps 2, 3, 4, and 5 to find the next longest escape segment, starting from the least significant bit (new checkpoint) and progressing towards the most significant bit.
7. Once all hardware instances have produced their output, calculate the total number of obstacles by summing up the lengths of all escape segments found in each sub-problem. If the total number of obstacles is greater than 5, reject the query; otherwise, accept it.

In the FPGA implementation of the Sneaky Snake algorithm, the input data cannot be directly sent as characters. Instead, it needs to be encoded into binary

form to be processed by the FPGA. To accomplish this, a specific mapping is used to represent each base pair as a binary code:

- **Base Pair A**: 00
- **Base Pair T**: 01
- **Base Pair G**: 10
- **Base Pair C**: 11

By applying this mapping, the input data consisting of DNA sequences is encoded into binary form, allowing the FPGA to process and manipulate the data using its parallel processing capabilities. This binary representation enables efficient bitwise operations and parallel computations, which are essential for the FPGA implementation of the Sneaky Snake algorithm.

The data is stored in the Block RAM before synthesis using a **.cof** file and hence is simply retrieved by providing the relevant address during the execution. A total of 16 different DNA fragments each of 100 base pair are texted with three of them having an edit distance threshold lower than 5.

6.2.2 Mathematical Foundation

The implementation of an algorithm as complex as Sneaky Snake on an FPGA means breaking the algorithm down to its basic Boolean mathematics and building upwards from that point inwards. This section explored the mathematical foundation of the Sneaky Snake algorithm.

6.2.2.1 4-Bit Leading Zero Counter

The 4-Bit Leading Zero counter is the most basic and integral part of the Sneaky Snake Algorithm for FPGA. The LZC is derived from Dimitrakopolous [37] proposed LZC design for an 8-Bit Leading Zero counter. We attempt to build a 4-Bit LZC working on the same principles.

Leading Zero Counter is a procedure of encoding the binary representation of numbers in terms of consecutive zeros that appear before the most significant bit. Assuming we have n bit number A represented as:

$$A = A_0\ A_1\ A_2 \ldots A_n$$

where A_n is the most significant bit. This will produce $log_2 n$bits of leading zeros count Z and a flag V that denotes all zero cases for the input A. A method to determine the leading zeros proposed is a two-step process where. The Leading Zero Count is Calculated The rest of the digits are turned to 0

$$00110100 \rightarrow 00110000$$

This representation is referred to as One Hotspot Representation. The same can be done with a slight change. The remaining numbers can be turned into 1 rather than 0

$$00110100 \rightarrow 00111111$$

For a binary number S, its ith bit is denoted as

$$S_i = A_{n-1} + A_{n-2} + \ldots + A_i \tag{6.1}$$

where + represents the logical or operation. In any case when $S_i = 1$ would mean existence of one or more bit that is equal to logic 1 between the most significant bit or the least significant bit. The One Hotspot representation of leading digital (L word) is represented as and determined by:

$$L_i = S_{i+1} + A_i = S'_{i+1}.S_i \tag{6.2}$$

where $0 \leq i \leq n-2$ and . represents AND Operation and + represents OR Operation. This is referred to as a priority encoder. A priority encoder is a digital

circuit that transforms multiple binary inputs into a single binary output that represents the input with the highest priority. It is frequently employed in digital systems to manage multiple inputs and determine which input has the highest priority or is active. The fundamental operation of a priority encoder is to detect and encode the first active (logic high) input. The position or priority of the active input is represented by the binary code, with the least significant bit (LSB) indicating the highest priority.

Once the L word is provided to a priority encoder it will encode the leading zeros to weighted binary representation hence providing us with an encoded decimal number for the count of leading zeros.

The aim is to construct a 4-Bit Leading Zero Counter. Hence the maximum number of leading zeros we can have is 3. This L-word representation of the leading zero counts can be done using 3 bits which is the size of the L word. The following Coding will be used.

L (2 1 0)			**Leading Zeros**	**Binary (Z_0 Z_1)**	
1	0	0	1	1	0
0	1	0	2	0	1
0	0	1	3	1	1

Table 6.1: Priority Encoder Truth Table

Based on Table 6.1 the equation for the priority encoder are

$$Z_0 = L_2 + L_0 \tag{6.3}$$

$$Z_1 = L_1 + L_0 \tag{6.4}$$

Using Equation 7.2, Equations 7.3 and 7.4 can be written as

$$Z_0 = (\overline{S_3}.S_2) + (\overline{S_1}.S_0) \tag{6.5}$$

$$Z_1 = (\overline{S_2}.S_1) + (\overline{S_1}.S_0) \tag{6.6}$$

An important observation here is that S is monotonically increasing hence it will never take the value of 10. Mathematically, (S_i, S_j) where $i > j$ can never be (1, 0).

The only possible values are 00, 01, and 11. Based on this the following four relations can be defined

$$S_i.S_j = S_i \tag{6.7}$$

$$S_i + S_j = S_j \tag{6.8}$$

$$\overline{S_i}.\overline{S_j} = \overline{S_j} \tag{6.9}$$

$$\overline{S_i} + \overline{S_j} = \overline{S_j} \tag{6.10}$$

Hence using the equation provided above we can further simplify the equations.

$$Z_1 = (\overline{S_2}.S_1) + (\overline{S_1}.S_0)$$

$$Z_1 = \overline{S_2}.S_1 + \overline{S_1}.\overline{S_1}.S_0$$

$$Z_1 = \overline{S_2}.(S_1 + \overline{S_1}.S_0)$$

$$Z_1 = \overline{S_2}.(S_0) \tag{6.11}$$

For the case of Z_0 we can follow the same process forming

$$Z_0 = \overline{S_3}.S_2 + \overline{S_1} \tag{6.12}$$

Let's decide to set these bits equal to 1 when the flag for all zero cases is true. Hence $V = S_0 = 1$. The equations are changed to the following:

$$Z_1 = \overline{S_2}.(S_0) + \overline{S_0} = \overline{S_2} \tag{6.13}$$

$$Z_0 = \overline{S_3}.S_2 + \overline{S_1}.S_0 + \overline{(\overline{S_3}.S_2) + \overline{S_1}} \tag{6.14}$$

The next step is adding the actual input in equation 7.13 and 7.14 which provide te following two equations.

$$Z_0' = A_3 + [\overline{A_2}.A_1] \tag{6.15}$$

$$Z_1' = A_3 + A_2 \tag{6.16}$$

For any four-bit number these equations can directly be implemented using Verilog to get a binary-coded output for the number of leading zeros.

6.2.2.2 Re-purposing the 8bit LZC to 4bit LZC

For the particular problem at hand the input for each leading zero counter is going to have 8-bit. Hence, using the 4-bit comparator the eight-bit comparator as established by Dimitrakpoulos [37] can be constructed by using the following concept. The eight-bit number can be represented as two parts. Each goes to a different 4-Bit leading zero counter.

Low Part: $b_0b_1b_2b_3$

High Part:$b_4b_5b_6b_7$

The output of low and high 4-Bit LZC can be denoted as Z_n4low and Z_n4high and a Valid output for both.

The equations for the eight-bit Leading Zero Counter are given as

$$Z_0 = Z_{04low} + Z_{04high}.\overline{validlow} \tag{6.17}$$

$$Z_1 = Z_{14low} + Z_{14high}.\overline{validlow} \tag{6.18}$$

$$Z_2 = validlow \tag{6.19}$$

6.2.2.3 Hierarchical Comparator

A 3-bit hierarchical comparator compares two 3-bit binary numbers and determines their relative magnitude. The output of the comparator is a set of three Boolean equations representing the comparison results for each bit position. Here's the Boolean equation for a 3-bit hierarchical comparator:

Let's assume the two 3-bit numbers to be compared are $A = A2A1A0$ and $B = B2B1B0$, where A2, A1, A0, B2, B1, and B0 represent the individual bits.

The Boolean equations for the 3-bit hierarchical comparator are as follows:

$Y2 = A2 > B2$

$Y1 = (A2 = B2) \wedge (A1 > B1) \vee (A2 > B2)$

$Y0 = (A2 = B2) \wedge (A1 = B1) \wedge (A0 > B0) \vee (A2 > B2) \vee (A2 = B2) \wedge (A1 > B1)$

In these equations, "¿" represents the greater-than comparison, "=" represents the equality comparison, and "&" represents the logical AND operation. Each output equation compares the corresponding bits of the two numbers and produces a result based on the relative magnitudes of those bits.

6.3 Implementation

The implementation of the Sneaky Snake algorithm followed a meticulous bottom-up methodology, ensuring that each component was rigorously tested before being incorporated into larger modules. This methodical approach enabled comprehensive verification and validation at each stage of the development process. By con-

centrating on testing and validating each component separately, potential issues and errors were uncovered early on, laying the groundwork for the algorithm's subsequent integration and assembly. This methodical approach not only improved the implementation's overall reliability and robustness but also facilitated efficient troubleshooting and refining, resulting in the successful integration of the algorithm on the target platform.

6.3.1 Hardware and Software

The following hardware and software were used for the implementation:

1. AMD Virtex 7 FPGA VC707 Evaluation Kit
2. Vivado 2021.2
3. Model Sim
4. Python

The FPGA of choice for the Sneaky Snake algorithm implementation was the Virtex 7, primarily due to its ample resources that could effectively handle the project's requirements. During the initial stages of development, ModelSim was utilized to rapidly test and validate the code, allowing for quick iterations and bug fixes. However, for major development tasks, Vivado 2021.2 emerged as the primary tool, offering a comprehensive suite of features and optimizations specific to FPGA designs. Additionally, Python played a crucial role in the project, serving as a valuable tool for data conversion and refinement, enabling efficient processing and manipulation of the input data. Overall, this combination of FPGA hardware, simulation tools, and programming languages contributed to a successful implementation of the Sneaky Snake algorithm.

6.3.2 Implementation Process

The implementation process followed a systematic approach, starting with the development of the chip maze printing module and the 4-bit Leading Zero Counter.

Each component was thoroughly tested at every stage to ensure accuracy and functionality, laying a solid foundation for the subsequent modules. The Sneaky Snake algorithm code was meticulously verified and tested to guarantee its desired output. In order to optimize the design, the IP Core catalog of Vivado was utilized to create a block RAM and a clocking wizard. Python played a crucial role in converting the dataset into binary format, which was then saved as a .coe file for testing purposes. To validate the functionality of each module and the overall implementation, comprehensive test benches were written and simulations were conducted. This rigorous testing approach ensured the reliability and effectiveness of the Sneaky Snake algorithm on the FPGA platform.

6.3.3 Shifted Hamming Distance Map/ Chip Maze

The module "ChipMaze" is an implementation of the Shifted Hamming Distance Map Maker or in other words the Chipmaze Maker. It is responsible for generating a chip maze pattern. The module takes various inputs such as a clock signal (clk), DNA sequences for reading and reference (dna_read and dna_ref), and produces multiple outputs representing different sections of the maze (dna_main, dna_up_1, dna_up_2, dna_up_3, dna_up_4, dna_up_5, dna_down_1, dna_down_2, dna_down_3, dna_down_4, dna_down_5).

The implementation works using the following approach: Inside an always block, the module performs several operations to generate the desired outputs. It calculates the XOR ($\hat{}$) of the dna_ref and dna_read sequences, storing the results in various registers (DNA_1 to DNA_11). Then, using a loop, it iterates through the DNA sequences and performs comparisons to determine the values of the output registers. The output is a shifted hamming distance mask which is later used as an input to the Sneaky Snake Algorithm.

The module utilizes the clock signal to synchronize its operations and ensure that the outputs are updated at each rising edge of the clock. By following a bottom-up approach, testing each component individually, and integrating them

into a larger module, the implementation ensures the accuracy and functionality of the final output.

This ChipMaze module, along with its sub-modules and the overall implementation of the Sneaky Snake algorithm, was thoroughly verified and tested for accuracy to meet the requirements of the project. Fig 6.3 shows the inputs and outputs to the Chip Maze Module.

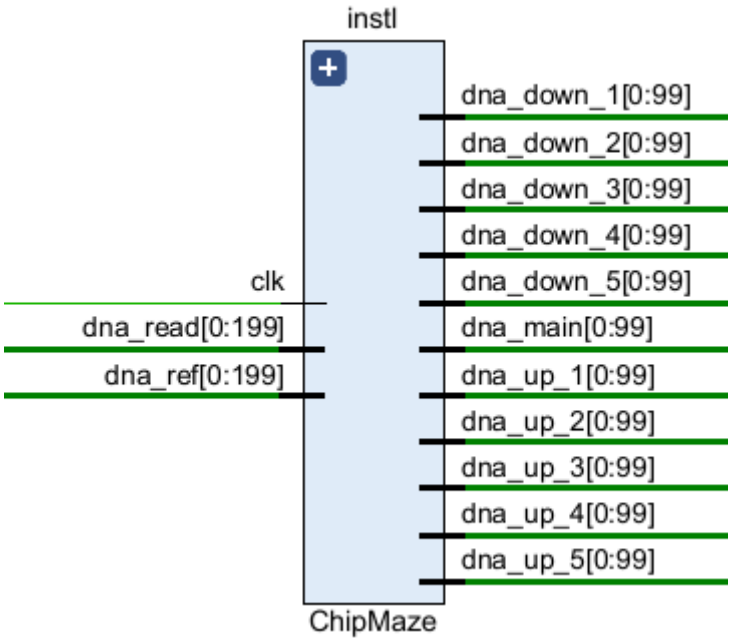

Figure 6.3: Chip Maze Schematic

6.3.4 Leading Zero Counter 4 bit

The Leading Zero Counter 4-bit as explained in Mathematical Foundation works through the implementation of Equation 7.15 and 7.16. The schematic diagram of the 4-Bit LZC is attached below. The implementation is purely done at the hardware abstraction level. In a single sub-problem solution, there are instances of a 4-Bit Leading Zero Counter Called.

6.3.5 Leading Zero Counter 8 bit

An 8-Bit Leading Zero Counter is formed by instantiating the 4-Bit Leading Zeros Counter modules and connecting their outputs according to Equations 7.17, 7.18,

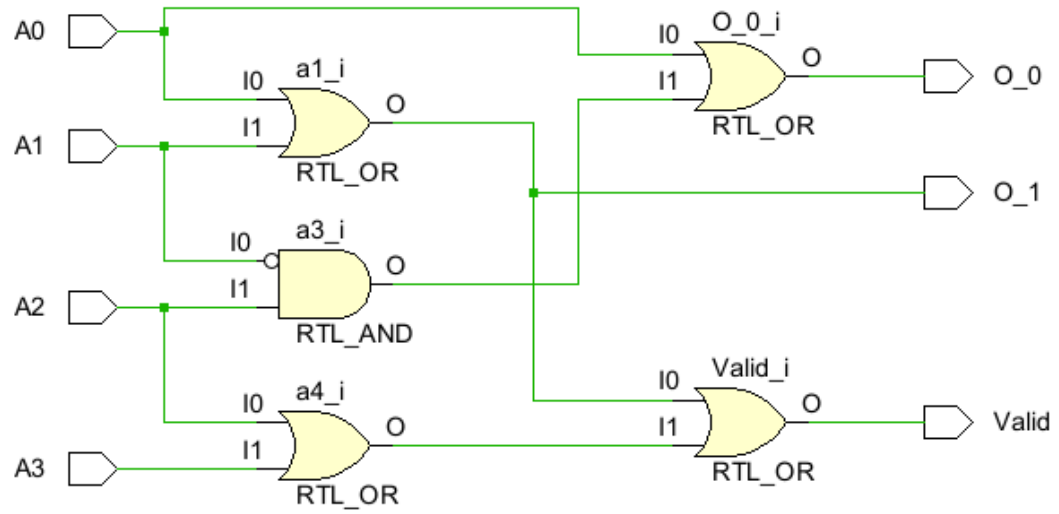

Figure 6.4: 4 Bit Leading Zero Counter

7.19. The attached schematic below shows the formation of the 8-Bit Leading Zero Counter.

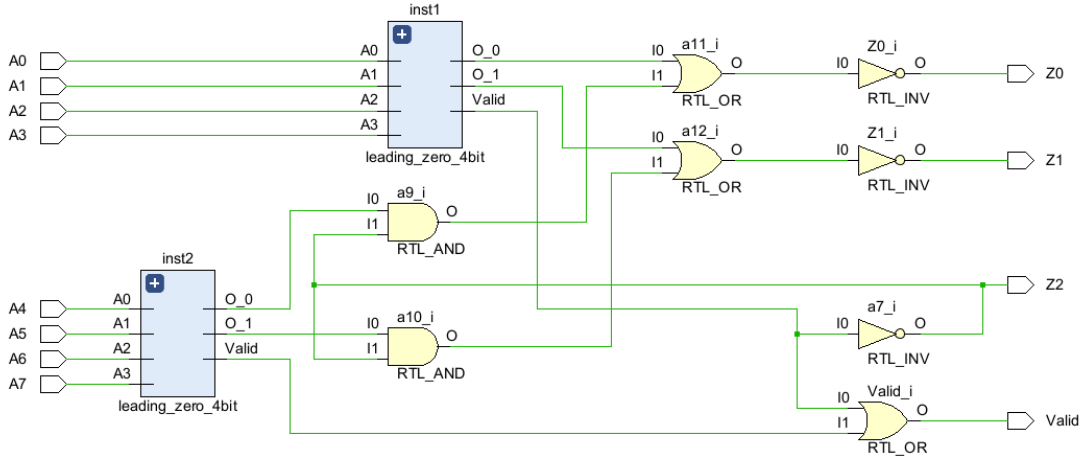

Figure 6.5: 8-Bit Leading Zero Counter

6.3.6 Comparator 3 bit

The Hierarchical Three Bit comparators take the input from the 8-Bit Leading Zero counter and provide an output of the value that is greater. Multiple levels of comparators are used. In the case of the specification of the project, there are four levels of comparators.

- Level 1: 6 Comparators
- Level 2: 3 Comparators
- Level 3: 2 Comparators
- Level 4: 1 Comparator

The schematic of the comparator is attached below.

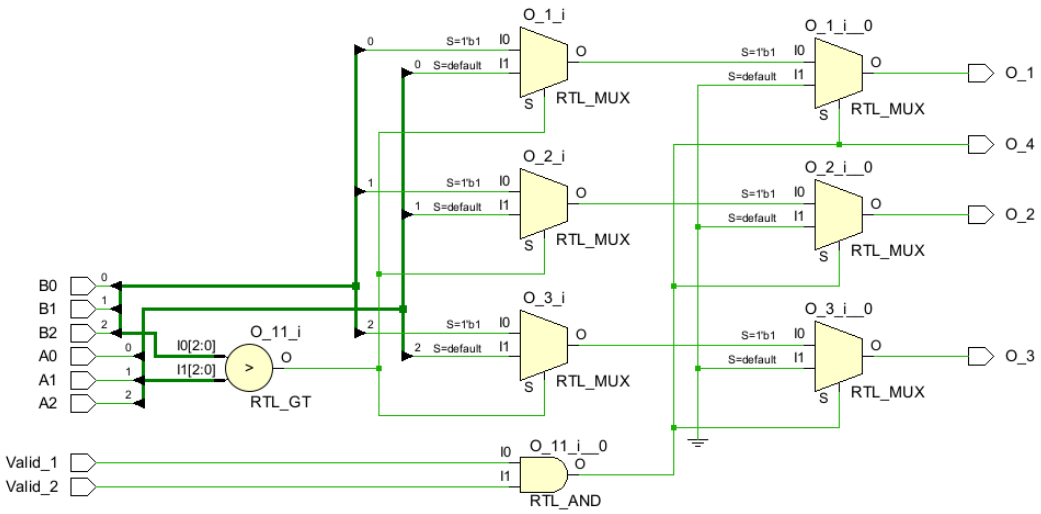

Figure 6.6: 3 Bit Comparator

6.3.7 Longest Escape Segment

The longest Escape Segment Module is which implements a logic circuit to find the longest sequence of leading zeros in a series of DNA strands. The module takes in multiple DNA sequences as inputs and generates four outputs, L0, L1, L2, and L3, which represent the length of the longest sequence of leading zeros found in each input sequence. The module uses a series of leading zero counters to

count the number of leading zeros in each input sequence. These counts are then fed into a hierarchy of comparator modules to compare and determine the longest sequence of leading zeros among the inputs. Finally, the result is obtained as an output through the L0, L1, L2, and L3 signals.

6.3.8 8-bit Sneaky Snake Module

The module takes inputs representing DNA strands and calculates the longest sequence of leading zeros among them. It uses a sub-module called "longestescape" to perform the computation and stores the result in four wires. Based on the calculated number of leading zeros, the module determines the outputs for the DNA strands, the sum of inputs, the edit output, and the number of lives. The module also includes conditional statements to handle different cases and update the outputs accordingly. Overall, the code performs the logic operations necessary to analyze DNA strands and produce corresponding outputs based on the longest sequence of leading zeros.

6.3.9 Top Sneaky Snake Module

The top module calls the Chip Maze module along with 37 instances of the 8-Bit Sneaky Snake Module to decide whether to accept or reject the input DNA reference. The Sneaky Snake main module is called within a top-level module where it receives the data from Block ROM and the clock from the clocking wizard and provides the output as the total accepted count. The schematic in Fig 6.7 shows the whole implementation interconnected with IP cores and low-level modules.

6.3.10 Block Read Only Memory

There were some limitations imposed by the availability of IP cores and GPIO on Virtex VC 707. After a thorough evaluation of the choice at hand the method to provide Sneaky Snake access to data was decided to be a Single Port Block Read

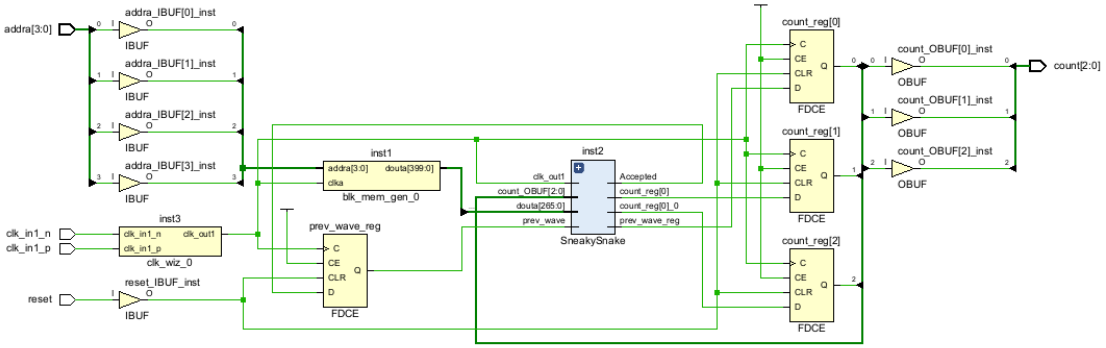

Figure 6.7: Complete Schematic

Only Memory. The BROM was generated using the Block Memory Generator IP Core.

The characteristics of the generated memory were:

- **Memory Type**: Single PORT ROM
- **Block RAM resources (18 KB RAM)** : 0
- **Block RAM resources (36 KB RAM)** : 6
- **Data Width** : 400
- **Data Depth** : 16

Using a COE File the contents of the BROM were initialized and provided to the Sneaky Snake algorithm.

6.3.11 Clock

The clock decided for the implementation was 10 MHz derived from a 200 Mhz clock on the FPGA. Clocking Wizard was used to generating the clock with specific requirements and directly called in the top module.

6.3.12 General Purpose Input Output (GPIO)

The following GPIO was used to provide input to memory and LED to display results.

1. 4 DIP Switch for address

2. 3 LEDs to display output

6.4 Data Modification and Generation

The data as used in CPU implementation was useless for the FPGA implementation. Hence it needed to be coded into a format that could be handled by the FPGA. For the sake of testing a total of 16 DNA sequences were taken and fed into a Python code which mapped the relevant base pair with its binary code saving the output into a file. 13 values along them had an Edit Distance of more than or equal to 10 and three of them had an Edit Distance of lower than 5. The data was then saved into a .coe file which was used to feed it into the Single Port Block ROM during the development phase.

6.5 Hardware Used

The Sneaky Snake algorithm was tested and implemented using Xilinx Virtex VC 707 FPGA. Virtex-7 VC707 houses the Virtex-7 XC7VX485T FPGA (Field-Programmable Gate Array). It was created by the industry-leading manufacturer of FPGAs and programmable logic devices, Xilinx. The VC707 board offers a platform for high-performance digital system prototypes and development, with applications in industries like aerospace, defense, communications, and industrial automation.

Figure 6.8: Xilinx Virtex VC-707 FPGA Evaluation Kit

6.5.1 Port Mapping

Detailed resource utilization can be found in the results section. The table below shows the inputs and outputs of the implementation and their respective port references.

Variable Name	Input/Output	IOSSTANDARD	PACKAGE PIN	TYPE OF COMPONENT
addra[3]	Input	LVCMOS18	AV30	DIP SWITCH
addra[2]	Input	LVCMOS18	AY33	DIP SWITCH
addra[1]	Input	LVCMOS18	BA31	DIP SWITCH
addra[0]	Input	LVCMOS18	BA32	DIP SWITCH
reset	Input	LVCMOS18	BB31	DIP SWITCH
count[0]	Output	LVCMOS18	AR37	LED
count[1]	Output	LVCMOS18	AM39	LED
count[2]	Output	LVCMOS18	AN39	LED

Table 6.2: Port Mapping

6.6 Software Used

The implementation was done using Verilog. The code was written and simulated in Vivado 2021.2. Vivado design suit allowed the utilization of IP cores including Block Memory Generator and Clocking Wizard to streamline the development process allowing us to save time and energy.

6.7 Limitations

The CPU-based implementation was very flexible in terms of modification. Users could enforce any and all parameters that they wanted to do without the need to change or modify the working code. However, as explained earlier in the case of the FPGA the implementation is a lot more specific. In the case of the implementation done this is only valid for a data query that has 100 base pairs and will be accepting only the values that are within the Edit Distance Threshold of 5. In case one wants to modify it for a different data set the code needs to be changed and built from the ground. The contribution we make is the availability of modular instances that can be easily used to build the Sneaky Snake for any type of data and Edit Distance

simply by following a bottom-up approach after dividing the main problem into multiple sub problems.

The testing was limited due to the lack of availability of licenses in Pakistan for IP cores for Ethernet and UART which would allow us to use Vitis SDK to develop a user interface with the on-chip implementation and explore its capabilities further. The approach used by Alser et al [26] to communicate between FPGA and PC was based on the use of PCI express which unfortunately is not supported in Laptops and hence needed a Desktop where the FPGA could be fixed. These resources were not available to us.

The approach of storing the data in Block ROM and using the on-chip memory to test the data was hence an outcome of these limitations.

Chapter 7

Cloud Implementation of Sneaky Snake Algorithm

7.1 Service Model

The Sneaky Snake Algorithm was implemented on the cloud using a virtual machine. A virtual machine comes under the category of 'Platform as a Service (PaaS)'. Thus, the service model utilized for the implementation of Sneaky Snake on the Cloud was PaaS. With PaaS, users can develop, test, and deploy apps without having to worry about managing the underlying infrastructure because it offers a comprehensive development and deployment environment. While the cloud provider manages the infrastructure, including servers, storage, networking, and operating systems, users may concentrate on creating and executing their applications. Databases, middleware, and programming frameworks are frequently included as part of PaaS services. Heroku, Google App Engine, and Microsoft Azure App Service are examples of well-known PaaS services.

7.2 Deployment Model

The deployment model can be divided into three basic categories i.e., public cloud, private cloud, and hybrid cloud. The Sneaky Snake algorithm was implemented

on cloud using the Microsoft Azure Services which is a prime example of a public cloud. With a public cloud, the cloud provider owns and manages all of the hardware, software, and other auxiliary infrastructure. Thus, the deployment model used for the implementation of the Sneaky Snake algorithm was the public cloud.

7.3 Virtual Machine

A virtual machine (VM) in cloud computing is an imitation of a computer system or server inside of another actual computer or server. It is a software solution that makes it possible for many operating system instances to operate simultaneously on a single physical machine.

The technology of virtualization, which enables the separation of a physical server into numerous separate virtual environments, allows for the creation of virtual machines. With its own CPU, memory, storage, network ports, and operating system, each virtual machine performs as if it were a distinct real computer. One of the core components of cloud computing is virtual machines. By operating several virtual machines on a single physical server, they offer a way to effectively use hardware resources. This makes it possible for cloud service providers to give their clients scalable and adaptable infrastructure.

Users can deploy different operating systems and programs inside virtual machines that they can create and administer as needed. Virtual machines can be created, scaled up or down, and terminated as needed in the cloud environment, offering flexibility and agility.

Many different workloads, including web servers, databases, application servers, development environments, and more, are frequently hosted on virtual machines. They offer separation between various instances, enabling better resource management and security.

B-series Virtual Machine was used for this particular implementation. VMs in the B-series are often appropriate for workloads with irregular or erratic CPU

utilization patterns. They are frequently used for workloads that encounter bursty demand, such as web servers, development and test environments, tiny databases, unpopular websites, and other workloads. The cost savings that B-series VMs provide for workloads that don't demand constant high CPU performance is an advantage. Users can benefit from decreased expenses while maintaining the ability to handle sporadic increases in demand by utilizing burstable performance. To provide continuous burst performance when required, CPU credits must be monitored and managed.

7.3.1 Specifications

The table below summarizes the specifications of the Virtual Machine used for implementation.

Characteristic	**Specification**
Operating System	Linux (ubuntu 20.04)
vCPUs	20
Size	Standard B20ms
RAM	80 GiB

Table 7.1: Virtual Machine Specifications

7.3.2 Communication and Connection

The Virtual Machine used to host the sneaky snake code is accessed using Secure Shell (SSH). A popular network protocol called SSH, or Secure Shell, allows for secure remote access and system administration over an unsecured network. In order to guarantee the confidentiality and integrity of data transferred between a client and a server, it creates a secure encrypted channel between them. SSH is an essential tool for remote system administration, file transfers, and secure communication because it offers reliable authentication mechanisms, encryption, and the ability for users to securely log in to remote systems, run commands, transfer files, and create secure tunnels for network traffic.

In order to connect with the VM a private key is used to establish a connection with the host on the selected port and IP address. SSH ensures that the data is kept encrypted and hidden hence providing layers of security to the implementation.

7.4 Implementation of Sneaky Snake on Microsoft Azure

The cloud-based implementation of the Sneaky Snake algorithm is an extension of the local implementation that has been modified to meet the cloud deployment requirements. The code that was originally intended for execution on a CPU was modified significantly to address the challenges associated with parallelism and to ensure data integrity.

In the cloud-based implementation, the protection of data from potential problems originating from parallel execution was given special attention. In order to accomplish this, a critical section ('pragma omp critical') was added to ensure that only one thread at a moment reads a line from the input file. This method precludes repeated access to identical data and preserves data consistency.

Additionally, the code was updated to include the reduction clause ('reduction(+:accepted_sum, rejected_sum)'). The cloud-based implementation uses this clause to conduct a sum reduction on the 'accepted_sum' and 'rejected_sum' variables. This eliminates the requirement for atomic operations and guarantees precise and reliable output.

The implementation procedure involved configuring a Virtual Machine (VM) with the required specifications using the Microsoft Azure Dashboard. A pair of SSH keys were generated and used to establish a secure internet connection to the VM. The code was then published to GitHub and downloaded into the virtual machine. For the code to be compiled, the g++ compiler was downloaded and installed on the virtual machine.

Once the environment was configured appropriately, the algorithm could be executed using the executable file that had been compiled. This enabled the

efficient use of cloud resources and the flawless execution of the Sneaky Snake algorithm.

By incorporating these modifications and employing the appropriate cloud infrastructure, the cloud-based implementation of the Sneaky Snake algorithm offers enhanced scalability, enhanced performance, and the ability to leverage the vast pool of cloud-based resources.

The Fig shows the dashboard for the Virtual Machine Created which allows the user to start, stop and observe the Virtual Machine.

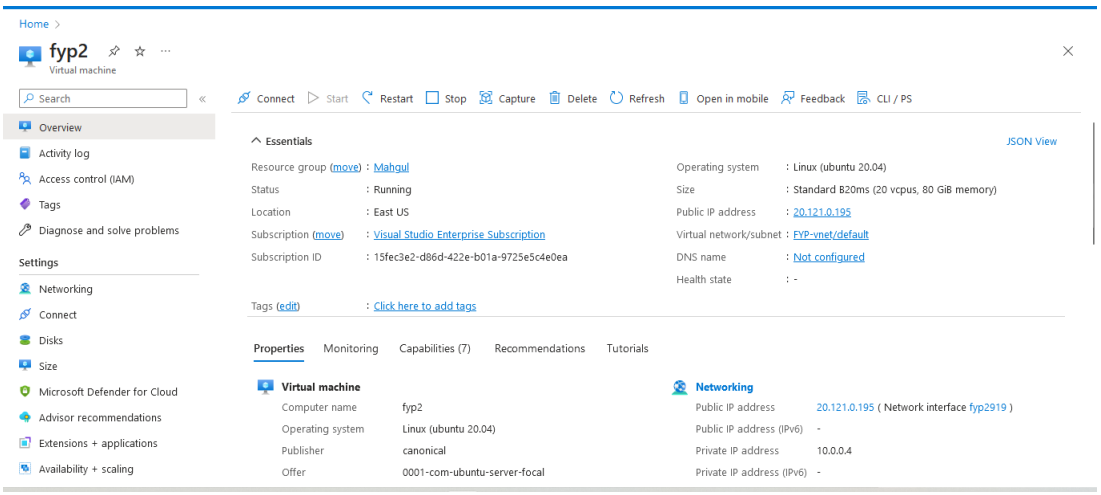

Figure 7.1: Microsoft Azure VM Dashboard

7.5 Cost and Pricing

The Virtual Machine family employed in this implementation costs $0.83 per hour of utilization when utilizing cloud-based services. Even for readings of up to 10,000 base pairs, the Sneaky Snake algorithm executes in a matter of seconds. Due to the minimal execution time compared to the hourly rate of the Virtual Machine, this solution provides an extremely cost-effective processing infrastructure for the implementation. Consequently, organizations can leverage the affordability of the cloud to perform efficient and cost-effective Sneaky Snake algorithm processing, even for large datasets.

Chapter 8

Results

The CPU based implementation of Sneaky Snake Algorithm led to results that went along with the understanding of the algorithm and provided insights to how various factors impacted the performance of the Sneaky Snake Algorithm.

8.1 CPU Implementation

8.1.1 Function and Output

The Figures 8.1 - 8.3 show the output of the Sneaky Snake algorithm with changing Edit Distance Threshold for 100_bp_1 data. The results obtained from the implementation match exactly with those obtained from Alser et al. [26] implementation with no error. It can be observed that as the edit distance threshold is increased more reads are accepted and lesser are rejected.

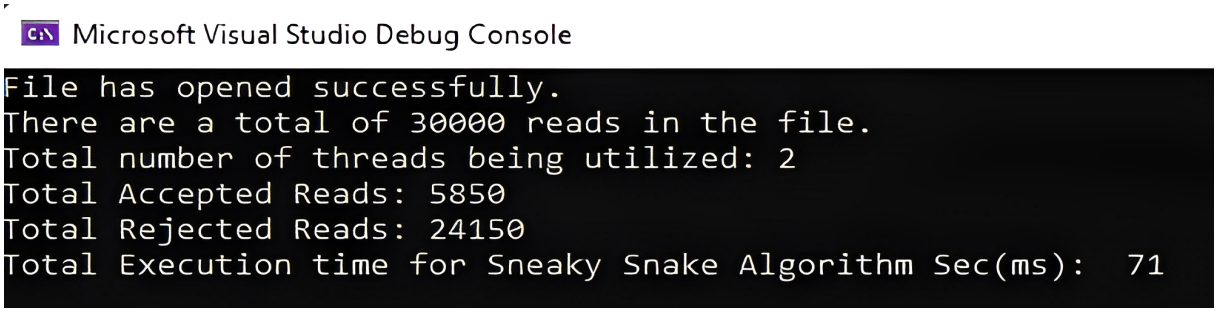

Figure 8.1: Sneaky Snake Output Data Set: 100_bp_1, E = 5, Threads = 2

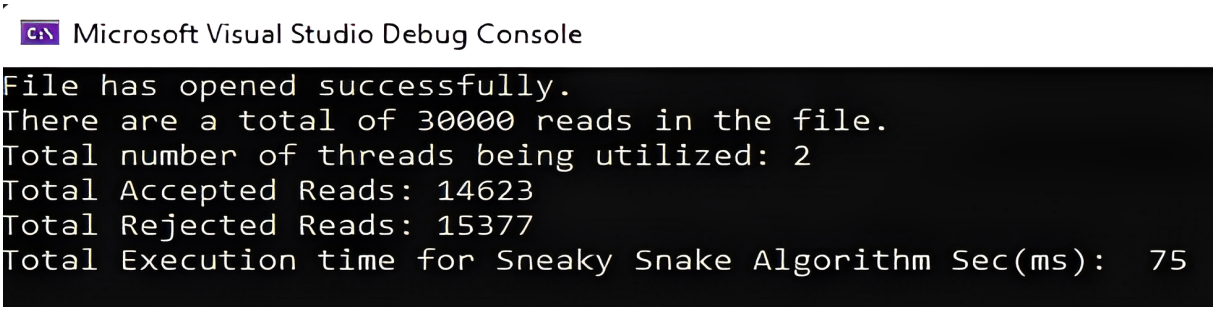

Figure 8.2: Sneaky Snake Output Data Set: 100_bp_1, E = 10, Threads = 2

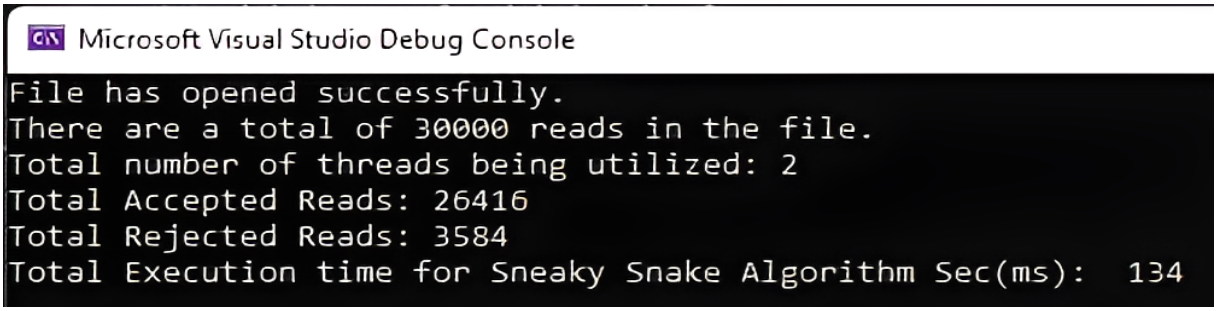

Figure 8.3: Sneaky Snake Output Data Set: 100_bp_1, E = 10, Threads = 2

An important part of heterogeneous systems is the ability of the system to adapt to the need of computation and utilize the resources available differently. The Figures 8.4 - 8.6 below show the algorithms variant CPU utilization for the same data set and number of cores but changing Edit Distance Threshold. In this particular test case the number of threads allowed to be used are 8, which are the maximum number of logical processors available on the hardware.

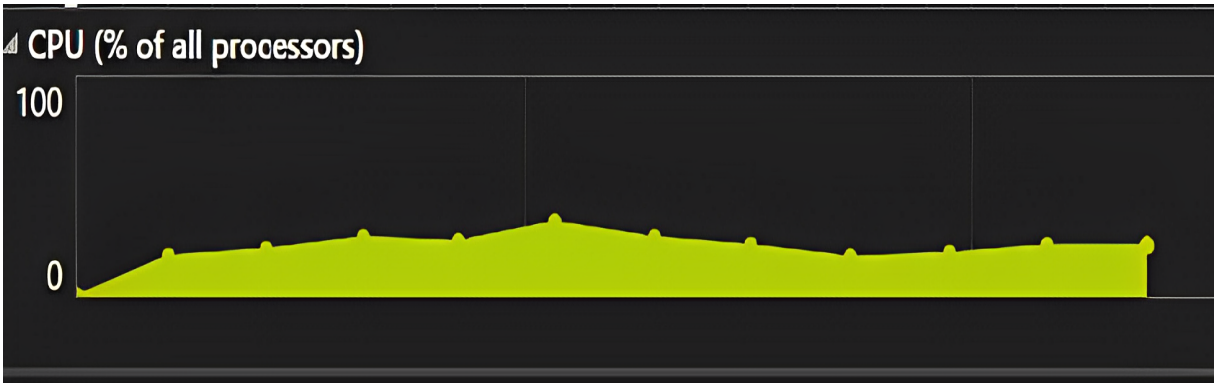

Figure 8.4: Sneaky Snake CPU Utilization Data Set = 100_bp_1, E = 200

Figure 8.5: Sneaky Snake CPU Utilization Data Set = 100_bp_1, E = 2000

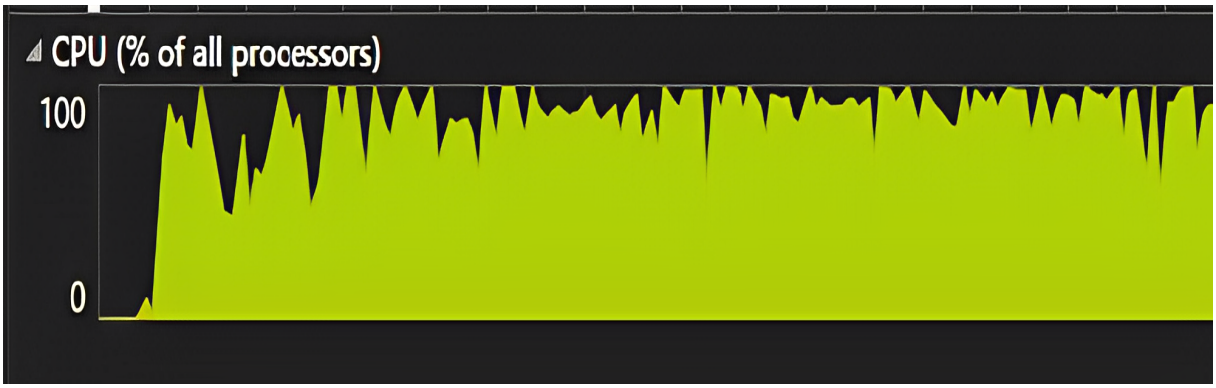

Figure 8.6: Sneaky Snake CPU Utilization Data Set = 100_bp_1, E = 20000

8.1.2 Performance

A performance analysis was conducted to evaluate how the algorithm utilized modern CPU multi-core architectures, comparing it to previous implementations. The analysis aimed to identify the algorithm's strengths and weaknesses in terms of performance and understand the areas where it excelled and where it fell short.

8.1.2.1 Accuracy

The first and foremost requirement of the algorithm is to replicate the results produced by Alser et al. [26]. 100_bp_1 and 250_bp_1 data sets were ran using both implementations and the outputs are shown in Table 8.1 and Table 8.2 below.

Data Set: 100_bp_1					
Edit Distance	**No of Threads**	**Accepted Experimental**	**Accepted Original**	**Rejected Experimental**	**Rejected Original**
0	2	243	243	29757	29757
2	2	1578	1578	28422	28422
5	4	5850	5850	24150	24150
10	6	14623	14623	15377	15377
20	8	26416	26416	3584	3584

Table 8.1: Accuracy Comparison using 100_bp_1 data set with the original implementation [26]

Data Set: 250_bp_1					
Edit Distance	**No of Threads**	**Accepted Experimental**	**Accepted Original**	**Rejected Experimental**	**Rejected Original**
10	2	14	14	29986	29986
20	2	109	109	29891	29891
25	4	4927	4927	25073	25073
30	6	29075	29075	925	925
50	8	30000	30000	0	0

Table 8.2: Accuracy Comparison using 250_bp_1 data set with the original implementation [26]

8.1.2.2 Execution Time

To comprehend the variation in execution time of the algorithm, multiple tests were performed using different data sets. These tests were carried out to investigate how the execution time fluctuates in relation to various parameters. By conducting repeated testing and analyzing the results, a comprehensive understanding of the algorithm's performance characteristics could be gained. The

initial test using data of 100 reads and upto 20 Edit Distance threshold is shown in Table 8.3 and Figure 8.7

Data Set: 100_bp_1					
Edit Distance	0	2	5	10	20
No of Threads	**Execution Time (ms)**				
1	55	59	64	83	128
2	54	59	65	84	120
4	53	63	64	79	117
6	51	54	62	80	120
8	56	60	65	79	121

Table 8.3: Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_1)

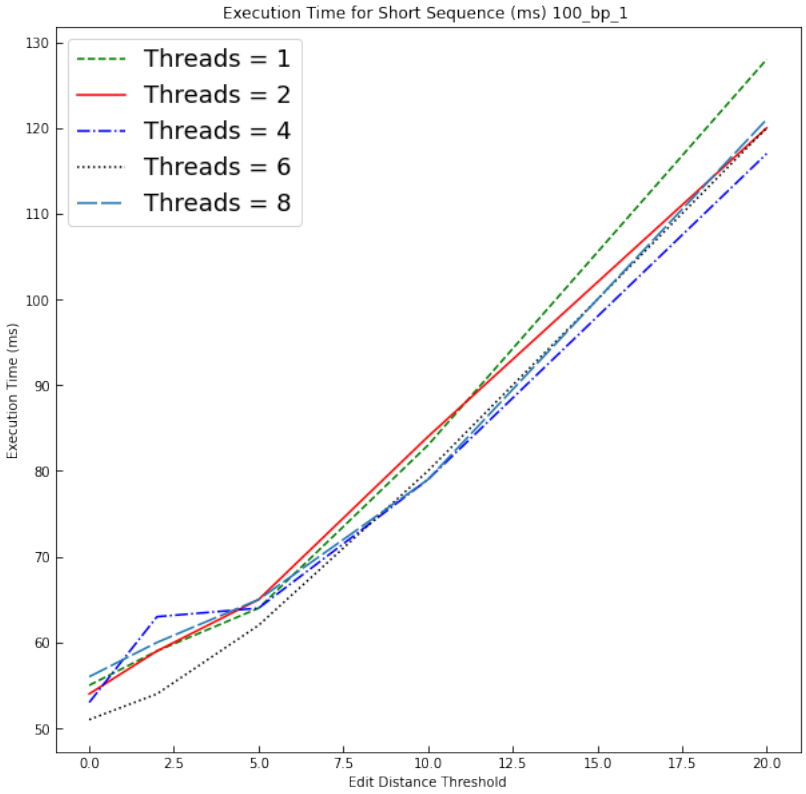

Figure 8.7: Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_1)

The next parameter of interest is the increase in number of reads and the performance of threads in optimizing the algorithm. The results are shown in Table 8.4 and Figure 8.8

Data Set: 100_bp_2					
Edit Distance	0	2	5	10	20
No of Threads	**Execution Time (ms)**				
1	1810	2333	2378	2566	3360
2	1093	1283	1611	1967	2601
4	999	1341	1744	2031	2533
6	1060	1272	1573	1965	2512
8	900	1282	1631	1999	2562

Table 8.4: Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_2)

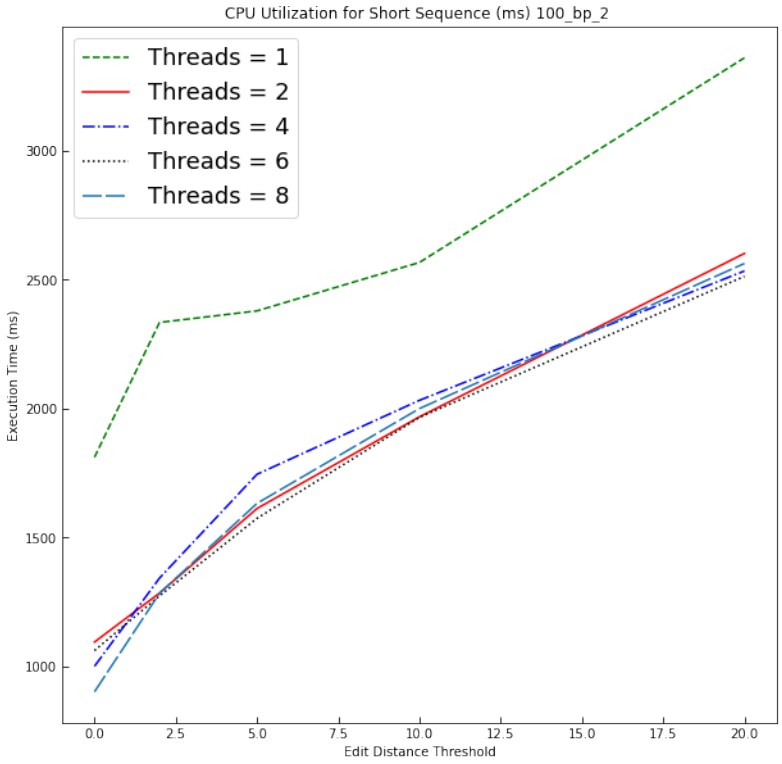

Figure 8.8: Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_2)

The algorithm was tested for longer sequences. The data set long_1 was used. It had 100 reads each of 10000 length. The Table 8.5 and Figure 8.9 shows the obtained result.

Data Set: long_1					
Edit Distance	5000	10000	15000	20000	25000
No of Threads	**Execution Time (ms)**				
1	6122	8847	10194	1115	14215
2	8235	11309	12708	14070	16029
4	5278	7418	8436	9527	10266
6	4501	6334	7027	7928	8622
8	4158	6046	7090	7336	8078

Table 8.5: Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: long_1)

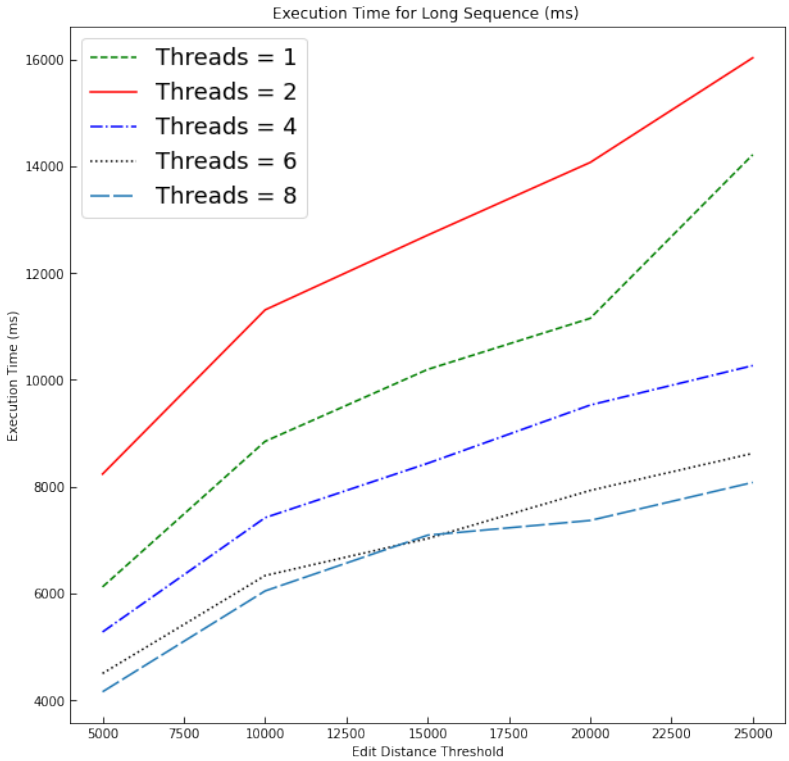

Figure 8.9: Execution Time vs Edit Distance Threshold for varying number of threads (Data Set: long_1)

8.1.2.3 CPU Utilization

To test for the CPU utilization two data sets were used 100_bp_1, 100_bp_2 and long_1. The Table 8.6 and Figure 8.10 shows the CPU utilization with increasing Edit Distance threshold and varying number of threads for 100_bp_1.

Data Set: 100_bp_1					
Edit Distance	0	2	5	10	20
No of Threads	**CPU Utilization (%)**				
1	13	13	13	13	13
2	18	18	18	18	18
4	20	20	21	21	21
6	20	20	20	20	20
8	21	21	21	21	21

Table 8.6: CPU Utilization vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_1)

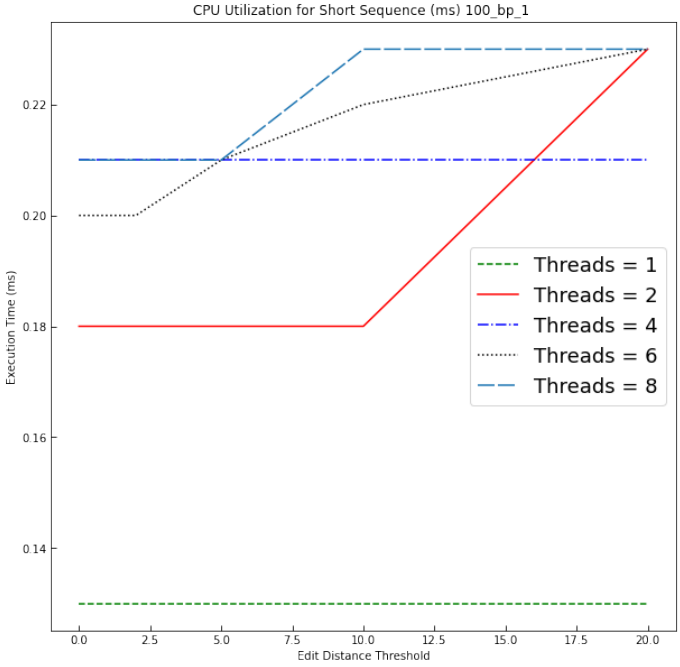

Figure 8.10: CPU Utilization vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_1)

The Table 8.7 and Figure 8.11 shows CPU utilization with increasing Edit Distance threshold and varying number of threads for long_1.

Data Set: long_1					
Edit Distance	0	2	5	10	20
No of Threads	**CPU Utilization (%)**				
1	13	13	13	13	13
2	21	21	21	22	23
4	47	48	48	50	48
6	72	83	77	76	82
8	82	91	94	93	99

Table 8.7: CPU Utilization vs Edit Distance Threshold for varying number of threads (Data Set: long_1)

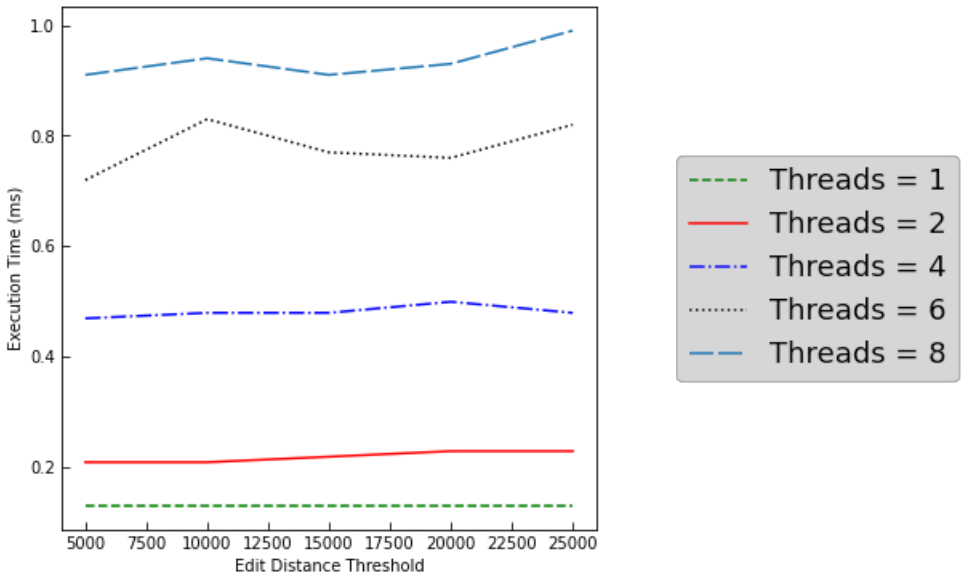

Figure 8.11: CPU Utilization vs Edit Distance Threshold for varying number of threads (Data Set: 100_bp_1)

8.1.2.4 Memory Utilization

The memory utilization remains almost the same for all the data sets mainly because of OpenMP dealing the shared memory access for the implemented algo-

rithm. The memory used is then freed before the next iteration and hence it does not causes a bottleneck for the implementation.

8.1.2.5 Bottleneck and Problems

The algorithm performed better than the single core implementation for most of the data sets except for the data sets where number of reads were significantly more. This is caused by the strncpy_s operation causing repeated access to a global memory file. The Figure 8.12 below shows the CPU Utilization report for the code that ran on 100_bp_2 where the number of reads were 3000000.

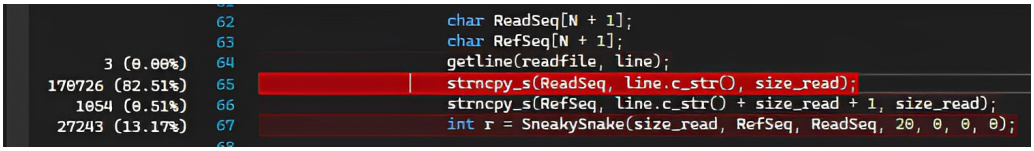

Figure 8.12: CPU Utilization impact shown due to strncpy_s operation

Below is another figure shown for a lower number of loop iterations clearly showing that the above stated factor is no longer the bottleneck. As the number of iterations are reduced, major part of the CPU is used by the Sneaky Snake algorithm. The more number of iterations there are, more copies are needed to be made from the file containing the read data and hence major part of CPU is used for that resulting in a much more execution time.

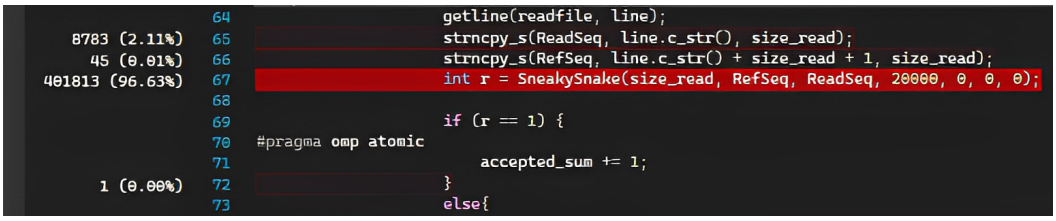

Figure 8.13: CPU Utilization impact shown due to strncpy_s operation

8.2 FPGA Implementation

The FPGA implementation is divided into two parts i.e., test bench simulation and implementation on actual hardware. In this section, the snippets of the testbench

simulation will be attached and discussed. Later on, in Chapter 8 the actual hardware implementation will be addressed.

8.2.1 Outputs

The figures in this section will be representing the results of simulation of the test-bench. The Sneaky Snake algorithm was designed using the bottom up approach so the results will begin from the lower level module i.e., the leading zero counter, and move on to the main Sneaky Snake algorithm at the end.

8.2.1.1 Simulation Results

The first step was to design a 4-bit leading zero counter. A leading zero counter basically counts the highest number of leading zeros in a given input string. The result of the simulation is given in Figure 8.14. The counter design was verified against many inputs and it gave accurate results each time.

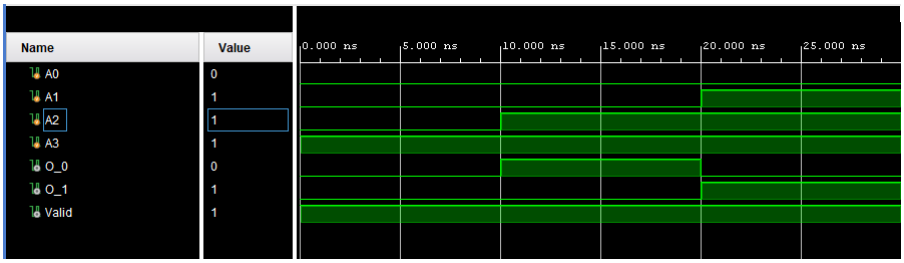

Figure 8.14: Simulation result of a 4-bit leading zero counter

The next step was to use the 4-bit leading zero counter to design an 8-bit leading zero counter. Two instances of a 4-bit leading zero counter were used to design this 8-bit leading zero counter. The result of the simulation is given in Figure 8.15. The counter was verified against many inputs and it gave accurate results each time.

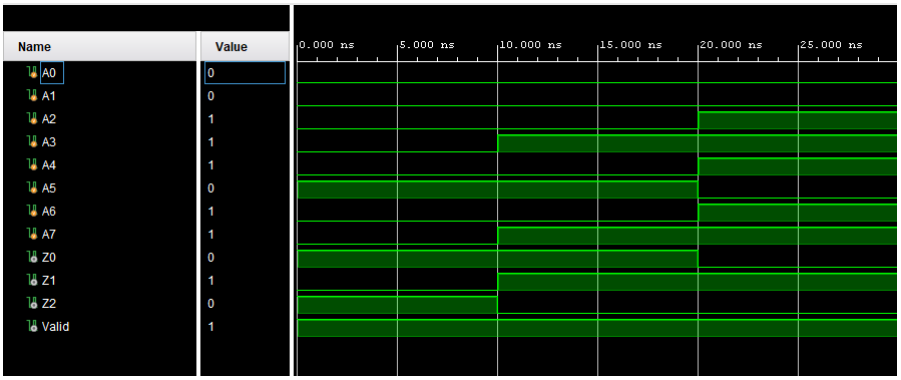

Figure 8.15: Simulation result of an 8-bit leading zero counter

The outputs from the counters are fed to the comparators, which compares the values and decides the longest escape segment. A 3-bit comparator was designed and 6 of these were used in the final design. The output from the comparator is fed to the Longest Escape Segment module. The result of the simulation is given in Figure 8.16. The comparator was verified against many inputs and it gave accurate results.

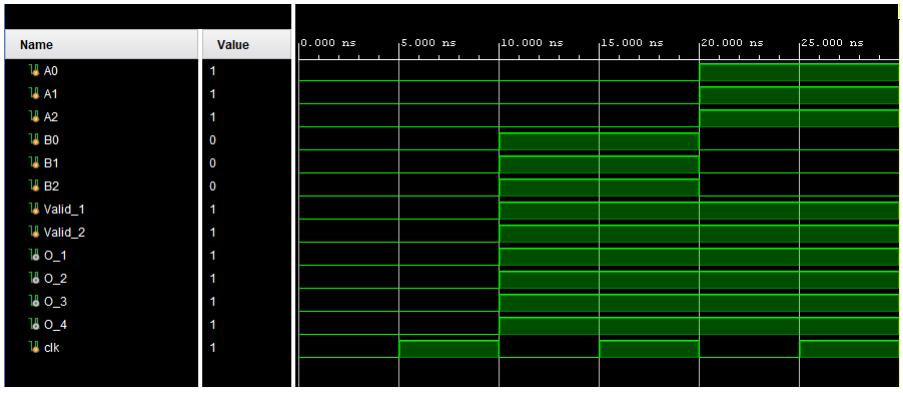

Figure 8.16: Simulation result of a 3-bit comparator

The output from the final two comparators is given as input to the longest escape segment. The longest escape segment then gives its output to the 8-bit Sneaky Snake module. The result of the simulation is given in Figure 8.17 and the values printed on the serial monitor are given in Figure 8.18. The design of the longest escape segment was verified and it gave accurate results.

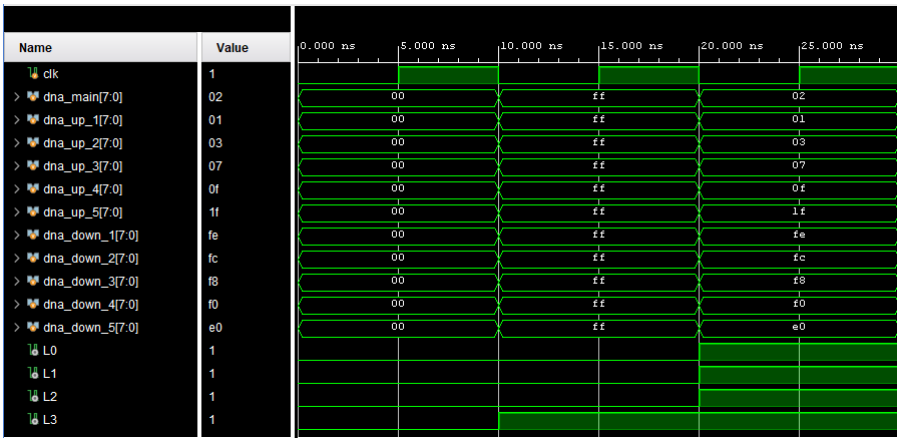

Figure 8.17: Simulation result of the longest escape segment module

```
Time resolution is 1 ps
Test Case 1: L0=0, L1=0, L2=0, L3=0
Test Case 2: L0=0, L1=0, L2=0, L3=1
Test Case 3: L0=1, L1=1, L2=1, L3=1
```

Figure 8.18: Simulation result printed on the serial monitor

Unlike CPU implementation, the chipmaze is designed on the fly here. 100 base pairs are used to design a single chipmaze from which the leading zero counters and comparator decide the longest escape segment. The result of the simulation is given in Figure 8.19. The design of the longest escape segment was verified and it gave accurate results.

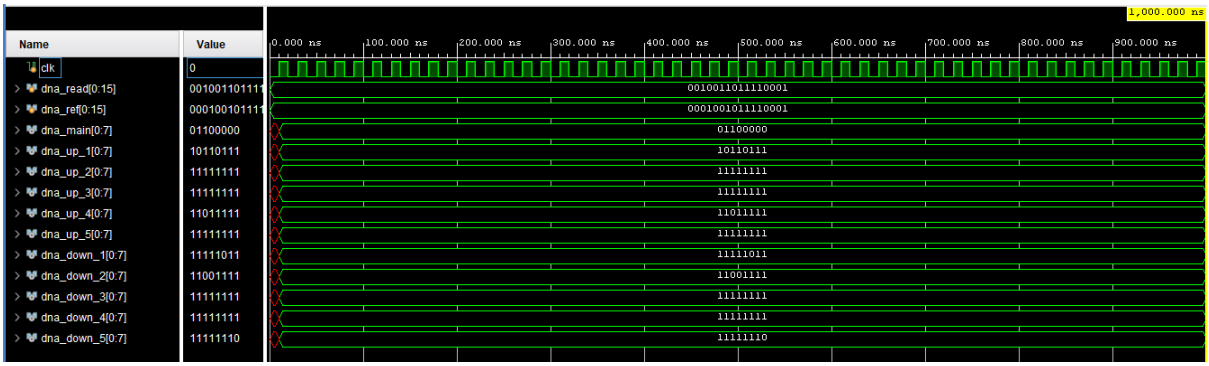

Figure 8.19: Simulation result of the designed Chipmaze

Since, FPGA supports parallel processing and this is the major advantage of using FPGA. An 8-bit Sneaky Snake module was designed whose input was fed from the longest escape segment module. The output from this module was given

to the main Sneaky Snake module. The result of the simulation is given in Figure 8.20 and is printed on the serial monitor as well as shown in Figure 8.21. The design of the 8-bit Sneaky Snake module was verified and it gave accurate results.

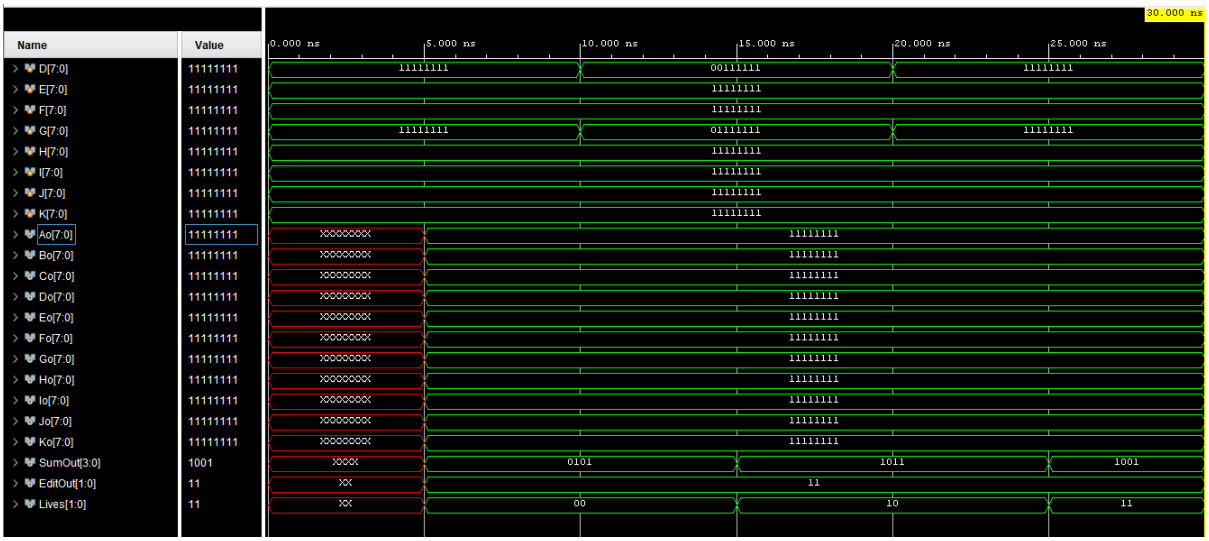

Figure 8.20: Simulation result of an 8-bit Sneaky Snake module

```
Test Case 1: Ao=11111111, Bo=11111111, Co=11111111, Do=11111111, Eo=11111111, Fo=11111111, Go=11111111, Ho=11111111, Io=
11111111, Jo=11111111, Ko=11111111, SumOut=0101, EditOut=11, Lives=00
Test Case 2: Ao=11111111, Bo=11111111, Co=11111111, Do=11111111, Eo=11111111, Fo=11111111, Go=11111111, Ho=11111111, Io=
11111111, Jo=11111111, Ko=11111111, SumOut=1011, EditOut=11, Lives=10
Test Case 3: Ao=11111111, Bo=11111111, Co=11111111, Do=11111111, Eo=11111111, Fo=11111111, Go=11111111, Ho=11111111, Io=
11111111, Jo=11111111, Ko=11111111, SumOut=1001, EditOut=11, Lives=11
```

Figure 8.21: Simulation result printed on a serial monitor

The final Sneaky Snake module is built using multiple instances of the 8-bit Sneaky Snake module since in FPGA implementation, the data is broken down into multiple pieces and all of them undergo parallel processing. The output of the final Sneaky Snake module tells whether the read has been accepted or not. The result of the simulation is given in Figure 8.22 and Figure 8.23. The design of the Sneaky Snake was verified against many inputs and the results obtained were similar to the ones obtained by Alser et. al. [26].

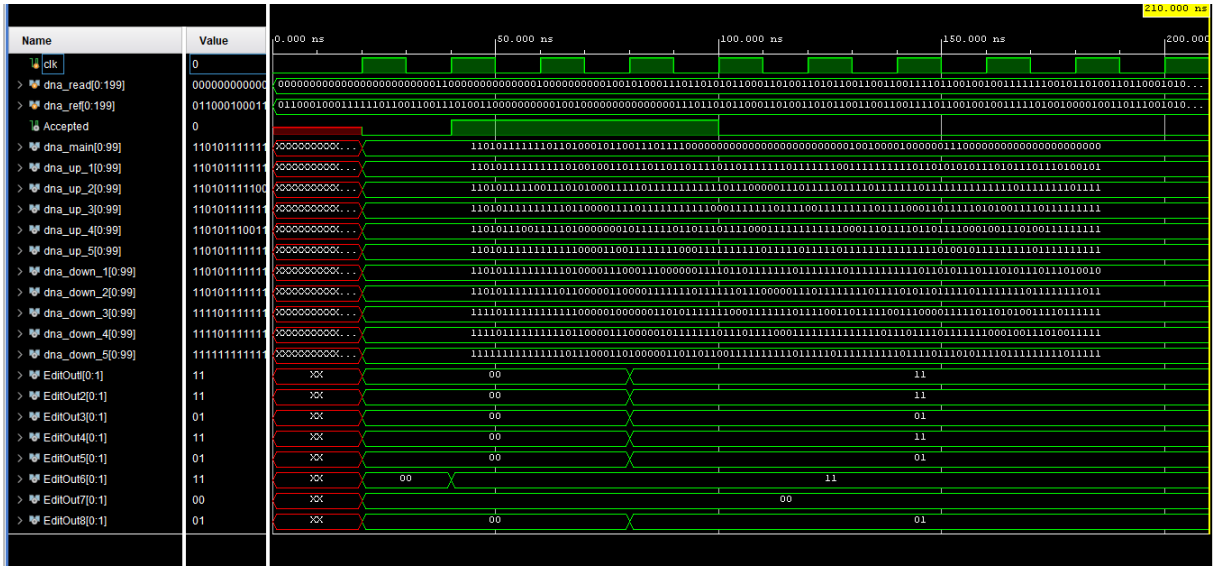

Figure 8.22: Simulation result of the Sneaky Snake module

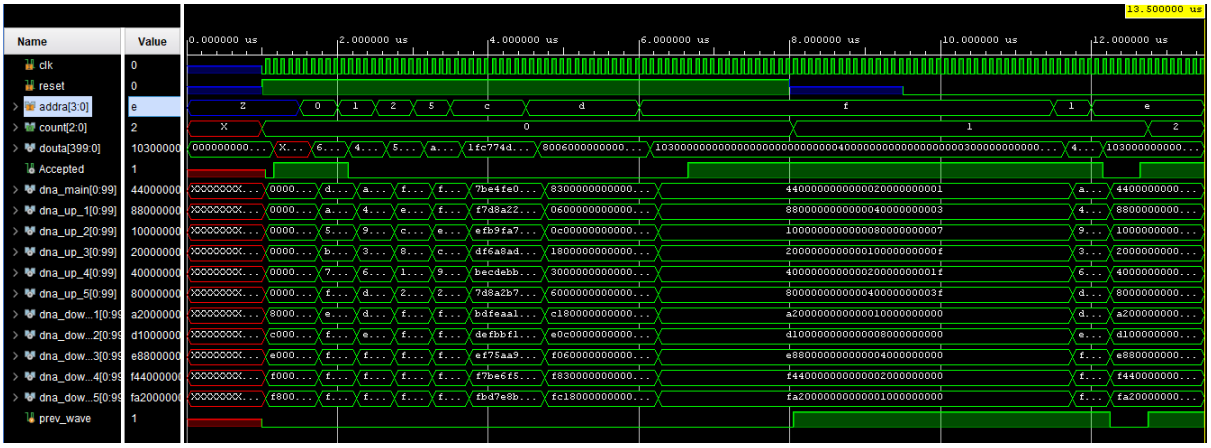

Figure 8.23: Simulation result of the final Sneaky Snake module

A summary of the resource utilization is provided in the figures given as follows.

Name	Waveform	Period (ns)	Frequency (MHz)
clk_in1_p	{0.000 2.500}	5.000	200.000
clk_out1_clk_wiz_0	{0.000 50.000}	100.000	10.000
clkfbout_clk_wiz_0	{0.000 10.000}	20.000	50.000

Figure 8.24: Clock Summary

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	92.252 ns	Worst Hold Slack (WHS):	0.089 ns	Worst Pulse Width Slack (WPWS):	1.100 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	2121	Total Number of Endpoints:	2121	Total Number of Endpoints:	1947

All user specified timing constraints are met.

Figure 8.25: Design Timing

Resource	Utilization	Available	Utilization %
LUT	3686	303600	1.21
FF	1931	607200	0.32
BRAM	5	1030	0.49
IO	10	700	1.43
BUFG	2	32	6.25
MMCM	1	14	7.14

Figure 8.26: Resources utilized

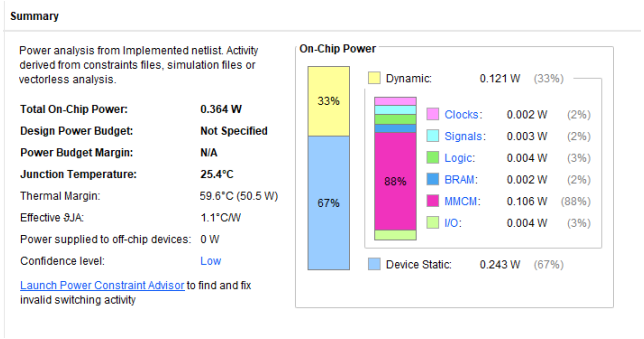

Figure 8.27: Power Summary

8.2.1.2 Results of Hardware Implementation

Sneaky Snake algorithm was implemented on Xilinx Virtex VC707. This section presents the results of the hardware implementation of Sneaky Snake.

Figure 8.28 shows the code being uploaded on FPGA and the board in its state of being programmed.

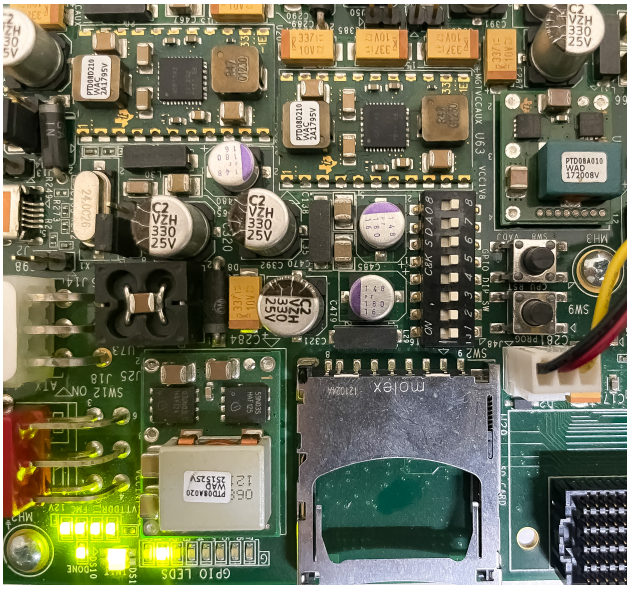

Figure 8.28: FPGA board being programmed

The LEDs on the board give the output of the counter. With every accepted read the counter increments the output by one and this is displayed on the LEDs. Figure 8.29 shows that a single read is accepted and the LEDs display '1'.

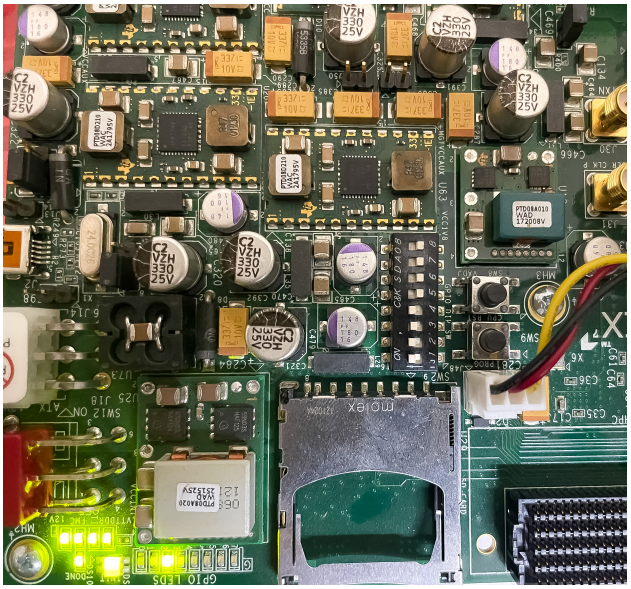

Figure 8.29: Only one read accepted (COUNT=1)

Figure 8.30 shows that another read is accepted not and the counter increments its value by 1. Thus, with two reads accepted the final value becomes two and that is displayed on the LEDs.

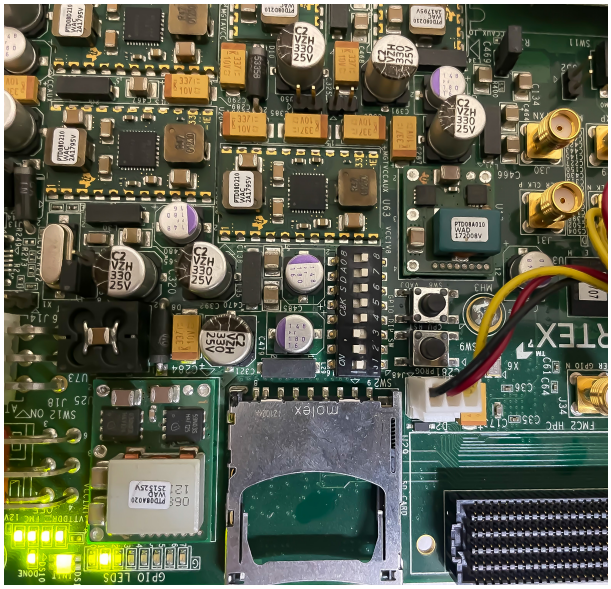

Figure 8.30: Two reads accepted (COUNT=2)

When a read is not accepted, the value of counter becomes zero. Thus, Figure 8.31 shows that the given read was not accepted and zero is displayed on the LEDs.

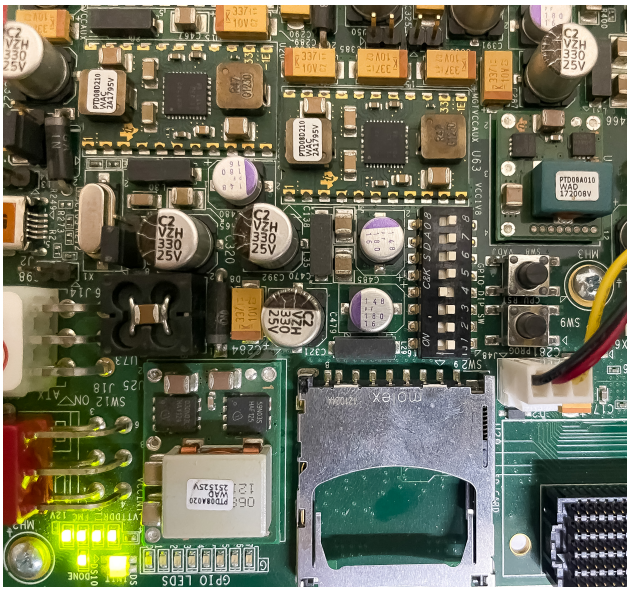

Figure 8.31: Read rejected (COUNT=0)

Figure 8.32 shows that another read is not accepted and the counter still gives a value of zero. Thus, a zero is displayed on the LEDs again.

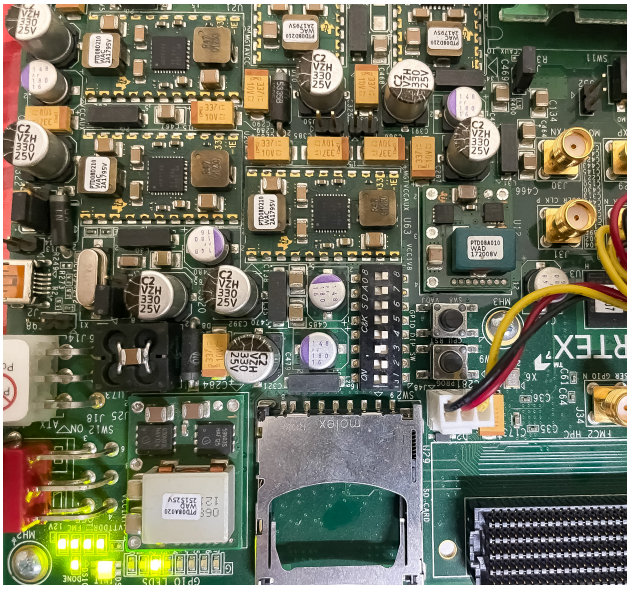

Figure 8.32: Read rejected again (COUNT=0)

When the RESET button is pressed, all the previous information is lost and the counter resets its value and now the process will start all over again. Figure 8.33 shows the board after the reset button is pressed.

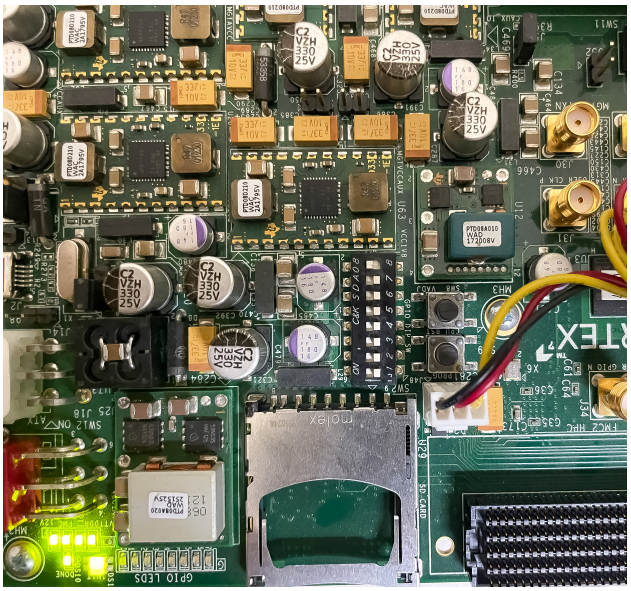

Figure 8.33: Reset button pressed

8.3 Cloud Implementation

Cloud Based implementation was simply re configuring the code written for CPU for a new processors. The major changes made were already discussed in Chapter. In this section the results obtained be testing the implementation on cloud are discussed.

8.3.1 Function and Output

SSH connection using Microsoft PowerShell was used to connect and log into Microsoft Azure VM on which the code is to run. Fig 8.34 shows the successful connection being established with the Virtual Machine over internet. The VM runs

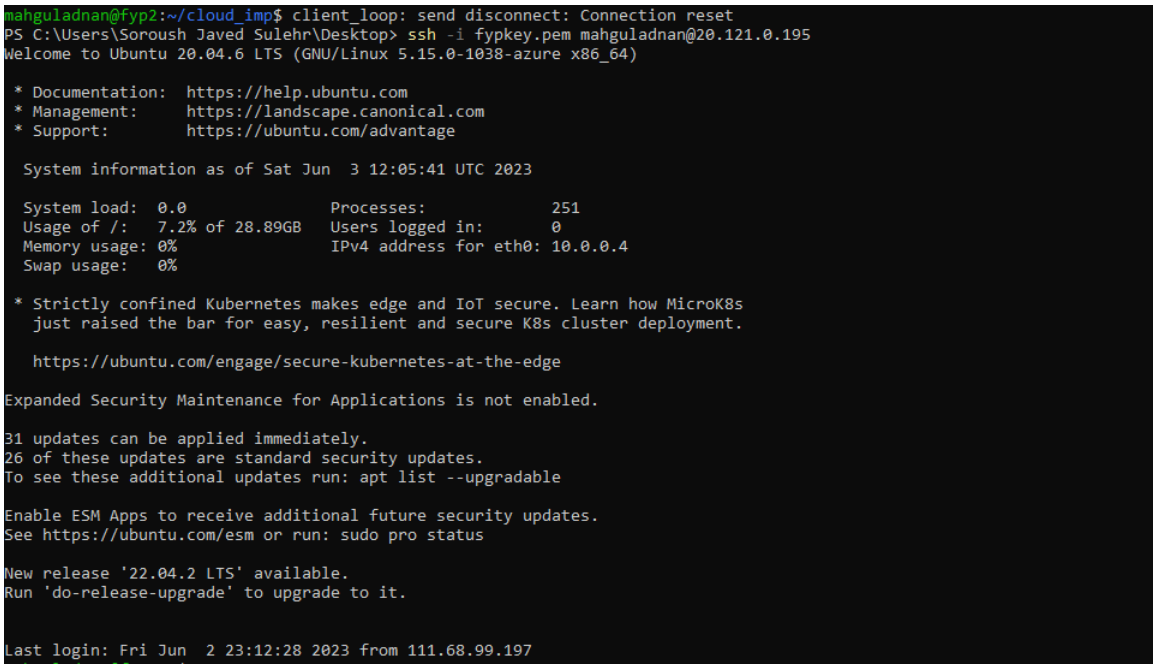

Figure 8.34: SHH based connection to Virtual Machine

Ubuntu 20.04 LTS version. Command Line Interface is used to compile and run the code. Fig 8.35 and Fig 8.36 show the outputs for two different complications of the code with changing parameters.

One of the most notable benefits of deploying a system in the cloud is the ability to access a vast and diverse reservoir of resources that exceeds the limitations of a local implementation. This vast resource pool enables organizations to effectively

```
File has opened successfully.
There are a total of 30000 reads in the file.
Total number of threads being utilized: 20
Total Accepted Reads: 14623
Total Rejected Reads: 15377
Total Execution time for Sneaky Snake Algorithm (ms): 26
```

Figure 8.35: Output Edit Distance = 5 Dataset = 100_bp_1

```
mahguladnan@fyp2:~/cloud_imp$ ./program2
File has opened successfully.
There are a total of 100 reads in the file.
Total number of threads being utilized: 20
Total Accepted Reads: 100
Total Rejected Reads: 0
Total Execution time for Sneaky Snake Algorithm (ms): 447
```

Figure 8.36: Output Edit Distance = 5000 Dataset = long_1

meet their specific needs by leveraging a broad range of computing power, storage capacity, and network bandwidth. The Fig 8.37 shows multiple threads (upto 20) running in a parallel fashion dealing with different iteration of for loop in a concurrent fashion.

8.3.2 Performance

A common method for evaluating the efficacy of a cloud-based implementation is to compare it to that of a local implementation. This evaluation framework enables organizations to assess the efficacy and efficiency of cloud resource utilization. By analyzing execution time and accuracy in comparison to a local implementation, the impact of leveraging cloud-based resources can be determined. This comparative analysis offers valuable insights into the benefits and optimizations offered by cloud implementations, highlighting the benefits obtained by leveraging the vast resource pool and scalability capabilities inherent to cloud platforms.

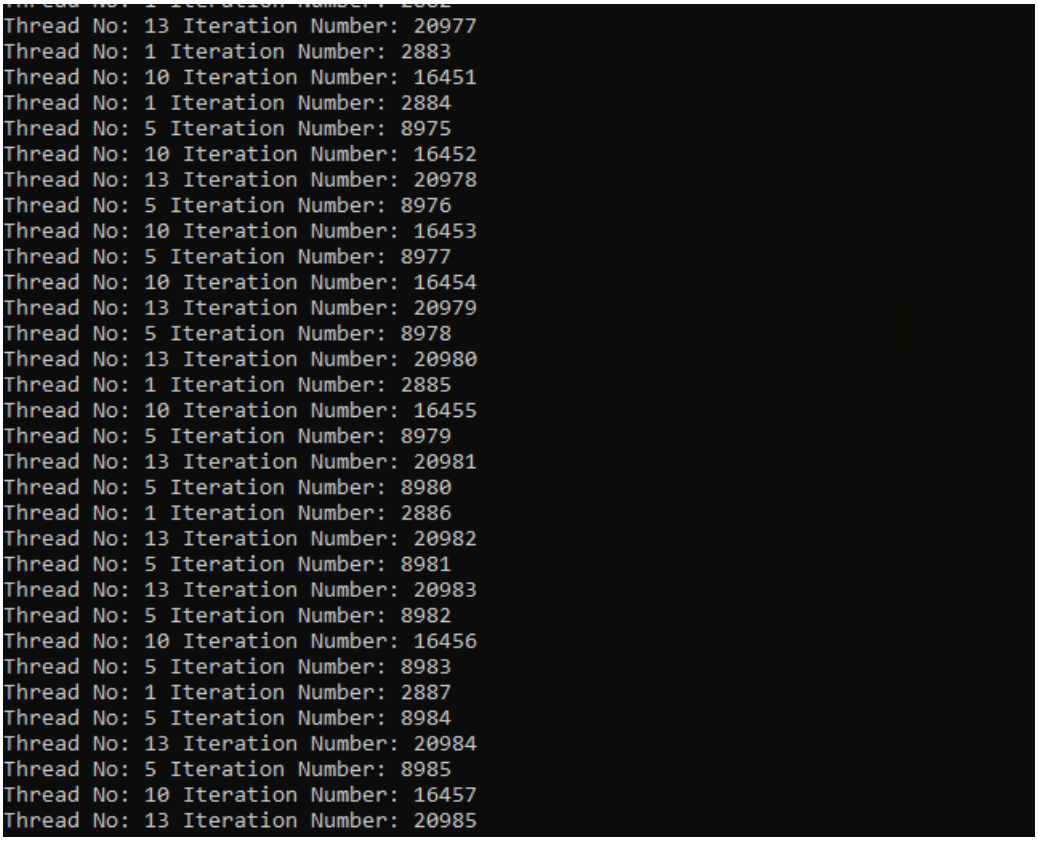

Figure 8.37: Parallel Execution of Sneaky Snake on Cloud

8.3.2.1 Accuracy

The accuracy of Sneaky Snake Implementation on the cloud was tested by running the code for 100_bp_1 data set for three different Edit Distances and comparing the output as achieved by previous implementation. The results are shown in Table 8.8. In most of the cases the outputs exactly matched those of the local implementation. However, there were some cases where the counts were of by 1 or 2 reads. Some of the execution cases are attached below in Fig 8.38 - Fig 8.40

Edit Distance	Accepted Cloud	Accepted Local	Rejected Cloud	Rejected Local
5	5850	5850	24150	24150
10	14623	14623	15377	15377
20	26415	26415	3585	3585

Table 8.8: Comparing Output for Varying Edit Distance Thresholds (100_bp_1 Dataset)

```
mahguladnan@fyp2:~/cloud_imp$ ./program2
File has opened successfully.
There are a total of 30000 reads in the file.
Total number of threads being utilized: 8
Total Accepted Reads: 26416
Total Rejected Reads: 3584
Total Execution time for Sneaky Snake Algorithm (ms): 24
```

Figure 8.38: 100_bp_1 E = 20 Output

```
File has opened successfully.
There are a total of 30000 reads in the file.
Total number of threads being utilized: 20
Total Accepted Reads: 14623
Total Rejected Reads: 15377
Total Execution time for Sneaky Snake Algorithm (ms): 26
```

Figure 8.39: 100_bp_1 E = 10 Output

```
mahguladnan@fyp2:~/cloud_imp$ ./program2
File has opened successfully.
There are a total of 30000 reads in the file.
Total number of threads being utilized: 8
Total Accepted Reads: 5850
Total Rejected Reads: 24150
Total Execution time for Sneaky Snake Algorithm (ms): 18
```

Figure 8.40: 100_bp_1 E = 5 Output

8.3.2.2 Execution Time

The second performance parameter that was to be tested was 'Execution Time'. As cloud based implementation has access to a greater number of threads and had nothing else on the CPU expect for a light version of Ubtuntu. A better performance was expected. The table below summarizes the execution times for varying parameters for 100_bp_1 and long_1 data sets. To obtain accurate execution time values, the code was run multiple times in both the cloud-based and local implementations. The goal was to identify the most consistent and reliable measurement among the repetitions. The execution time measurement methodology used in the cloud-based implementation was similar to that employed in the local implementation to ensure fair and comparable results.

By following this approach, the obtained execution time values reflect the actual performance of each implementation method. The consistency in measuring execution time allows for a reliable comparison between the two approaches, indicating that the cloud-based implementation indeed exhibited a significant reduction in execution time when compared to the local implementation.

Data Set	**long_1**		
Edit Distance	**5000**	**10000**	**25000**
Number of Threads	**Execution Time (ms)**		
8	1206	5057	4773
10	963	3600	4338
20	909	3877	4812
Data Set	**100_bp_1**		
Edit Distance	**5**	**10**	**20**
Number of Threads	**Execution Time (ms)**		
8	6	8	18
10	5	7	17
20	6	6	9

Table 8.9: Execution Time Cloud Implementation

8.3.3 Comparison with CPU Implementation

Comparing cloud-based and local implementations revealed a significant reduction in execution time, especially for lengthier reads. This enhancement is attributable to cloud computing's inherent benefits, such as access to bountiful resources, scalability, and elasticity. Utilizing the infrastructure of the cloud enabled the implementation to efficiently process tasks and manage workloads, resulting in improved overall performance and decreased execution times.

In addition, the analysis of lengthy reads revealed a consistent pattern: as the number of threads increased, the execution time decreased even further. In contrast, for shorter reads, an increase in the number of threads did not have a significant impact on the execution time, but the cloud-based implementation was substantially faster than the local implementation.

These results lend credence to the notion that the cloud-based approach offers significant advantages for processing larger readings, capitalizing on increased

thread utilization to optimize performance. In terms of reduced execution time, the cloud-based implementation demonstrated advantages over the local approach even for brief reads. These findings demonstrate the efficacy of leveraging cloud resources to improve execution time and performance for varying read lengths within the context of the implemented system.

Chapter 9

Discussion and Conclusion

9.1 Discussion

9.1.1 CPU Implementation

Genome Sequencing today is being normalized daily. More and more people and doctors use genome analysis for better diagnosis and medication. Hence, at the dawn of the modern era, where we will soon touch the end of Moore's law to accelerate computationally intensive processes such as genome sequencing, honestly, better approaches must be found. Heterogeneous computing systems are the answer to the limitation that will be placed on humanity due to the lack of smaller sizes for transistors in the future. The results clearly show that even for an ingenious algorithm such as Sneaky Snake, there are limitations imposed by the use of CPU on the functionality.

The first and foremost result expected from the implementation was the reproduction of the same results as that of Alser et al. [26] in terms of accepted and rejected reads. Table 6.1 and Table 6.2 show the comparison between accepted reads and rejected reads between this and the original implementation. The data was obtained by running both algorithms and varying parameters, such as the Edit Distance Threshold and Threads Number. As observed, no matter what parameters were provided, the results produced by this and the original implementation

did not differ even by a single read. These results validate the implementation's accuracy and precision in giving the correct output for different read data.

However, the core concept considered relevant to this project is the performance of this algorithm on the CPU. As described in the methodology, multiple tests were run using different data sets, and the obtained results gave some insights into the performance of the implementation. Table 6.3 shows the data collected by running the code with varying edits and threads on 100_bp_1 dataset. The difference between execution increases with increasing edit distance threshold. However, there is little to no significant change seen in the execution times as the number of threads used increases, which can also be observed in Figure 6.7. The results explain that even though the number of threads allowed to be used by the code is up to 8, the maximum number offered by the hardware used, the code still uses only enough percent of CPU that seems to be justified for the data set being tested. This hypothesis is also supported by the data obtained for CPU utilization in Table 6.6 and Figure 6.10. CPU utilization is an essential resource that must be managed when executing any algorithm, as it can act as a significant bottleneck if not implemented correctly.

The implementation of the Sneaky Snake needs to manage the resources used by saving them where possible, but it is equally essential to utilize them when needed. Table 6.4 and Figure 6.8 show the execution time of the algorithm for a data set of 3000000 reads. This is where the difference between a multi-threaded implementation to a single-core implementation becomes visible. The multi-threaded implementation, on average, takes almost 23% lower time to work on the same data set under the same parameters. Justifying that with the increased workload, the code adapts accordingly and utilizes the resources available smartly.

The multi-threading of the algorithm was done by parallelizing the single for loop in the main code responsible for calling the sneaky snake algorithm on the read data. Hence, the optimal working condition for the code is when there are

many reads to be sent to the algorithm. This is actually shown quite clearly in Table 6.5 and Figure 6.9 where the read length is increased to up to 10000, but the total number of reads is only 100. It is seen that when using a single thread, we get a better performance than using two threads. This can be explained by the parallelization overhead cost, which seems to become significant when only two threads are used—the time needed for creating threads results in a loss in performance. As the number of threads is increased beyond two, the algorithm's performance improves concerning single-core implementation. This is because the speed of execution compensates for the thread creation time. Hence an important observation here is that adjusting the number of threads following the computational load is integral for good performance in the case of multi-threaded implementation.

Finally, in the end, performance reports from Visual Studio were analyzed to see the code's computational load division. As the number of iterations increased, the most CPU utilization was done by the read operation that copies data from the file and loads it into an array. This is a significant bottleneck for reading data where the number of reads is large.

The limitation of this study is placed by the hardware being used. Better CPU units were not available for testing; hence the speed of the algorithm was not tested using more powerful hardware. However, the functionality test was reproduced on online compilers, and the results were accurate and precise. Furthermore, data with a more significant number of reads was unavailable; hence, the same data was duplicated to increase the number of reads for testing.

The analysis of results shows a clear relationship between the algorithm's performance in terms of execution time with an increasing number of cores. In most cases, performance acceleration is observed as the number of cores allowed to be used is increased.

9.1.2 FPGA Implementation

In certain applications, FPGAs (Field-Programmable Gate Arrays) offer distinct advantages over CPUs (Central Processing Units). In contrast to CPUs, which are designed for general-purpose computing, FPGAs can be customized and re-programmed to efficiently execute specific tasks. This flexibility enables parallel processing and the ability to implement hardware-level optimizations, resulting in significantly quicker execution speeds and reduced power consumption compared to conventional CPUs. FPGAs are ideally suited for applications such as high-performance computation, digital signal processing, artificial intelligence, and accelerated data analytics due to their proficiency with massive data parallelism and real-time processing. FPGAs are a valuable tool for increasing performance and efficiency in specialized computing domains due to their adaptability to changing requirements and their potential for hardware acceleration.

FPGAs offer numerous advantages over CPUs, but algorithm design for FPGAs necessitates careful consideration of certain factors. The need to accommodate specific limitations, such as an Edit Distance Threshold and a maximum number of base pairs in the input, is an essential factor. This necessitates an approach to design that is modular and readily adaptable to various parameter sets. In the case of implementing Sneaky Snake, a bottom-up strategy was employed, with separate modules created for each implementation component. This design strategy guarantees both modularity and scalability, allowing the algorithm to be efficiently extended and optimized. By exploiting the inherent modularity and scalability of the FPGA design, the implementation becomes highly adaptable and capable of meeting a wide range of computational needs.

Thorough testing was done by creating specialized test benches for each module in order to guarantee the precision and operation of each manufactured module. These test benches served as thorough test cases that made it possible to compare the module's outputs to anticipated outcomes. The outcomes of these thorough tests are shown in Figures 8.14 to 8.23, which show that each individual module has

been successfully validated. Any potential problems or faults might be found and fixed at the root cause by thoroughly evaluating each module. This methodical approach considerably sped up the design process overall by enabling the early detection and resolution of any issues, resulting in a more stable and dependable FPGA implementation.

An optimal method for facilitating proper testing of the FPGA implementation would involve computer interfacing. However, restrictions on the available communication protocols limited the alternatives to PCI and Ethernet. Using Xilinx Virtex VC 707 for Ethernet-based communication necessitated the use of IP processors that were unavailable in Pakistan due to licensing restrictions. Similarly, the incompatibility of the PIC Express interface with laptops rendered it inappropriate for testing. Consequently, an alternative strategy was developed. Along with the remainder of the implementation, the data was directly programmed into a Block ROM module on the FPGA and synthesized. To select various data inputs and observe outputs, DPI switches were utilized, and LEDs were used to visually display the results. The effectiveness of this testing method is illustrated in Figures 8.28 through 8.33, which provide a practical illustration of the implemented strategy.

There were two performance considerations that were decided to be measured for the FPGA implementation: accuracy and resource utilization. In order to test the accuracy a dataset derived from 100_bp_1 was constructed. There was total of 16 DNA fragments in the data set. 13 of them had an Edit Distance threshold greater than 5 while three of them had an Edit Distance Threshold lesser than 5. Figure 8.23 shows how when a read is accepted the total accept count increments. As the data set for testing was customized the addresses at which the DNA fragments rested were known. Hence, Figure 8.23 also validates the accuracy of the implementation as those data inputs which had edit distance lower than 5 were accepted while the rest were rejected. Moving this implementation to

hardware the accuracy was again verified by providing specific addresses to the implementation and nothing about the count value being expressed by the LEDs.

Resource use was a crucial performance factor when the design was put into practice. Figure 8.26 gives a summary of the resources that were used. Surprisingly, the method only required 3686 Look Up Tables (LUT) and 1931 Flip-flops for the input data of 100 base pairs and a 5 edit distance. This exemplifies the extremely effective use of resources. In fact, it is anticipated that the implementation would be simple even when scaling up to larger DNA segments having 10,000 base pairs in the FPGA's overall resources. This is because just a small portion of the FPGA's entire capacity is being used by the used resources. Such smart resource management emphasizes the viability and efficacy of the established system, providing enough space for scaling and supporting larger datasets without degrading performance. Furthermore, as verified by Fig 8.25 all user-specific timing constraints for the implementation were met and the implementation had minimal power consumption as shown in Fig 8.27.

The bottleneck produced by the string copying operations used in C++ was the main reason for the CPU implementation's restrictions. In particular, the Chip Maze's on-the-fly construction necessitated numerous calls to file read operations and string manipulations, which decreased speed. However, the FPGA implementation successfully overcame these constraints by making use of the special features of FPGAs.

The use of bitwise parallelism in FPGA implementation provided a big benefit. Bypassing the need for tedious file read operations and string manipulation, this made it possible to instantly construct the entire Chip Maze. The FPGA implementation outperformed the CPU implementation significantly thanks to the use of parallel processing capabilities of FPGAs.

The ability to use Look Up Tables (LUTs) in FPGAs also significantly sped up the calculation of the Edit Distance. The FPGA implementation was able to carry out calculations immediately utilizing LUTs as opposed to the CPU counterpart,

which often accessed memory and copied strings. As a result, the time needed to calculate Edit Distance was greatly decreased because there was no longer a requirement for frequent memory access. As a result, the FPGA implementation performed faster and more effectively than the CPU implementation.

The inadequacy of the interface between the FPGA and a physical device, such as a PC, is the main cause of the constraints of the FPGA-based implementation in this project. However, by ensuring the availability of licenses for IP cores that can provide either an Ethernet-based or PCI-based connection, these restrictions can be efficiently overcome.

It is feasible to create a dependable and quick communication path between the FPGA and the physical device by getting the required IP core licenses. For example, a PC and FPGA could communicate and transfer data with ease using an Ethernet-based connection, making it possible to test, debug, and interact with the implemented design quickly and effectively.

Similar to this, a PCI-based connection can give the FPGA and the physical device a direct and effective interface, facilitating quick data transmission and real-time communication. This would improve the usefulness and adaptability of the FPGA implementation by enabling more thorough testing, monitoring, and control.

Finally, in specialized computing sectors, FPGA-based systems offer significant advantages over CPUs. Faster execution rates, lower power consumption, and effective resource use are made possible by the flexibility, parallel processing abilities, and hardware-level optimizations of FPGAs. While meticulous testing and validation of each module ensure accuracy and operation, the modular and scalable design approach ensures adaptation to changing parameters. Alternative tactics and direct programming on the FPGA provide for efficient testing and verification, despite the fact that communication protocol restrictions may present difficulties for computer interfacing. FPGAs seem to be a useful tool for boosting

performance, efficiency, and adaptability in specialized computing applications by getting beyond the constraints of CPU implementations.

9.2 Cloud Based Implementation

The code for the Microsoft Azure Sneaky Snake implementation was run on a Virtual Machine (VM) with 20 virtual CPUs. The usage of VMs made it simple to obtain computational resources on a pay-as-you-go basis by utilizing the Platform as a Service (PaaS) model, which eliminated the need for expensive infrastructure. Utilizing output counts and built-in timing metrics, the cloud-based implementation's performance was evaluated.

When compared to the outcomes of the local implementation, Table 8.8 shows that there were no errors in the number of accepted and rejected reads. Minor inaccuracies in the values could be anticipated, nevertheless, as a result of the use of so many threads. It is crucial to remember that even in the event that such errors do occur, they normally only account for one or two reads, which is negligible when compared to the overall higher number of reads processed.

The execution time for lengthier readings was shown to be the main benefit of cloud computing. The completion time for larger readings was twice as long as that on the cloud, even when the local implementation used the maximum number of CPU cores. By contrasting the outcomes from Tables 8.5 and 8.9, it is possible to see this substantial discrepancy. However, because of the nature of the workload, adding more cores did not significantly improve performance for shorter readings. In comparison to lengthier reads, shorter reads use less computational effort.

The cloud-based approach outperformed the local implementation in terms of performance and cost-effectiveness. Local installation of the system would necessitate a capital investment, but cloud computing charges are based on actual usage, offering a more adaptable and affordable solution.

Overall, the PaaS architecture and the Sneaky Snake implementation on Microsoft Azure's cloud platform illustrated the advantages of using VMs for effective resource use, faster execution times, and lower costs. The cloud infrastructure is an appealing option for conducting computationally intensive operations in a cost-effective way due to the correctness of the results and the scalability it offers.

9.3 Conclusion

The study investigated how the Sneaky Snake algorithm might work on several platforms, including CPU, FPGA, and cloud-based infrastructure. The objective was to assess how well the algorithm worked in each context. The results of the CPU implementation were precise and accurate, matching those of the initial implementation. Multi-threading enhanced speed, especially when working with larger datasets, according to the examination of execution times and CPU usage. However, inefficiencies brought on by string copying operations made the shortcomings of CPU-based methods clear. The FPGA implementation, however, showed a number of advantages over CPU-based versions. Faster execution speeds and less power consumption were possible thanks to the FPGAs' built-in parallel processing abilities and effective resource utilization. The FPGA performed better due to the usage of Look Up Tables (LUTs) for calculations and the avoidance of string manipulations. The cloud-based version displayed great performance in terms of execution time for larger readings by leveraging a virtual machine with many CPUs. Cloud computing was a desirable alternative because of its scalability and affordability, especially for computationally demanding workloads. Overall, the study emphasized how crucial it is to choose the best platform for a given set of computational requirements. For smaller datasets and when real-time contact is necessary, CPUs are still a dependable option. Applications that require parallel processing and hardware-level improvements benefit greatly from the use of FPGAs. Scalability, affordability, and effective resource management are all features of cloud-based solutions.

In conclusion, this study provides valuable insights into the performance and capabilities of the Sneaky Snake algorithm across different platforms. The findings can guide researchers and practitioners in selecting the most suitable implementation approach based on their specific requirements and computational needs.

9.4 Future Works

Further improvements could be made by exploring advanced communication protocols for FPGA-based implementations, such as Ethernet-based or PCI-based connections, to enable seamless interaction with physical devices. Additionally, investigating alternative techniques for handling string copying operations in CPU implementations could help overcome the performance limitations.

The cloud-based implementation is simply the tip of the iceberg. Microsoft Azure offers many other services such as Functions and Serverless Computing which can leverage the cloud infrastructure to further develop a more optimized and cost-effective solution. The algorithm can be inculcated into an app that can allow users to simply upload DNA data and get their results through a web interface without the need of any technical expertise.

In the realm of genome sequencing and the Sneaky Snake algorithm, future work includes exploring advanced communication protocols for FPGA-based implementations and investigating alternative techniques for handling string copying operations in CPU implementations. Additionally, optimizing cloud-based solutions, integrating the algorithm into user-friendly apps, and leveraging hardware accelerators and advanced data analysis techniques hold promise. Distributed computing frameworks like Apache Hadoop and Apache Spark also offer opportunities for scalability and efficiency. These avenues of research aim to enhance the speed, accuracy, and accessibility of genomic analysis, ultimately advancing our understanding of genetics and human health.

References

[1] C. Song, "Analysis on heterogeneous computing," *J. Phys. Conf. Ser.*, vol. 2031, p. 012049, Sept. 2021.

[2] J. B. Krier, S. S. Kalia, and R. C. Green, "Genomic sequencing in clinical practice: applications, challenges, and opportunities," *Dialogues Clin. Neurosci.*, vol. 18, pp. 299–312, Sept. 2016.

[3] J. D. Watson and F. H. Crick, "Molecular structure of nucleic acids; a structure for deoxyribose nucleic acid," *Nature*, vol. 171, pp. 737–738, Apr. 1953.

[4] M. Alser, J. Rotman, K. Taraszka, H. Shi, P. I. Baykal, H. T. Yang, V. Xue, S. Knyazev, B. D. Singer, B. Balliu, D. Koslicki, P. Skums, A. Zelikovsky, C. Alkan, O. Mutlu, and S. Mangul, "Technology dictates algorithms: Recent developments in read alignment," 2020.

[5] B. Langmead, C. Trapnell, M. Pop, and S. L. Salzberg, "Ultrafast and memory-efficient alignment of short DNA sequences to the human genome," *Genome Biol.*, vol. 10, p. R25, Mar. 2009.

[6] I. Medina, J. Tárraga, H. Martínez, S. Barrachina, M. I. Castillo, J. Paschall, J. Salavert-Torres, I. Blanquer-Espert, V. Hernández-García, E. S. Quintana-Ortí, and J. Dopazo, "Highly sensitive and ultrafast read mapping for RNA-seq analysis," *DNA Res.*, vol. 23, pp. 93–100, Apr. 2016.

[7] N. Malhis and S. J. M. Jones, "High quality SNP calling using illumina data at shallow coverage," *Bioinformatics*, vol. 26, pp. 1029–1035, Apr. 2010.

[8] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic local alignment search tool," *J. Mol. Biol.*, vol. 215, pp. 403–410, Oct. 1990.

[9] S. F. Altschul, T. L. Madden, A. A. Schäffer, J. Zhang, Z. Zhang, W. Miller, and D. J. Lipman, "Gapped BLAST and PSI-BLAST: a new generation of protein database search programs," *Nucleic Acids Res.*, vol. 25, pp. 3389–3402, Sept. 1997.

[10] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," *J. Mol. Biol.*, vol. 147, pp. 195–197, Mar. 1981.

[11] R. E. Bellman, *The Theory of Dynamic Programming.* Santa Monica, CA: RAND Corporation, 1954.

[12] A. Backurs and P. Indyk, "Edit distance cannot be computed in strongly subquadratic time (unless SETH is false)," 2014.

[13] H. Li, "Minimap2: pairwise alignment for nucleotide sequences," *Bioinformatics*, vol. 34, pp. 3094–3100, Sept. 2018.

[14] W. Huangfu, S. Li, X. Hu, and Y. Xie, "RADAR: A 3D-ReRAM based DNA alignment accelerator architecture," in *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*, IEEE, June 2018.

[15] W. J. Masek and M. S. Paterson, "A faster algorithm computing string edit distances," *J. Comput. Syst. Sci.*, vol. 20, pp. 18–31, Feb. 1980.

[16] E. Ukkonen, "Algorithms for approximate string matching," *Inf. Contr.*, vol. 64, pp. 100–118, Jan. 1985.

[17] J. Daily, "Parasail: SIMD C library for global, semi-global, and local pairwise sequence alignments," *BMC Bioinformatics*, vol. 17, p. 81, Feb. 2016.

[18] N. Ahmed, J. Lévy, S. Ren, H. Mushtaq, K. Bertels, and Z. Al-Ars, "GASAL2: a GPU accelerated sequence alignment library for high-throughput NGS data," *BMC Bioinformatics*, vol. 20, p. 520, Oct. 2019.

[19] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals.," *Soviet Physics Doklady*, vol. 10, pp. 707–710, feb 1966. Doklady Akademii Nauk SSSR, V163 No4 845-848 1965.

[20] E. W. Myers, "A fast bit-vector algorithm for approximate string matching based on dynamic programming," in *JACM*, 1998.

[21] J. S. Kim, D. Senol Cali, H. Xin, D. Lee, S. Ghose, M. Alser, H. Hassan, O. Ergin, C. Alkan, and O. Mutlu, "GRIM-Filter: Fast seed location filtering in DNA read mapping using processing-in-memory technologies," *BMC Genomics*, vol. 19, p. 89, May 2018.

[22] M. Alser, H. Hassan, A. Kumar, O. Mutlu, and C. Alkan, "Shouji: a fast and efficient pre-alignment filter for sequence alignment," *Bioinformatics*, vol. 35, pp. 4255–4263, Nov. 2019.

[23] H. Xin, J. Greth, J. Emmons, G. Pekhimenko, C. Kingsford, C. Alkan, and O. Mutlu, "Shifted hamming distance: a fast and accurate SIMD-friendly filter to accelerate alignment verification in read mapping," *Bioinformatics*, vol. 31, pp. 1553–1560, May 2015.

[24] M. Alser, H. Hassan, H. Xin, O. Ergin, O. Mutlu, and C. Alkan, "GateKeeper: a new hardware architecture for accelerating pre-alignment in DNA short read mapping," *Bioinformatics*, vol. 33, pp. 3355–3363, Nov. 2017.

[25] M. Alser, O. Mutlu, and C. Alkan, "MAGNET: Understanding and improving the accuracy of genome pre-alignment filtering," 2017.

[26] M. Alser, T. Shahroodi, J. Gomez-Luna, C. Alkan, and O. Mutlu, "SneakySnake: A fast and accurate universal genome Pre-Alignment filter for CPUs, GPUs, and FPGAs," 2019.

[27] M. Alser, Z. Bingol, D. S. Cali, J. Kim, S. Ghose, C. Alkan, and O. Mutlu, "Accelerating genome analysis: A primer on an ongoing journey," *IEEE Micro*, vol. 40, pp. 65–75, Sept. 2020.

[28] J. S. Kim, D. Senol Cali, H. Xin, D. Lee, S. Ghose, M. Alser, H. Hassan, O. Ergin, C. Alkan, and O. Mutlu, "GRIM-Filter: Fast seed location filtering in DNA read mapping using processing-in-memory technologies," *BMC Genomics*, vol. 19, p. 89, May 2018.

[29] S. Zhao, K. Prenger, and L. Smith, "Stormbow: A cloud-based tool for reads mapping and expression quantification in large-scale RNA-Seq studies," *ISRN Bioinform.*, vol. 2013, p. 481545, Sept. 2013.

[30] K. J. Karczewski, G. H. Fernald, A. R. Martin, M. Snyder, N. P. Tatonetti, and J. T. Dudley, "STORMSeq: an open-source, user-friendly pipeline for processing personal genomics data in the cloud," *PLoS One*, vol. 9, p. e84860, Jan. 2014.

[31] C. Wang, X. Li, P. Chen, A. Wang, X. Zhou, and H. Yu, "Heterogeneous cloud framework for big data genome sequencing," *IEEE/ACM Trans. Comput. Biol. Bioinform.*, vol. 12, pp. 166–178, Jan. 2015.

[32] A. K. Talukder, S. Gandham, H. A. Prahalad, and N. P. Bhattacharyya, "Cloud-maq: The cloud-enabled scalable whole genome reference assembly application," in *2010 Seventh International Conference on Wireless and Optical Communications Networks - (WOCN)*, pp. 1–5, Sep. 2010.

[33] M. J. Flynn, "Very high-speed computing systems," *Proc. IEEE Inst. Electr. Electron. Eng.*, vol. 54, no. 12, pp. 1901–1909, 1966.

[34] J. E. Stone, D. Gohara, and G. Shi, "OpenCL: A parallel programming standard for heterogeneous computing systems," *Comput. Sci. Eng.*, vol. 12, pp. 66–72, May 2010.

[35] T. Dillon, C. Wu, and E. Chang, "Cloud computing: Issues and challenges," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications*, IEEE, 2010.

[36] "Cloud deployment models explaining and comparing," 2022.

[37] G. Dimitrakopoulos, K. Galanopoulos, C. Mavrokefalidis, and D. Nikolos, "Low-power leading-zero counting and anticipation logic for high-speed floating point units," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 16, no. 7, p. 837–850, 2008.